

《星渊旅人》 Qt Creator 完整开发指南

部分内容由豆包生成

版本：v1.0

适用Qt版本：Qt 6.10.1

适用IDE：Qt Creator 13.0

开发语言：C++17

构建系统：CMake

目录

- 第一部分：项目概览与架构设计
 - 1.1 技术选型说明
 - 1.2 整体架构设计
 - 1.3 目录结构规划
 - 1.4 开发环境准备
- 第二部分：第一阶段开发 - 项目搭建与基础框架
 - 2.1 第一步：创建Qt项目
 - 2.2 第二步：核心框架 - 插件管理器
 - 2.3 第三步：核心框架 - 事件总线
 - 2.4 第四步：核心框架 - 资源管理器
 - 2.5 第五步：基础服务 - 渲染服务
 - 2.6 第六步：基础服务 - 输入服务
 - 2.7 第七步：基础服务 - 音频服务
 - 2.8 第八步：基础服务 - 状态机服务
 - 2.9 第九步：基础服务 - 存档服务
 - 2.10 第十步：主菜单UI实现
 - 2.11 第十一步：游戏主循环框架
 - 2.12 第一阶段总结与验证
- 第三部分：第二阶段开发 - 核心玩法系统

- 3.1 玩家角色系统
- 3.2 星核共鸣系统
- 3.3 时间褶皱系统
- 3.4 生态共生系统
- 3.5 战斗系统
- 3.6 地图与关卡系统
- 3.7 UI系统
- 第四部分：第三阶段开发 - 内容填充与优化
 - 4.1 层级内容开发流程
 - 4.2 剧情与任务系统
 - 4.3 性能优化
 - 4.4 Bug修复与测试
- 第五部分：第四阶段开发 - 发布与后续
 - 5.1 打包与部署
 - 5.2 MOD支持
 - 5.3 DLC规划
- 第六部分：Qt Creator工具链深度使用指南
 - 6.1 开发调试工具
 - 6.2 项目管理工具
 - 6.3 UI设计工具
 - 6.4 性能优化工具
 - 6.5 部署工具

第一部分：项目概览与架构设计

1.1 技术选型说明

为什么选择Qt?

特性	说明	对游戏开发的价值
----	----	----------

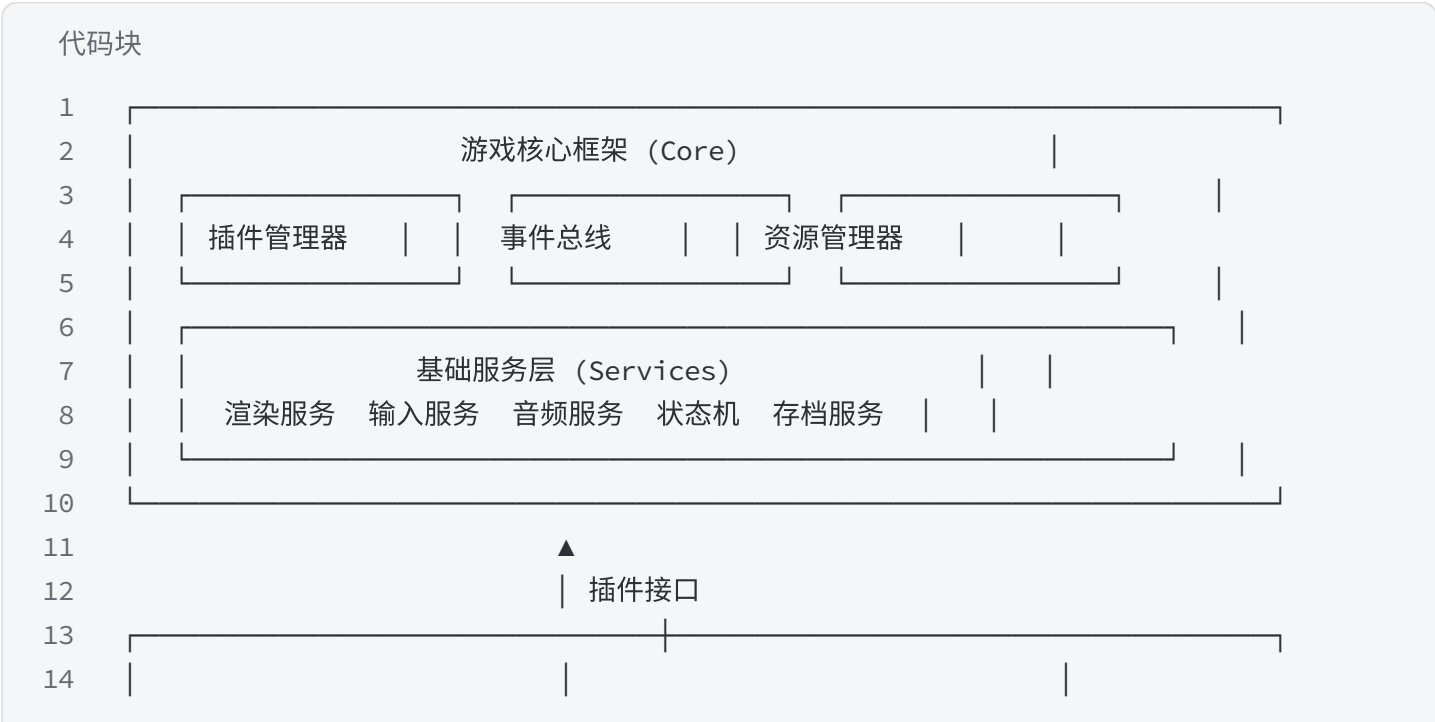
Graphics View框架	成熟的2D渲染引擎，自带BSP空间索引、Z轴排序、视口裁剪	完美适配箱庭探索游戏，不用自研渲染引擎
信号与槽机制	松耦合的事件系统	插件间通信、状态切换、UI交互都靠它
QSS样式表	类似CSS的样式系统	UI主题切换、皮肤系统零成本实现
属性动画框架	QPropertyAnimation	角色动画、UI动效、场景过渡统一用这个
状态机框架	QStateMachine	游戏状态管理可视化，调试方便
跨平台支持	一套代码编译多平台	Windows/macOS/Linux一键打包
完整工具链	Qt Creator一站式IDE	编码、调试、性能分析、部署全在一个IDE里

为什么不选Unity/Godot？

- 1. 单人开发控制力：纯代码开发，每个像素都可控，没有黑箱
- 2. 插件化架构天然契合：Qt Plugin机制就是为模块化设计的
- 3. 学习成本可控：C++是通用技能，不用学特定引擎的脚本语言
- 4. 性能有保障：原生C++性能比脚本语言高很多
- 5. MOD友好：基于Qt Plugin的MOD生态比引擎MOD更强大

1.2 整体架构设计

纯插件化架构图




```

14 | | | └─ ResourceManager.h    # 资源管理器
15 | | | └─ ResourceManager.cpp
16 | | | └─ interfaces/        # 插件接口定义
17 | | | |   └─ IPlugin.h
18 | | | |   └─ IGameplayPlugin.h
19 | | | |   └─ IContentPlugin.h
20 | | | |   └─ IUIPlugin.h
21 | | | └─ services/          # 基础服务
22 | | | |   └─ IRenderService.h
23 | | | |   └─ IInputService.h
24 | | | |   └─ IAudioService.h
25 | | | |   └─ IStateMachineService.h
26 | | | |   └─ ISaveService.h
27 | | └─ services/            # 服务实现
28 | | |   └─ CMakeLists.txt
29 | | |   └─ render/          # 渲染服务实现
30 | | |   └─ input/           # 输入服务实现
31 | | |   └─ audio/           # 音频服务实现
32 | | |   └─ statemachine/    # 状态机服务实现
33 | | |   └─ save/            # 存档服务实现
34 | | └─ plugins/             # 官方插件
35 | | |   └─ CMakeLists.txt
36 | | |   └─ gameplay/        # 核心玩法插件
37 | | | |   └─ timefold/      # 时间褶皱系统
38 | | | |   └─ starcore/      # 星核共鸣系统
39 | | | |   └─ symbiosis/     # 生态共生系统
40 | | |   └─ content/         # 游戏内容插件
41 | | | |   └─ layer1-light/   # 第1层光明层
42 | | | |   └─ layer2-nature/  # 第2层自然层
43 | | | |   └─ ...
44 | | |   └─ ui/              # UI界面插件
45 | | | |   └─ mainmenu/      # 主菜单
46 | | | |   └─ hud/           # 游戏HUD
47 | | | |   └─ inventory/     # 背包界面
48 | | └─ app/                 # 应用入口
49 | | |   └─ CMakeLists.txt
50 | | |   └─ main.cpp
51 | | |   └─ GameApplication.h
52 | | |   └─ GameApplication.cpp
53 | └─ plugins/                # 编译后的插件输出目录
54 | └─ resources/              # 游戏资源
55 | |   └─ images/             # 图片资源
56 | |   └─ sounds/             # 音效
57 | |   └─ music/              # 音乐
58 | |   └─ fonts/              # 字体
59 | |   └─ styles/             # QSS样式
60 | |   └─ data/               # 数据配置 (JSON)

```

```
61  └─ tests/           # 单元测试
62  └─ tools/           # 开发工具脚本
```

1.4 开发环境准备

1.4 开发环境准备

必备软件清单

软件	版本	用途
Qt	6.10.1	核心框架
Qt Creator	13.0	IDE
CMake	3.26+	构建系统（Qt自带）
编译器	MSVC 2022 / Clang / GCC 11+	C++编译
Git	最新版	版本控制

安装步骤（Windows）

代码块

```
1  步骤1：下载Qt在线安装器
2      地址：https://www.qt.io/download-qt-installer
3
4  步骤2：运行安装器，选择"自定义安装"
5
6  步骤3：选择Qt 6.10.1组件（必选）：
7      ☒ MSVC 2022 64-bit
8      ☒ MinGW 11.2.0 64-bit（备选）
9      ☒ Qt 5 Compatibility Module（可选，兼容旧代码）
10     ☒ Qt Shader Tools
11     ☒ Qt Network Authorization
12
13 步骤4：选择开发工具（必选）：
14     ☒ Qt Creator 13.0.0
15     ☒ CMake 3.26.4
16     ☒ Ninja 1.11.1
17
18 步骤5：选择附加库（按需）：
19     ☐ Qt Multimedia（我们用核心模块自带的）
```

```
20      ☐ Qt State Machine (我们用核心模块自带的)
21
22  步骤6: 等待安装完成 (约5-10GB, 取决于选择的组件)
23
24  步骤7: 验证安装
25      打开Qt Creator → 工具 → 选项 → 套件
26      确认有"Desktop Qt 6.10.1 MSVC2022 64bit"套件
```

配置Git (Qt Creator内置)

代码块

```
1  步骤1: 工具 → 选项 → 版本控制 → Git
2  步骤2: 设置Git路径 (通常自动检测到)
3  步骤3: 配置用户信息:
4      用户名: 你的名字
5      邮箱: 你的邮箱
6  步骤4: 勾选"显示工具提示"和"在提交对话框中显示变更说明"
```

第二部分：第一阶段开发 - 项目搭建与基础框架（第1-2个月）

目标：搭建完整的插件化框架，实现基础服务，能跑通主菜单→进入游戏→退出的完整流程

预计耗时：约2个月（单人开发）

验收标准：主菜单正常显示，点击"开始游戏"进入空白游戏场景，能正常退出

2.1 第一步：创建Qt项目

详细操作步骤

代码块

```
1  步骤1: 打开Qt Creator
2      欢迎页面 → 新建项目
3
4  步骤2: 选择项目模板
5      选择"应用程序" → "Qt Widgets Application"
6      点击"Choose..."
7
8  步骤3: 项目介绍和位置
9      名称: StarAbyss
10     创建路径: D:/Projects/StarAbyss (自己选个路径)
11     构建系统: CMake
```

```
12         点击"下一步"
13
14     步骤4: 定义构建系统
15         类名: GameApplication
16         基类: QMainWindow (后面我们会改成自己的)
17         头文件: gameapplication.h
18         源文件: gameapplication.cpp
19         界面文件: 不勾选 (我们不用Designer做主窗口)
20         点击"下一步"
21
22     步骤5: 翻译文件
23         直接点击"下一步" (后面再加翻译)
24
25     步骤6: 套件选择
26         勾选"Desktop Qt 6.10.1 MSVC2022 64bit"
27         点击"下一步"
28
29     步骤7: 项目管理
30         版本控制: Git (如果已安装)
31         点击"完成"
```

初始项目结构

创建完成后，你会看到以下文件：

代码块

```
1  StarAbyss/
2  |—— CMakeLists.txt
3  |—— gameapplication.h
4  |—— gameapplication.cpp
5  |—— main.cpp
6  |—— StarAbyss.pro.user (Qt Creator配置文件)
```

修改CMakeLists.txt

让我们先把CMake配置改好，为插件化架构做准备。

文件：CMakeLists.txt

代码块

```
1  # CMake最低版本要求
2  cmake_minimum_required(VERSION 3.26)
3
4  # 项目名称和版本
5  project(StarAbyss VERSION 0.1.0 LANGUAGES CXX)
```

```
6
7  # C++标准设置
8  set(CMAKE_CXX_STANDARD 17)
9  set(CMAKE_CXX_STANDARD_REQUIRED ON)
10 set(CMAKE_CXX_EXTENSIONS OFF)
11
12  # 开启Qt的MOC、UIC、RCC自动处理
13  set(CMAKE_AUTOMOC ON)
14  set(CMAKE_AUTOUIC ON)
15  set(CMAKE_AUTORCC ON)
16
17  # 查找Qt6模块
18  find_package(Qt6 6.10 REQUIRED COMPONENTS
19      Core
20      Gui
21      Widgets
22      Multimedia
23      StateMachine
24  )
25
26  # 输出目录设置
27  set(CMAKE_RUNTIME_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/bin)
28  set(CMAKE_LIBRARY_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/plugins)
29  set(CMAKE_ARCHIVE_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/lib)
30
31  # 全局包含目录
32  include_directories(${CMAKE_SOURCE_DIR}/src)
33
34  # 子目录
35  add_subdirectory(src/core)          # 核心框架
36  add_subdirectory(src/services)      # 服务实现
37  add_subdirectory(src/plugins)       # 官方插件
38  add_subdirectory(src/app)           # 应用入口
39
40  # 资源文件
41  qt_add_resources(RESOURCES resources/resources.qrc)
42
43  # 打印配置信息
44  message(STATUS "=====")
45  message(STATUS "项目名称: ${PROJECT_NAME}")
46  message(STATUS "项目版本: ${PROJECT_VERSION}")
47  message(STATUS "Qt版本: ${Qt6_VERSION}")
48  message(STATUS "C++标准: C++${CMAKE_CXX_STANDARD}")
49  message(STATUS "构建类型: ${CMAKE_BUILD_TYPE}")
50  message(STATUS "=====")
```

创建目录结构

在Qt Creator中，右键项目 → 新建文件或文件夹，创建以下目录结构：

代码块

```
1  src/
2  |— core/
3  |   |— interfaces/
4  |   |— services/
5  |— services/
6  |   |— render/
7  |   |— input/
8  |   |— audio/
9  |   |— statemachine/
10 |   |— save/
11 |— plugins/
12 |   |— gameplay/
13 |   |— content/
14 |   |— ui/
15 |— app/
16
17 resources/
18 |— images/
19 |— sounds/
20 |— styles/
21 |— data/
```

Qt Creator小技巧：在项目视图中，右键→"新建文件或文件夹"→选择"文件夹"，输入路径即可创建。也可以直接在资源管理器里创建，然后在Qt Creator中右键项目→"刷新"。

2.2 第二步：核心框架 - 插件管理器

插件管理器是整个架构的核心，负责加载、卸载、管理所有插件。

2.2.1 定义插件基础接口

首先创建所有插件的基接口 `IPlugin`。

文件：src/core/interfaces/IPlugin.h

代码块

```
1  #ifndef IPLUGIN_H
2  #define IPLUGIN_H
3
4  #include <QtPlugin>
```

```
5  #include <QString>
6  #include <QVariantMap>
7
8  // 前向声明
9  class PluginManager;
10
11  /**
12   * @brief 插件基础接口
13   *
14   * 所有插件都必须实现这个接口。
15   * 插件管理器通过这个接口来加载、初始化、卸载插件。
16   *
17   * 设计原则：
18   * - 接口尽量简单，只包含必要的生命周期方法
19   * - 插件之间通过事件总线通信，不直接依赖
20   * - 每个插件有唯一的ID，用于识别和查找
21   */
22  class IPlugin
23  {
24  public:
25      // 虚析构函数，确保正确释放资源
26      virtual ~IPlugin() = default;
27
28      /**
29       * @brief 获取插件ID
30       * @return 插件的唯一标识符（格式："com.company.pluginname"）
31       *
32       * 每个插件必须有唯一的ID，用于在插件管理器中查找。
33       * 建议使用反向域名格式，避免冲突。
34       */
35      virtual QString pluginId() const = 0;
36
37      /**
38       * @brief 获取插件名称
39       * @return 插件的显示名称
40       *
41       * 用于在UI中显示给用户看的名字，比如"时间褶皱系统"。
42       */
43      virtual QString pluginName() const = 0;
44
45      /**
46       * @brief 获取插件版本
47       * @return 版本号字符串（格式："1.0.0"）
48       */
49      virtual QString pluginVersion() const = 0;
50
51      /**
```

```

52     * @brief 获取插件作者
53     * @return 作者名称
54     */
55     virtual QString pluginAuthor() const = 0;
56
57     /**
58     * @brief 获取插件描述
59     * @return 插件功能描述
60     */
61     virtual QString pluginDescription() const = 0;
62
63     /**
64     * @brief 获取依赖的其他插件ID列表
65     * @return 依赖的插件ID列表
66     *
67     * 插件管理器会确保依赖的插件先加载。
68     * 如果依赖不存在，本插件不会被加载。
69     */
70     virtual QStringList dependencies() const { return {}; }
71
72     /**
73     * @brief 初始化插件
74     * @param manager 插件管理器指针
75     * @param config 插件配置
76     * @return 初始化是否成功
77     *
78     * 在所有插件加载完成后调用。
79     * 插件可以在这里：
80     * - 注册事件监听器
81     * - 注册服务
82     * - 加载资源
83     * - 初始化内部状态
84     *
85     * 注意：不要在这里做耗时操作，会影响启动速度。
86     * 耗时的初始化应该延后到真正需要时再做（懒加载）。
87     */
88     virtual bool initialize(PluginManager* manager, const QVariantMap& config
= {}) = 0;
89
90     /**
91     * @brief 关闭插件
92     *
93     * 游戏退出时调用，插件应该在这里释放所有资源。
94     * 包括：
95     * - 断开信号槽连接
96     * - 释放内存
97     * - 保存状态

```

```

98      * - 注销服务
99      */
100     virtual void shutdown() = 0;
101
102     /**
103      * @brief 插件是否已初始化
104      * @return 是否已初始化
105      */
106     virtual bool isInitialized() const = 0;
107
108     protected:
109         // 插件管理器指针，子类可以通过这个访问其他插件和服务
110         PluginManager* m_pluginManager = nullptr;
111
112         // 插件是否已初始化
113         bool m_initialized = false;
114     };
115
116     // 声明插件接口（Qt插件系统需要）
117     #define IPlugin_iid "com.starabyss.IPlugin"
118     Q_DECLARE_INTERFACE(IPlugin, IPlugin_iid)
119
120     #endif // IPLUGIN_H

```

2.2.2 实现插件管理器

文件：src/core/PluginManager.h

代码块

```

1  #ifndef PLUGINMANAGER_H
2  #define PLUGINMANAGER_H
3
4  #include <QObject>
5  #include <QDir>
6  #include <QList>
7  #include <QMap>
8  #include <QPluginLoader>
9  #include <QVariantMap>
10
11  #include "interfaces/IPlugin.h"
12
13  /**
14   * @brief 插件管理器
15   *
16   * 负责扫描、加载、初始化、管理所有插件。
17   * 这是整个插件化架构的核心。

```

```
18  *
19  * 功能：
20  * 1. 扫描插件目录，发现可用插件
21  * 2. 按依赖顺序加载插件
22  * 3. 初始化所有插件
23  * 4. 提供插件查找接口
24  * 5. 管理插件生命周期
25  *
26  * 使用方式：
27  * - 单例模式，通过 PluginManager::instance() 获取实例
28  * - 游戏启动时调用 loadPlugins() 加载所有插件
29  * - 游戏退出时调用 shutdownAll() 关闭所有插件
30  */
31 class PluginManager : public QObject
32 {
33     Q_OBJECT
34
35 public:
36     // 获取单例实例
37     static PluginManager* instance();
38
39     // 禁止拷贝和赋值
40     PluginManager(const PluginManager&) = delete;
41     PluginManager& operator=(const PluginManager&) = delete;
42
43     /**
44      * @brief 设置插件搜索目录
45      * @param path 插件目录路径
46      *
47      * 插件管理器会在这个目录下搜索所有的插件库文件。
48      * 默认是应用程序目录下的 plugins/ 文件夹。
49      */
50     void setPluginDirectory(const QString& path);
51
52     /**
53      * @brief 加载所有插件
54      * @return 成功加载的插件数量
55      *
56      * 加载流程：
57      * 1. 扫描插件目录，发现所有插件文件
58      * 2. 尝试加载每个插件，读取元数据
59      * 3. 根据依赖关系排序
60      * 4. 按顺序初始化插件
61      *
62      * 注意：如果某个插件加载失败，会记录警告，但不会中断其他插件的加载。
63      */
64     int loadPlugins();
```

```

65
66     /**
67      * @brief 关闭所有插件
68      *
69      * 按加载的逆序关闭所有插件，确保依赖的插件后关闭。
70      */
71     void shutdownAll();
72
73     /**
74      * @brief 根据ID获取插件
75      * @param pluginId 插件ID
76      * @return 插件指针，找不到返回nullptr
77      */
78     IPlugin* getPlugin(const QString& pluginId) const;
79
80     /**
81      * @brief 获取所有已加载的插件
82      * @return 插件列表
83      */
84     QList<IPlugin*> getAllPlugins() const;
85
86     /**
87      * @brief 获取指定类型的插件
88      *
89      * 模板方法，获取实现了特定接口的插件。
90      * 例如：getPluginsOfType<IGameplayPlugin>()
91      */
92     template<typename T>
93     QList<T*> getPluginsOfType() const
94     {
95         QList<T*> result;
96         for (IPlugin* plugin : m_plugins) {
97             T* typed = qobject_cast<T*>(plugin);
98             if (typed) {
99                 result.append(typed);
100             }
101         }
102         return result;
103     }
104
105     /**
106      * @brief 检查插件是否已加载
107      * @param pluginId 插件ID
108      * @return 是否已加载
109      */
110     bool isPluginLoaded(const QString& pluginId) const;
111

```

```

112     /**
113      * @brief 获取插件加载错误信息
114      * @return 错误信息列表
115      */
116     QStringList getErrors() const { return m_errors; }
117
118     signals:
119     /**
120      * @brief 插件加载完成信号
121      * @param pluginId 插件ID
122      *
123      * 当某个插件成功加载并初始化后发出。
124      * 其他插件可以监听这个信号来做响应。
125      */
126     void pluginLoaded(const QString& pluginId);
127
128     /**
129      * @brief 所有插件加载完成信号
130      *
131      * 当所有插件都加载完成后发出。
132      */
133     void allPluginsLoaded();
134
135     private:
136     // 私有构造函数（单例模式）
137     explicit PluginManager(QObject *parent = nullptr);
138     ~PluginManager();
139
140     // 单例实例
141     static PluginManager* s_instance;
142
143     // 插件目录
144     QDir m_pluginDir;
145
146     // 已加载的插件（按加载顺序）
147     QList<IPlugin*> m_plugins;
148
149     // 插件ID到插件的映射
150     QMap<QString, IPlugin*> m_pluginMap;
151
152     // 插件加载器列表（保持QPluginLoader对象，防止插件被卸载）
153     QList<QPluginLoader*> m_loaders;
154
155     // 错误信息
156     QStringList m_errors;
157
158     /**

```

```

159     * @brief 扫描插件目录
160     * @return 找到的插件文件路径列表
161     */
162     QStringList scanPluginDirectory();
163
164     /**
165     * @brief 加载单个插件
166     * @param filePath 插件文件路径
167     * @return 插件指针，失败返回nullptr
168     */
169     IPlugin* loadSinglePlugin(const QString& filePath);
170
171     /**
172     * @brief 按依赖关系排序插件
173     * @param plugins 待排序的插件列表
174     * @return 排序后的插件列表
175     *
176     * 确保被依赖的插件排在前面。
177     * 如果有循环依赖，会记录错误并跳过有问题的插件。
178     */
179     QList<IPlugin*> sortByDependencies(QList<IPlugin*> plugins);
180
181     /**
182     * @brief 检查依赖是否满足
183     * @param plugin 要检查的插件
184     * @param loadedPlugins 已加载的插件
185     * @return 依赖是否满足
186     */
187     bool checkDependencies(IPlugin* plugin, const QList<IPlugin*>&
        loadedPlugins);
188 };
189
190 #endif // PLUGINMANAGER_H

```

文件：src/core/PluginManager.cpp

代码块

```

1  #include "PluginManager.h"
2
3  #include <QCoreApplication>
4  #include <QDebug>
5  #include <QLibrary>
6
7  // 初始化静态单例
8  PluginManager* PluginManager::s_instance = nullptr;

```

```
9
10 PluginManager* PluginManager::instance()
11 {
12     if (!s_instance) {
13         s_instance = new PluginManager();
14     }
15     return s_instance;
16 }
17
18 PluginManager::PluginManager(QObject *parent)
19     : QObject(parent)
20 {
21     // 默认插件目录：应用程序目录下的plugins文件夹
22     QString appDir = QApplication::applicationDirPath();
23     m_pluginDir.setPath(appDir + "/plugins");
24
25     qDebug() << "[PluginManager] 插件管理器初始化完成";
26     qDebug() << "[PluginManager] 默认插件目录:" << m_pluginDir.absolutePath();
27 }
28
29 PluginManager::~~PluginManager()
30 {
31     shutdownAll();
32     qDebug() << "[PluginManager] 插件管理器已销毁";
33 }
34
35 void PluginManager::setPluginDirectory(const QString& path)
36 {
37     m_pluginDir.setPath(path);
38     qDebug() << "[PluginManager] 插件目录设置为:" << m_pluginDir.absolutePath();
39 }
40
41 int PluginManager::loadPlugins()
42 {
43     qDebug() << "[PluginManager] 开始加载插件...";
44
45     // 清空之前的错误
46     m_errors.clear();
47
48     // 步骤1：扫描插件目录
49     QStringList pluginFiles = scanPluginDirectory();
50     qDebug() << "[PluginManager] 找到" << pluginFiles.size() << "个插件文件";
51
52     // 步骤2：加载所有插件
53     QList<IPlugin*> loadedPlugins;
54     for (const QString& file : pluginFiles) {
55         IPlugin* plugin = loadSinglePlugin(file);
```

```

56         if (plugin) {
57             loadedPlugins.append(plugin);
58         }
59     }
60     qDebug() << "[PluginManager] 成功加载" << loadedPlugins.size() << "个插件";
61
62     // 步骤3: 按依赖关系排序
63     QList<IPlugin*> sortedPlugins = sortByDependencies(loadedPlugins);
64     qDebug() << "[PluginManager] 依赖排序完成";
65
66     // 步骤4: 按顺序初始化插件
67     int successCount = 0;

```

代码块

```

1  int successCount = 0;
2  for (IPlugin* plugin : sortedPlugins) {
3      if (plugin->initialize(this)) {
4          m_plugins.append(plugin);
5          m_pluginMap[plugin->pluginId()] = plugin;
6          successCount++;
7
8          qDebug() << "[PluginManager] ✓ 插件初始化成功:"
9                  << plugin->pluginName()
10                 << "(" << plugin->pluginId() << ")";
11
12         emit pluginLoaded(plugin->pluginId());
13     } else {
14         m_errors.append(QString("插件初始化失败: %1").arg(plugin->pluginId()));
15         qWarning() << "[PluginManager] ✗ 插件初始化失败:" << plugin->pluginId();
16     }
17 }
18
19 qDebug() << "[PluginManager] 插件加载完成, 成功" << successCount << "个";
20
21 emit allPluginsLoaded();
22
23 return successCount;

```

```

}

```

```

void PluginManager::shutdownAll()

```

```

{
    qDebug() << "[PluginManager] 开始关闭所有插件...";

```

```

1 // 按加载的逆序关闭
2 for (int i = m_plugins.size() - 1; i >= 0; --i) {
3     IPlugin* plugin = m_plugins[i];
4     if (plugin && plugin->isInitialized()) {
5         plugin->shutdown();
6         qDebug() << "[PluginManager] 插件已关闭:" << plugin->pluginName();
7     }
8 }
9
10 // 清理插件加载器（会卸载插件库）
11 qDeleteAll(m_loaders);
12 m_loaders.clear();
13
14 m_plugins.clear();
15 m_pluginMap.clear();
16
17 qDebug() << "[PluginManager] 所有插件已关闭";

```

```

}

```

```

IPlugin* PluginManager::getPlugin(const QString& pluginId) const

```

```

{
return m_pluginMap.value(pluginId, nullptr);
}

```

```

QList<IPlugin*> PluginManager::getAllPlugins() const

```

```

{
return m_plugins;
}

```

```

bool PluginManager::isPluginLoaded(const QString& pluginId) const

```

```

{
return m_pluginMap.contains(pluginId);
}

```

```

QStringList PluginManager::scanPluginDirectory()

```

```

{
QStringList result;

```

代码块

```

1 if (!m_pluginDir.exists()) {
2     qWarning() << "[PluginManager] 插件目录不存在:" <<
    m_pluginDir.absolutePath();
3     return result;
4 }

```

```

5
6 // 获取目录下所有的库文件
7 QStringList filters;
8
9 // 根据平台设置不同的库文件后缀

```

```

#ifdef Q_OS_WIN
filters << ".dll";
#elif defined(Q_OS_MAC)
filters << ".dylib" << ".so";
#else
filters << ".so";
#endif

```

代码块

```

1 m_pluginDir.setNameFilters(filters);
2
3 QFileInfoList fileInfoList = m_pluginDir.entryInfoList(QDir::Files);
4 for (const QFileInfo& fileInfo : fileInfoList) {
5     // 跳过调试版本的库（带d后缀的）
6     QString fileName = fileInfo.fileName();
7     if (fileName.endsWith("d.dll") || fileName.endsWith("d.lib")) {
8         continue;
9     }
10
11     result.append(fileInfo.absoluteFilePath());
12 }
13
14 return result;

```

```

}

IPlugin* PluginManager::loadSinglePlugin(const QString& filePath)
{
QPluginLoader* loader = new QPluginLoader(filePath);

```

代码块

```

1 // 尝试加载插件
2 QObject* instance = loader->instance();
3 if (!instance) {
4     m_errors.append(QString("无法加载插件 %1: %2").arg(filePath, loader-
>errorString()));

```

```

5     qWarning() << "[PluginManager] 加载插件失败:" << filePath << loader-
    >errorString();
6     delete loader;
7     return nullptr;
8 }
9
10 // 转换为IPlugin接口
11 IPlugin* plugin = qobject_cast<IPlugin*>(instance);
12 if (!plugin) {
13     m_errors.append(QString("插件 %1 没有实现IPlugin接口").arg(filePath));
14     qWarning() << "[PluginManager] 插件不是有效的IPlugin:" << filePath;
15     delete loader;
16     return nullptr;
17 }
18
19 // 保存loader, 防止插件被卸载
20 m_loaders.append(loader);
21
22 qDebug() << "[PluginManager] 发现插件:" << plugin->pluginName()
23         << "v" << plugin->pluginVersion()
24         << "by" << plugin->pluginAuthor();
25
26 return plugin;

```

```

}

```

```

QList<IPlugin*> PluginManager::sortByDependencies(QList<IPlugin*> plugins)

```

```

{

```

```

    QList<IPlugin*> sorted;

```

```

    QList<IPlugin*> remaining = plugins;

```

代码块

```

1 // 简单的拓扑排序
2 // 循环直到所有插件都排好序
3 int maxIterations = remaining.size() * remaining.size(); // 防止死循环
4 int iteration = 0;
5
6 while (!remaining.isEmpty() && iteration < maxIterations) {
7     iteration++;
8
9     bool foundOne = false;
10
11     for (int i = 0; i < remaining.size(); ++i) {
12         IPlugin* plugin = remaining[i];
13

```

```

14         // 检查这个插件的依赖是否都已经在sorted里了
15         if (checkDependencies(plugin, sorted)) {
16             sorted.append(plugin);
17             remaining.removeAt(i);
18             foundOne = true;
19             break;
20         }
21     }
22
23     // 如果一轮下来一个都没找到，说明有循环依赖
24     if (!foundOne) {
25         qWarning() << "[PluginManager] 检测到循环依赖或缺失依赖，剩余插件:";
26         for (IPlugin* p : remaining) {
27             qWarning() << "  -" << p->pluginId() << "依赖:" << p->
dependencies();
28         }
29         m_errors.append("存在循环依赖或缺失依赖");
30         break;
31     }
32 }
33
34 // 把剩下的插件也加进去（虽然有依赖问题，但尽量加载）
35 for (IPlugin* p : remaining) {
36     sorted.append(p);
37 }
38
39 return sorted;

```

```

}

```

```

bool PluginManager::checkDependencies(IPlugin* plugin, const QList<IPlugin*>& loadedPlugins)
{
    QStringList deps = plugin->dependencies();

```

代码块

```

1     for (const QString& depId : deps) {
2         bool found = false;
3         for (IPlugin* loaded : loadedPlugins) {
4             if (loaded->pluginId() == depId) {
5                 found = true;
6                 break;
7             }
8         }
9         if (!found) {
10             return false; // 有依赖没满足
11         }

```

```
12 }
13
14 return true; // 所有依赖都满足
```

```
}
```

代码块

```
1
2 ##### 2.2.3 core模块的CMake配置
3
4 **文件: src/core/CMakeLists.txt**
5 ```cmake
6 # 核心框架库
7 # 这是一个静态库，所有插件都链接它
8
9 add_library(Core STATIC
10     PluginManager.h
11     PluginManager.cpp
12     EventBus.h
13     EventBus.cpp
14     ResourceManager.h
15     ResourceManager.cpp
16
17     interfaces/IPlugin.h
18     interfaces/IGameplayPlugin.h
19     interfaces/IContentPlugin.h
20     interfaces/IUIPlugin.h
21
22     services/IRenderService.h
23     services/IInputService.h
24     services/IAudioService.h
25     services/IStateMachineService.h
26     services/ISaveService.h
27 )
28
29 # 链接Qt模块
30 target_link_libraries(Core PUBLIC
31     Qt6::Core
32     Qt6::Gui
33     Qt6::Widgets
34 )
35
36 # 包含目录
37 target_include_directories(Core PUBLIC
38     ${CMAKE_CURRENT_SOURCE_DIR}
```

```

39     ${CMAKE_CURRENT_SOURCE_DIR}/interfaces
40     ${CMAKE_CURRENT_SOURCE_DIR}/services
41 )
42
43 # 设置目标属性
44 set_target_properties(Core PROPERTIES
45     OUTPUT_NAME "starabyss-core"
46 )
47
48 message(STATUS "Core模块配置完成")

```

注意：上面的CMakeLists.txt里引用了EventBus、ResourceManager等文件，我们会在后面的步骤中创建。现在先把框架搭起来。

2.3 第三步：核心框架 - 事件总线

事件总线是插件之间通信的核心机制。插件之间不直接调用，而是通过事件总线发送和接收事件，实现完全解耦。

2.3.1 事件总线实现

文件：src/core/EventBus.h

代码块

```

1  #ifndef EVENTBUS_H
2  #define EVENTBUS_H
3
4  #include <QObject>
5  #include <QString>
6  #include <QVariantMap>
7  #include <QList>
8  #include <functional>
9
10 /**
11  * @brief 事件结构体
12  *
13  * 所有事件都用这个通用结构表示。
14  * type是事件类型，data是事件数据。
15  *
16  * 为什么用QVariantMap而不是强类型？
17  * - 灵活性高，不用为每种事件定义一个类
18  * - 插件可以自定义事件，不用修改核心代码
19  * - 序列化方便，便于调试和日志
20  * - 性能对于游戏事件来说完全足够

```

```

21  */
22  struct GameEvent
23  {
24      QString type;           // 事件类型 (如 "player.move"、"enemy.dead")
25      QVariantMap data;       // 事件数据
26      quint64 timestamp = 0;  // 事件时间戳 (毫秒)
27      QString senderId;       // 发送者ID (可选)
28
29      // 便捷构造函数
30      GameEvent() = default;
31
32      GameEvent(const QString& eventType, const QVariantMap& eventData = {})
33          : type(eventType), data(eventData)
34      {
35          // 记录时间戳
36          timestamp = QDateTime::currentMSecsSinceEpoch();
37      }
38  };
39
40  /**
41   * @brief 事件监听器接口
42   *
43   * 可以继承这个类来监听事件,
44   * 也可以用lambda函数的方式 (更灵活)。
45   */
46  class IEventListener
47  {
48  public:
49      virtual ~IEventListener() = default;
50
51      /**
52       * @brief 处理事件
53       * @param event 事件对象
54       *
55       * 当监听的事件发生时调用。
56       * 注意：不要在事件处理函数中做耗时操作,
57       * 否则会阻塞整个事件总线。
58       */
59      virtual void onEvent(const GameEvent& event) = 0;
60  };
61
62  /**
63   * @brief 事件总线
64   *
65   * 发布-订阅模式的事件系统。
66   * 插件之间通过事件总线通信, 实现完全解耦。
67   */

```

```

68  * 使用方式：
69  * 1. 订阅事件：eventBus->subscribe("event.type", this, &MyClass::onEvent);
70  * 2. 发布事件：eventBus->publish(GameEvent("event.type", data));
71  *
72  * 特点：
73  * - 支持按事件类型订阅
74  * - 支持通配符订阅（如 "player.*" 订阅所有player相关事件）
75  * - 支持优先级（高优先级的监听器先收到事件）
76  * - 支持事件取消（高优先级监听器可以阻止事件继续传播）
77  * - 线程安全（核心功能在主线程执行）
78  */
79 class EventBus : public QObject
80 {
81     Q_OBJECT
82
83 public:
84     // 获取单例
85     static EventBus* instance();
86
87     // 禁止拷贝
88     EventBus(const EventBus&) = delete;
89     EventBus& operator=(const EventBus&) = delete;
90
91     /**
92      * @brief 订阅事件
93      * @param eventType 事件类型（支持通配符*）
94      * @param receiver 接收者对象
95      * @param member 成员函数指针
96      * @param priority 优先级（数字越大越先收到，默认0）
97      *
98      * 这是最常用的订阅方式，用Qt的信号槽机制。
99      * 优点：自动管理生命周期，对象销毁后自动断开
100     */
101     template<typename Receiver, typename MemberFunc>
102     void subscribe(const QString& eventType, Receiver* receiver, MemberFunc
member, int priority = 0)
103     {
104         // 这里用Qt的信号槽机制实现
105         // 具体实现见.cpp文件
106         subscribeImpl(eventType, receiver,
107                        std::bind(member, receiver, std::placeholders::_1),
108                        priority);
109     }
110
111     /**
112      * @brief 订阅事件（lambda版本）
113      * @param eventType 事件类型

```

```
114     * @param handler 处理函数
115     * @param priority 优先级
116     * @return 订阅ID, 用于取消订阅
117     *
118     * 用lambda函数订阅, 更灵活。
119     * 但需要手动管理取消订阅。
120     */
121     int subscribe(const QString& eventType,
122                  std::function<void(const GameEvent&)> handler,
123                  int priority = 0);
124
125     /**
126     * @brief 取消订阅
127     * @param subscriptionId 订阅ID
128     */
129     void unsubscribe(int subscriptionId);
130
131     /**
132     * @brief 取消对象的所有订阅
133     * @param receiver 接收者对象
134     */
135     void unsubscribeAll(QObject* receiver);
136
137     /**
138     * @brief 发布事件
139     * @param event 事件对象
140     *
141     * 同步发布事件, 所有监听器处理完后才返回。
142     * 对于大多数游戏事件, 同步处理是没问题的。
143     */
144     void publish(const GameEvent& event);
145
146     /**
147     * @brief 发布事件 (便捷重载)
148     * @param eventType 事件类型
149     * @param data 事件数据
150     */
151     void publish(const QString& eventType, const QVariantMap& data = {})
152     {
153         publish(GameEvent(eventType, data));
154     }
155
156     /**
157     * @brief 异步发布事件
158     * @param event 事件对象
159     *
160     * 把事件放到队列里, 下一帧再处理。
```

```

161     * 用于不紧急的事件，避免阻塞当前逻辑。
162     */
163 void publishAsync(const GameEvent& event);
164
165 /**
166  * @brief 检查是否有订阅者
167  * @param eventType 事件类型
168  * @return 是否有订阅者
169  */
170 bool hasSubscribers(const QString& eventType) const;
171
172 signals:
173     // 内部使用的信号，不要直接连接
174     void eventPublished(const GameEvent& event);
175
176 private:
177     explicit EventBus(QObject* parent = nullptr);
178     ~EventBus();
179
180     static EventBus* s_instance;
181
182     // 订阅信息结构体
183     struct Subscription
184     {
185         int id;
186         QString eventType;
187         std::function<void(const GameEvent&)> handler;
188         int priority;
189         QObject* receiver = nullptr; // 用于自动断开
190     };
191
192     // 所有订阅
193     QList<Subscription> m_subscriptions;
194
195     // 下一个订阅ID
196     int m_nextSubscriptionId = 1;
197
198     // 异步事件队列
199     QList<GameEvent> m_eventQueue;
200
201     // 内部实现
202     void subscribeImpl(const QString& eventType,
203                       QObject* receiver,
204                       std::function<void(const GameEvent&)> handler,
205                       int priority);
206
207     // 检查事件类型是否匹配（支持通配符）

```

```

208     bool matchEventType(const QString& pattern, const QString& eventType)
        const;
209
210     // 处理异步事件队列
211     void processEventQueue();
212 };
213
214 #endif // EVENTBUS_H

```

文件：src/core/EventBus.cpp

代码块

```

1  #include "EventBus.h"
2
3  #include <QDateTime>
4  #include <QDebug>
5  #include <QCoreApplication>
6
7  // 初始化单例
8  EventBus* EventBus::s_instance = nullptr;
9
10 EventBus* EventBus::instance()
11 {
12     if (!s_instance) {
13         s_instance = new EventBus();
14     }
15     return s_instance;
16 }
17
18 EventBus::EventBus(QObject *parent)
19     : QObject(parent)
20 {
21     qDebug() << "[EventBus] 事件总线初始化完成";
22 }
23
24 EventBus::~EventBus()
25 {
26     qDebug() << "[EventBus] 事件总线已销毁";
27 }
28
29 int EventBus::subscribe(const QString& eventType,
30                         std::function<void(const GameEvent*)> handler,
31                         int priority)
32 {
33     Subscription sub;

```

```
34     sub.id = m_nextSubscriptionId++;
35     sub.eventType = eventType;
36     sub.handler = handler;
37     sub.priority = priority;
38     sub.receiver = nullptr;
39
40     m_subscriptions.append(sub);
41
42     // 按优先级排序（高优先级在前）
43     std::sort(m_subscriptions.begin(), m_subscriptions.end(),
44             [](const Subscription& a, const Subscription& b) {
45                 return a.priority > b.priority;
46             });
47
48     qDebug() << "[EventBus] 订阅事件:" << eventType
49             << "ID:" << sub.id
50             << "优先级:" << priority;
51
52     return sub.id;
53 }
54
55 void EventBus::unsubscribe(int subscriptionId)
56 {
57     for (int i = 0; i < m_subscriptions.size(); ++i) {
58         if (m_subscriptions[i].id == subscriptionId) {
59             qDebug() << "[EventBus] 取消订阅 ID:" << subscriptionId;
60             m_subscriptions.removeAt(i);
61             return;
62         }
63     }
64 }
65
66 void EventBus::unsubscribeAll(QObject* receiver)
67 {
68     if (!receiver) return;
69
70     QMutableListIterator<Subscription> it(m_subscriptions);
71     while (it.hasNext()) {
72         const Subscription& sub = it.next();
73         if (sub.receiver == receiver) {
74             it.remove();
75         }
76     }
77
78     qDebug() << "[EventBus] 取消对象的所有订阅:" << receiver->objectName();
79 }
80
```

```
81 void EventBus::publish(const GameEvent& event)
82 {
83     // 遍历所有订阅, 找到匹配的
84     for (const Subscription& sub : m_subscriptions) {
85         if (matchEventType(sub.eventType, event.type)) {
86             try {
87                 sub.handler(event);
88             } catch (const std::exception& e) {
89                 qWarning() << "[EventBus] 事件处理异常:" << e.what()
90                     << "事件类型:" << event.type;
91             }
92         }
93     }
94 }
95
96 void EventBus::publishAsync(const GameEvent& event)
97 {
98     m_eventQueue.append(event);
99
100     // 如果队列之前是空的, 安排下一帧处理
101     if (m_eventQueue.size() == 1) {
102         QMetaObject::invokeMethod(this, "processEventQueue",
Qt::QueuedConnection);
103     }
104 }
105
106 bool EventBus::hasSubscribers(const QString& eventType) const
107 {
108     for (const Subscription& sub : m_subscriptions) {
109         if (matchEventType(sub.eventType, eventType)) {
110             return true;
111         }
112     }
113     return false;
114 }
115
116 void EventBus::subscribeImpl(const QString& eventType,
117                             QObject* receiver,
118                             std::function<void(const GameEvent&)> handler,
119                             int priority)
120 {
121     Subscription sub;
122     sub.id = m_nextSubscriptionId++;
123     sub.eventType = eventType;
124     sub.handler = handler;
125     sub.priority = priority;
126     sub.receiver = receiver;
```

```
127
128     m_subscriptions.append(sub);
129
130     // 按优先级排序
131     std::sort(m_subscriptions.begin(), m_subscriptions.end(),
132             [](const Subscription& a, const Subscription& b) {
133                 return a.priority > b.priority;
134             });
135
136     // 如果receiver销毁了, 自动取消订阅
137     if (receiver) {
138         connect(receiver, &QObject::destroyed, this, [this, receiver]() {
139             unsubscribeAll(receiver);
140         });
141     }
142
143     qDebug() << "[EventBus] 订阅事件:" << eventType
144             << "ID:" << sub.id
145             << "接收者:" << (receiver ? receiver->objectName() : "null");
146 }
147
148 bool EventBus::matchEventType(const QString& pattern, const QString&
149                               eventType) const
150 {
151     // 简单的通配符匹配
152     // 支持 * 匹配任意字符
153     // 支持 ? 匹配单个字符
154     // 支持 ** 匹配多层级
155
156     // 完全匹配
157     if (pattern == eventType) {
158         return true;
159     }
160
161     // 通配符匹配
162     QRegExp regex(pattern, Qt::CaseSensitive, QRegExp::Wildcard);
163     return regex.exactMatch(eventType);
164 }
165
166 void EventBus::processEventQueue()
167 {
168     QList<GameEvent> queue = m_eventQueue;
169     m_eventQueue.clear();
170
171     for (const GameEvent& event : queue) {
172         publish(event);
173     }
174 }
```

2.3.2 常用事件类型定义

为了方便使用，我们定义一些核心事件类型的常量。

文件：**src/core/EventTypes.h**

代码块

```

1  #ifndef EVENTTYPES_H
2  #define EVENTTYPES_H
3
4  #include <QString>
5
6  /**
7   * @brief 核心事件类型定义
8   *
9   * 所有系统级的事件类型都在这里定义常量。
10  * 插件可以自定义自己的事件类型，但建议遵循命名规范：
11  * - 系统事件: "system.*"
12  * - 游戏事件: "game.*"
13  * - 玩家事件: "player.*"
14  * - 敌人事件: "enemy.*"
15  * - UI事件: "ui.*"
16  * - 输入事件: "input.*"
17  */
18 namespace EventTypes
19 {
20     // ===== 系统事件 =====
21
22     /// 游戏启动完成
23     const QString System_StartupComplete = "system.startup_complete";
24
25     /// 游戏即将关闭
26     const QString System_Shutdown = "system.shutdown";
27
28     /// 场景切换开始
29     const QString System_SceneChangeStart = "system.scene_change_start";
30
31     /// 场景切换完成
32     const QString System_SceneChangeComplete = "system.scene_change_complete";
33
34     /// 游戏状态改变
35     const QString System_StateChanged = "system.state_changed";
36
37     // ===== 游戏事件 =====

```

```
38
39     /// 游戏暂停
40     const QString Game_Paused = "game.paused";
41
42     /// 游戏恢复
43     const QString Game_Resume = "game.resume";
44
45     /// 游戏时间更新（每帧）
46     const QString Game_Tick = "game.tick";
47
48     /// 新游戏开始
49     const QString Game_NewGame = "game.new_game";
50
51     /// 游戏存档加载
52     const QString Game_LoadSave = "game.load_save";
53
54     // ===== 玩家事件 =====
55
56     /// 玩家移动
57     const QString Player_Moved = "player.moved";
58
59     /// 玩家受伤
60     const QString Player_Damaged = "player.damaged";
61
62     /// 玩家死亡
63     const QString Player_Dead = "player.dead";
64
65     /// 玩家复活
66     const QString Player_Revived = "player.revived";
67
68     /// 玩家等级提升
69     const QString Player_LevelUp = "player.level_up";
70
71     /// 玩家获得物品
72     const QString Player_ItemAdded = "player.item_added";
73
74     /// 玩家失去物品
75     const QString Player_ItemRemoved = "player.item_removed";
76
77     /// 玩家星核形态切换
78     const QString Player_FormChanged = "player.form_changed";
79
80     // ===== 战斗事件 =====
81
82     /// 战斗开始
83     const QString Combat_Started = "combat.started";
84
```

```
85     /// 战斗结束
86     const QString Combat_Ended = "combat.ended";
87
88     /// 敌人死亡
89     const QString Enemy_Dead = "enemy.dead";
90
91     /// BOSS死亡
92     const QString Boss_Dead = "boss.dead";
93
94     // ===== UI事件 =====
95
96     /// 主菜单打开
97     const QString UI_MainMenuOpened = "ui.main_menu_opened";
98
99     /// 主菜单关闭
100    const QString UI_MainMenuClosed = "ui.main_menu_closed";
101
102    /// 暂停菜单打开
103    const QString UI_PauseMenuOpened = "ui.pause_menu_opened";
104
105    /// 暂停菜单关闭
106    const QString UI_PauseMenuClosed = "ui.pause_menu_closed";
107
108    /// 对话框打开
109    const QString UI_DialogOpened = "ui.dialog_opened";
110
111    /// 对话框关闭
112    const QString UI_DialogClosed = "ui.dialog_closed";
113
114    /// 通知消息
115    const QString UI_Notification = "ui.notification";
116
117    // ===== 输入事件 =====
118
119    /// 按键按下
120    const QString Input_KeyPressed = "input.key_pressed";
121
122    /// 按键释放
123    const QString Input_KeyReleased = "input.key_released";
124
125    /// 鼠标移动
126    const QString Input_MouseMoved = "input.mouse_moved";
127
128    /// 鼠标点击
129    const QString Input_MouseClicked = "input.mouse_clicked";
130
131    // ===== 资源事件 =====
```

```
132
133     /// 资源加载开始
134     const QString Resource_LoadStarted = "resource.load_started";
135
136     /// 资源加载完成
137     const QString Resource_LoadComplete = "resource.load_complete";
138
139     /// 资源加载失败
140     const QString Resource_LoadFailed = "resource.load_failed";
141
142     // ===== 时间褶皱系统事件 =====
143
144     /// 时间回溯开始
145     const QString TimeRewind_Started = "time.rewind_started";
146
147     /// 时间回溯结束
148     const QString TimeRewind_Ended = "time.rewind_ended";
149
150     /// 时间残影创建
151     const QString TimeEcho_Created = "time.echo_created";
152
153     /// 时间视野开启/关闭
154     const QString TimeVision_Toggled = "time.vision_toggled";
155
156     // ===== 星核共鸣系统事件 =====
157
158     /// 星核形态切换
159     const QString StarCore_FormSwitched = "starcore.form_switched";
160
161     /// 星核碎片获得
162     const QString StarCore_FragmentAdded = "starcore.fragment_added";
163
164     /// 共鸣组合技释放
165     const QString StarCore_ComboUsed = "starcore.combo_used";
166
167     // ===== 生态共生系统事件 =====
168
169     /// 共生伙伴加入
170     const QString Symbiosis_PartnerJoined = "symbiosis.partner_joined";
171
172     /// 共生伙伴切换
173     const QString Symbiosis_PartnerSwitched = "symbiosis.partner_switched";
174
175     /// 共生伙伴进化
176     const QString Symbiosis_PartnerEvolved = "symbiosis.partner_evolved";
177 }
178
```

设计思路：用命名空间+常量字符串的方式定义事件类型，比用枚举灵活，插件可以自由扩展自己的事件类型，不用修改核心代码。

2.4 第四步：核心框架 - 资源管理器

资源管理器负责统一管理游戏中的所有资源：图片、音频、字体、数据文件等。

2.4.1 资源管理器实现

文件：src/core/ResourceManager.h

代码块

```
1  #ifndef RESOURCEMANAGER_H
2  #define RESOURCEMANAGER_H
3
4  #include <QObject>
5  #include <QPixmap>
6  #include <QSoundEffect>
7  #include <QFont>
8  #include <QJsonDocument>
9  #include <QMap>
10 #include <QString>
11 #include <QDir>
12
13 /**
14  * @brief 资源管理器
15  *
16  * 统一管理游戏中的所有资源，提供缓存机制，避免重复加载。
17  *
18  * 支持的资源类型：
19  * - 图片 (QPixmap)
20  * - 音效 (QSoundEffect)
21  * - 字体 (QFont)
22  * - JSON数据
23  * - 文本文件
24  *
25  * 特点：
26  * - 自动缓存：加载过的资源会缓存起来，下次直接用
27  * - 引用计数：可以手动管理资源生命周期
28  * - 异步加载：大资源可以异步加载，不卡主线程
29  * - 搜索路径：可以设置多个资源搜索路径
30  *
31  * 使用方式：
```

```

32  * - 图片: QPixmap pix = ResourceManager::instance()->getPixmap("player.png");
33  * - 音效: ResourceManager::instance()->playSound("jump.wav");
34  * - JSON: QJsonDocument doc = ResourceManager::instance()-
>getJson("config.json");
35  */
36  class ResourceManager : public QObject
37  {
38      Q_OBJECT
39
40  public:
41      // 获取单例
42      static ResourceManager* instance();
43
44      // 禁止拷贝
45      ResourceManager(const ResourceManager&) = delete;
46      ResourceManager& operator=(const ResourceManager&) = delete;
47
48      /**
49       * @brief 添加资源搜索路径
50       * @param path 路径
51       *
52       * 加载资源时会按添加顺序在这些路径中查找。
53       * 先添加的路径优先级高。
54       */
55      void addSearchPath(const QString& path);
56
57      /**
58       * @brief 设置资源根目录
59       * @param path 根目录路径
60       *
61       * 所有相对路径都相对于这个目录。
62       */
63      void setResourceRoot(const QString& path);
64
65      /**
66       * @brief 获取图片资源
67       * @param name 图片名称 (相对路径)
68       * @return 图片, 加载失败返回空图片
69       *
70       * 会自动缓存, 第二次调用直接返回缓存的图片。
71       */
72      QPixmap getPixmap(const QString& name);
73
74      /**
75       * @brief 获取图片 (指定大小)
76       * @param name 图片名称
77       * @param width 宽度

```

```

78      * @param height 高度
79      * @param mode 缩放模式
80      * @return 缩放后的图片
81      */
82      QPixmap getPixmapScaled(const QString& name, int width, int height,
83                              Qt::AspectRatioMode mode = Qt::KeepAspectRatio);
84
85      /**
86       * @brief 播放音效
87       * @param name 音效文件名
88       * @param volume 音量 (0.0 - 1.0)
89       * @param loop 循环次数 (-1为无限循环)
90       *
91       * 音效会被缓存，第一次加载后直接播放。
92       * 适合短音效（攻击、UI点击等）。
93       */
94      void playSound(const QString& name, float volume = 1.0f, int loop = 0);
95
96      /**
97       * @brief 获取音效对象
98       * @param name 音效文件名
99       * @return 音效对象指针，可自己控制播放
100      */
101      QSoundEffect* getSoundEffect(const QString& name);
102
103      /**
104       * @brief 获取字体
105       * @param name 字体文件名
106       * @param size 字号
107       * @return 字体
108       */
109      QFont getFont(const QString& name, int size = 12);
110
111      /**
112       * @brief 加载JSON数据
113       * @param name JSON文件名
114       * @return JSON文档
115       */
116      QJsonDocument getJson(const QString& name);
117
118      /**
119       * @brief 加载文本文件
120       * @param name 文件名
121       * @return 文件内容
122       */
123      QString getTextFile(const QString& name);
124

```

```
125     /**
126      * @brief 检查资源是否存在
127      * @param name 资源名称
128      * @return 是否存在
129      */
130     bool exists(const QString& name) const;
131
132     /**
133      * @brief 获取资源的完整路径
134      * @param name 资源名称
135      * @return 完整路径，找不到返回空字符串
136      */
137     QString getResourcePath(const QString& name) const;
138
139     /**
140      * @brief 清除所有缓存
141      *
142      * 场景切换时可以调用，释放不用的资源。
143      */
144     void clearCache();
145
146     /**
147      * @brief 清除指定资源的缓存
148      * @param name 资源名称
149      */
150     void removeFromCache(const QString& name);
151
152     /**
153      * @brief 获取缓存统计信息
154      * @return 统计信息字符串（用于调试）
155      */
156     QString getCacheStats() const;
157
158     signals:
159     /**
160      * @brief 资源加载进度信号
161      * @param percent 进度百分比（0-100）
162      * @param current 当前加载的资源名
163      *
164      * 批量加载资源时发出，用于显示加载进度条。
165      */
166     void loadProgress(int percent, const QString& current);
167
168     /**
169      * @brief 所有资源加载完成
170      */
171     void loadComplete();
```

```

172
173 private:
174     explicit ResourceManager(QObject *parent = nullptr);
175     ~ResourceManager();
176
177     static ResourceManager* s_instance;
178
179     // 资源根目录
180     QString m_resourceRoot;
181
182     // 搜索路径列表
183     QStringList m_searchPaths;
184
185     // 缓存
186     QMap<QString, QPixmap> m_pixmapCache;
187     QMap<QString, QSoundEffect*> m_soundCache;
188     QMap<QString, QFont> m_fontCache;
189     QMap<QString, QJsonDocument> m_jsonCache;
190     QMap<QString, QString> m_textCache;
191
192     // 已加载的字体ID（用于addApplicationFont）
193     QList<int> m_loadedFontIds;
194
195     /**
196      * @brief 在搜索路径中查找文件
197      * @param fileName 文件名
198      * @return 完整路径，找不到返回空字符串
199      */
200     QString findFile(const QString& fileName) const;
201 };
202
203 #endif // RESOURCEMANAGER_H

```

文件：src/core/ResourceManager.cpp

代码块

```

1  #include "ResourceManager.h"
2
3  #include <QDebug>
4  #include <QFile>
5  #include <QFontDatabase>
6  #include <QCoreApplication>
7
8  // 初始化单例
9  ResourceManager* ResourceManager::s_instance = nullptr;

```

```

10
11 ResourceManager* ResourceManager::instance()
12 {
13     if (!s_instance) {
14         s_instance = new ResourceManager();
15     }
16     return s_instance;
17 }
18
19 ResourceManager::ResourceManager(QObject *parent)
20     : QObject(parent)
21 {
22     // 默认资源根目录：应用程序目录下的resources文件夹
23     QString appDir = QApplication::applicationDirPath();
24     m_resourceRoot = appDir + "/resources";
25
26     // 添加默认搜索路径
27     addSearchPath(m_resourceRoot + "/images");
28     addSearchPath(m_resourceRoot + "/sounds");
29     addSearchPath(m_resourceRoot + "/fonts");
30     addSearchPath(m_resourceRoot + "/data");
31     addSearchPath(m_resourceRoot + "/styles");
32
33     // 同时支持qrc资源路径
34     addSearchPath(":/images");
35     addSearchPath(":/sounds");
36     addSearchPath(":/fonts");
37     addSearchPath(":/data");
38     addSearchPath(":/styles");
39
40     qDebug() << "[ResourceManager] 资源管理器初始化完成";
41     qDebug() << "[ResourceManager] 资源根目录:" << m_resourceRoot;
42 }
43
44 ResourceManager::~ResourceManager()
45 {
46     // 释放音效资源
47     qDeleteAll(m_soundCache);
48     m_soundCache.clear();
49
50     qDebug() << "[ResourceManager] 资源管理器已销毁";
51 }
52
53 void ResourceManager::addSearchPath(const QString& path)
54 {
55     if (!m_searchPaths.contains(path)) {
56         m_searchPaths.prepend(path); // 新添加的优先级高

```

```
57         qDebug() << "[ResourceManager] 添加搜索路径:" << path;
58     }
59 }
60
61 void ResourceManager::setResourceRoot(const QString& path)
62 {
63     m_resourceRoot = path;
64     qDebug() << "[ResourceManager] 资源根目录设置为:" << path;
65 }
66
67 QPixmap ResourceManager::getPixmap(const QString& name)
68 {
69     // 先查缓存
70     if (m_pixmapCache.contains(name)) {
71         return m_pixmapCache[name];
72     }
73
74     // 查找文件
75     QString filePath = findFile(name);
76     if (filePath.isEmpty()) {
77         qWarning() << "[ResourceManager] 图片未找到:" << name;
78         return QPixmap();
79     }
80
81     // 加载图片
82     QPixmap pixmap(filePath);
83     if (pixmap.isNull()) {
84         qWarning() << "[ResourceManager] 图片加载失败:" << filePath;
85         return QPixmap();
86     }
87
88     // 存入缓存
89     m_pixmapCache[name] = pixmap;
90
91     qDebug() << "[ResourceManager] 图片已加载:" << name
92         << "尺寸:" << pixmap.size();
93
94     return pixmap;
95 }
96
97 QPixmap ResourceManager::getPixmapScaled(const QString& name, int width, int
height,
98                                         Qt::AspectRatioMode mode)
99 {
100     QPixmap original = getPixmap(name);
101     if (original.isNull()) {
102         return QPixmap();
```

```
103     }
104
105     // 生成缓存键
106     QString cacheKey = QString("%1_%2x%3_%4")
107         .arg(name)
108         .arg(width)
109         .arg(height)
110         .arg(mode);
111
112     if (m_pixmapCache.contains(cacheKey)) {
113         return m_pixmapCache[cacheKey];
114     }
115
116     // 缩放
117     QPixmap scaled = original.scaled(width, height, mode,
Qt::SmoothTransformation);
118     m_pixmapCache[cacheKey] = scaled;
119
120     return scaled;
121 }
122
123 void ResourceManager::playSound(const QString& name, float volume, int loop)
124 {
125     QSoundEffect* sound = getSoundEffect(name);
126     if (!sound) {
127         return;
128     }
129
130     sound->setVolume(volume);
131     sound->setLoopCount(loop);
132     sound->play();
133 }
134
135 QSoundEffect* ResourceManager::getSoundEffect(const QString& name)
136 {
137     // 先查缓存
138     if (m_soundCache.contains(name)) {
139         return m_soundCache[name];
140     }
141
142     // 查找文件
143     QString filePath = findFile(name);
144     if (filePath.isEmpty()) {
145         qWarning() << "[ResourceManager] 音效未找到:" << name;
146         return nullptr;
147     }
148
```

```
149 // 加载音效
150 QSoundEffect* sound = new QSoundEffect(this);
151 sound->setSource(QUrl::fromLocalFile(filePath));
152
153 // 等待加载完成（同步加载，因为音效一般很小）
154 // 如果是大音效，建议用QMediaPlayer异步加载
155 while (sound->status() == QSoundEffect::Loading) {
156     QCoreApplication::processEvents();
157 }
158
159 if (sound->status() != QSoundEffect::Ready) {
160     qWarning() << "[ResourceManager] 音效加载失败:" << filePath;
161     delete sound;
162     return nullptr;
163 }
164
165 // 存入缓存
166 m_soundCache[name] = sound;
167
168 qDebug() << "[ResourceManager] 音效已加载:" << name;
169
170 return sound;
171 }
172
173 QFont ResourceManager::getFont(const QString& name, int size)
174 {
175     // 生成缓存键
176     QString cacheKey = QString("%1_%2").arg(name).arg(size);
177
178     if (m_fontCache.contains(cacheKey)) {
179         return m_fontCache[cacheKey];
180     }
181
182     // 查找字体文件
183     QString filePath = findFile(name);
184     if (filePath.isEmpty()) {
185         qWarning() << "[ResourceManager] 字体未找到:" << name;
186         return QFont(); // 返回默认字体
187     }
188
189     // 加载字体
190     int fontId = QFontDatabase::addApplicationFont(filePath);
191     if (fontId == -1) {
192         qWarning() << "[ResourceManager] 字体加载失败:" << filePath;
193         return QFont();
194     }
195 }
```

```
196     m_loadedFontIds.append(fontId);
197
198     // 获取字体家族名
199     QString family = QFontDatabase::applicationFontFamilies(fontId).first();
200     QFont font(family, size);
201
202     m_fontCache[cacheKey] = font;
203
204     qDebug() << "[ResourceManager] 字体已加载:" << name
205             << "家族:" << family
206             << "字号:" << size;
207
208     return font;
209 }
210
211 QJsonDocument ResourceManager::getJson(const QString& name)
212 {
213     if (m_jsonCache.contains(name)) {
214         return m_jsonCache[name];
215     }
216
217     QString filePath = findFile(name);
218     if (filePath.isEmpty()) {
219         qWarning() << "[ResourceManager] JSON文件未找到:" << name;
220         return QJsonDocument();
221     }
222
223     QFile file(filePath);
224     if (!file.open(QIODevice::ReadOnly)) {
225         qWarning() << "[ResourceManager] JSON文件无法打开:" << filePath;
226         return QJsonDocument();
227     }
228
229     QByteArray data = file.readAll();
230     QJsonDocument doc = QJsonDocument::fromJson(data);
231
232     if (doc.isNull()) {
233         qWarning() << "[ResourceManager] JSON解析失败:" << filePath;
234         return QJsonDocument();
235     }
236
237     m_jsonCache[name] = doc;
238
239     qDebug() << "[ResourceManager] JSON已加载:" << name;
240
241     return doc;
242 }
```

```
243
244 QString ResourceManager::getTextFile(const QString& name)
245 {
246     if (m_textCache.contains(name)) {
247         return m_textCache[name];
248     }
249
250     QString filePath = findFile(name);
251     if (filePath.isEmpty()) {
252         qWarning() << "[ResourceManager] 文本文件未找到:" << name;
253         return QString();
254     }
255
256     QFile file(filePath);
257     if (!file.open(QIODevice::ReadOnly | QIODevice::Text)) {
258         qWarning() << "[ResourceManager] 文本文件无法打开:" << filePath;
259         return QString();
260     }
261
262     QString content = file.readAll();
263     m_textCache[name] = content;
264
265     qDebug() << "[ResourceManager] 文本文件已加载:" << name;
266
267     return content;
268 }
269
270 bool ResourceManager::exists(const QString& name) const
271 {
272     return !findFile(name).isEmpty();
273 }
274
275 QString ResourceManager::getResourcePath(const QString& name) const
276 {
277     return findFile(name);
278 }
279
280 void ResourceManager::clearCache()
281 {
282     m_pixmapCache.clear();
283
284     // 音效需要手动delete
285     qDeleteAll(m_soundCache);
286     m_soundCache.clear();
287
288     m_fontCache.clear();
289     m_jsonCache.clear();
```

```
290     m_textCache.clear();
291
292     qDebug() << "[ResourceManager] 所有缓存已清除";
293 }
294
295 void ResourceManager::removeFromCache(const QString& name)
296 {
297     m_pixmapCache.remove(name);
298
299     if (m_soundCache.contains(name)) {
300         delete m_soundCache[name];
301         m_soundCache.remove(name);
302     }
303
304     m_jsonCache.remove(name);
305     m_textCache.remove(name);
306
307     qDebug() << "[ResourceManager] 资源已从缓存移除:" << name;
308 }
309
310 QString ResourceManager::getCacheStats() const
311 {
312     return QString("图片: %1张, 音效: %2个, JSON: %3个, 文本: %4个")
313         .arg(m_pixmapCache.size())
314         .arg(m_soundCache.size())
315         .arg(m_jsonCache.size())
316         .arg(m_textCache.size());
317 }
318
319 QString ResourceManager::findFile(const QString& fileName) const
320 {
321     // 先检查是不是绝对路径
322     QFileInfo fileInfo(fileName);
323     if (fileInfo.isAbsolute() && fileInfo.exists()) {
324         return fileName;
325     }
326
327     // 在搜索路径中查找
328     for (const QString& path : m_searchPaths) {
329         QString fullPath = path + "/" + fileName;
330         if (QFile::exists(fullPath)) {
331             return fullPath;
332         }
333     }
334
335     // 检查qrc资源
336     QString qrcPath = ":" + fileName;
```

```
337     if (QFile::exists(qrcPath)) {
338         return qrcPath;
339     }
340
341     return QString(); // 没找到
342 }
```

2.5 第五步：基础服务 - 渲染服务（Graphics View封装）

渲染服务基于Qt的Graphics View框架，是游戏画面渲染的核心。

2.5.1 渲染服务接口

文件：src/core/services/IRenderService.h

代码块

```
1  #ifndef IRENDERSERVICE_H
2  #define IRENDERSERVICE_H
3
4  #include <QObject>
5  #include <QGraphicsScene>
6  #include <QGraphicsView>
7  #include <QGraphicsItem>
8  #include <QRectF>
9  #include <QPointF>
10
11 /**
12  * @brief 渲染服务接口
13  *
14  * 封装Qt Graphics View框架，提供游戏场景渲染功能。
15  *
16  * 核心概念：
17  * - Scene（场景）：存放所有游戏对象的容器
18  * - View（视图）：观察场景的窗口，可以缩放、平移
19  * - Item（图元）：场景中的物体（玩家、敌人、道具等）
20  *
21  * Graphics View的优势：
22  * - BSP空间索引，自动优化碰撞检测和渲染
23  * - Z轴排序，自动处理遮挡关系
24  * - 视口裁剪，只渲染可见区域
25  * - 内置坐标变换，支持缩放、旋转
26  */
27 class IRenderService
28 {
```

```
29 public:
30     virtual ~IRenderService() = default;
31
32     /**
33      * @brief 获取主场景
34      * @return 场景指针
35      */
36     virtual QGraphicsScene* getScene() const = 0;
37
38     /**
39      * @brief 获取主视图
40      * @return 视图指针
41      */
42     virtual QGraphicsView* getView() const = 0;
43
44     /**
45      * @brief 添加图元到场景
46      * @param item 图元指针
47      *
48      * 图元的内存管理：
49      * - 场景会作为图元的父对象吗？不会，QGraphicsItem不是QObject
50      * - 场景销毁时会自动删除所有图元
51      * - 也可以手动调用 scene->removeItem() 然后 delete
52      */
53     virtual void addItem(QGraphicsItem* item) = 0;
54
55     /**
56      * @brief 从场景移除图元
57      * @param item 图元指针
58      *
59      * 注意：移除后图元不会被自动删除，需要手动delete。
60      * 或者把图元加到对象池里复用。
61      */
62     virtual void removeItem(QGraphicsItem* item) = 0;
63
64     /**
65      * @brief 设置视图中心位置
66      * @param pos 中心坐标（场景坐标）
67      *
68      * 相机会平滑移动到目标位置。
69      */
70     virtual void centerOn(const QPointF& pos) = 0;
71
72     /**
73      * @brief 设置视图缩放比例
74      * @param scale 缩放比例（1.0为原始大小）
75      */
```

```

76     virtual void setZoom(qreal scale) = 0;
77
78     /**
79      * @brief 获取当前缩放比例
80      * @return 缩放比例
81      */
82     virtual qreal getZoom() const = 0;
83
84     /**
85      * @brief 设置场景大小
86      * @param rect 场景矩形范围
87      */
88     virtual void setSceneRect(const QRectF& rect) = 0;

```

代码块

```

1  virtual void setSceneRect(const QRectF& rect) = 0;
2
3  /**
4   * @brief 获取视口矩形（场景坐标）
5   * @return 视口矩形
6   *
7   * 返回当前视图在场景中能看到的区域。
8   * 用于优化：只更新视口内的物体。
9   */
10 virtual QRectF getViewportRect() const = 0;
11
12 /**
13  * @brief 屏幕坐标转场景坐标
14  * @param screenPos 屏幕坐标
15  * @return 场景坐标
16  */
17 virtual QPointF mapToScene(const QPoint& screenPos) const = 0;
18
19 /**
20  * @brief 场景坐标转屏幕坐标
21  * @param scenePos 场景坐标
22  * @return 屏幕坐标
23  */
24 virtual QPoint mapFromScene(const QPointF& scenePos) const = 0;
25
26 /**
27  * @brief 设置背景颜色
28  * @param color 颜色
29  */
30 virtual void setBackgroundColor(const QColor& color) = 0;

```

```

31
32  /**
33   * @brief 启用/禁用OpenGL渲染
34   * @param enabled 是否启用
35   *
36   * 启用OpenGL可以提升复杂场景的性能，
37   * 但会增加兼容性问题。默认关闭。
38   */
39  virtual void setOpenGL(bool enabled) = 0;
40
41  /**
42   * @brief 更新整个场景
43   *
44   * 强制重绘整个场景。
45   * 一般不需要手动调用，Graphics View会自动处理。
46   */
47  virtual void update() = 0;

```

```
};
```

```
#endif // IRENDERSERVICE_H
```

代码块

```

1
2  ##### 2.5.2 渲染服务实现
3
4  **文件：src/services/render/RenderService.h**
5  ```cpp
6  #ifndef RENDERSERVICE_H
7  #define RENDERSERVICE_H
8
9  #include <QObject>
10 #include <QGraphicsScene>
11 #include <QGraphicsView>
12 #include <QTimer>
13 #include <QPropertyAnimation>
14
15 #include "IRenderService.h"
16
17  /**
18   * @brief 渲染服务实现
19   *
20   * 基于Qt Graphics View框架的渲染服务。
21   *
22   * 性能优化策略：
23   * 1. 视口裁剪：Graphics View自动做了

```

```

24  * 2. 分层渲染：背景层、游戏层、前景层、UI层
25  * 3. 图元缓存：静态图元用CacheMode缓存
26  * 4. 对象池：频繁创建销毁的对象用对象池
27  * 5. 可选OpenGL：复杂场景可以启用
28  */
29  class RenderService : public QObject, public IRenderService
30  {
31      Q_OBJECT
32      Q_PROPERTY(qreal zoom READ getZoom WRITE setZoom NOTIFY zoomChanged)
33
34  public:
35      explicit RenderService(QObject *parent = nullptr);
36      ~RenderService() override;
37
38      // IRenderService接口实现
39      QGraphicsScene* getScene() const override { return m_scene; }
40      QGraphicsView* getView() const override { return m_view; }
41
42      void addItem(QGraphicsItem* item) override;
43      void removeItem(QGraphicsItem* item) override;
44
45      void centerOn(const QPointF& pos) override;
46      void setZoom(qreal scale) override;
47      qreal getZoom() const override { return m_zoom; }
48
49      void setSceneRect(const QRectF& rect) override;
50      QRectF getViewportRect() const override;
51
52      QPointF mapToScene(const QPoint& screenPos) const override;
53      QPoint mapFromScene(const QPointF& scenePos) const override;
54
55      void setBackgroundColor(const QColor& color) override;
56      void setOpenGL(bool enabled) override;
57      void update() override;
58
59      /**
60       * @brief 设置目标FPS
61       * @param fps 帧率
62       *
63       * 默认60帧。
64       * 注意：这只是目标帧率，实际帧率取决于硬件性能。
65       */
66      void setTargetFps(int fps);
67
68      /**
69       * @brief 获取实际FPS
70       * @return 当前帧率

```

```
71     */
72     int getCurrentFps() const { return m_currentFps; }
73
74     /**
75      * @brief 开始游戏主循环
76      *
77      * 启动定时器，每帧调用advance()。
78      */
79     void startGameLoop();
80
81     /**
82      * @brief 停止游戏主循环
83      */
84     void stopGameLoop();
85
86 signals:
87     /**
88      * @brief 每帧更新信号
89      * @param deltaTime 距离上一帧的时间（秒）
90      *
91      * 游戏逻辑应该连接这个信号来更新。
92      */
93     void tick(qreal deltaTime);
94
95     /**
96      * @brief 缩放改变信号
97      */
98     void zoomChanged(qreal zoom);
99
100 private slots:
101     /**
102      * @brief 游戏主循环的一帧
103      */
104     void onGameLoopTick();
105
106 private:
107     // 场景和视图
108     QGraphicsScene* m_scene = nullptr;
109     QGraphicsView* m_view = nullptr;
110
111     // 缩放
112     qreal m_zoom = 1.0;
113
114     // 游戏循环
115     QTimer* m_gameTimer = nullptr;
116     int m_targetFps = 60;
117     int m_currentFps = 0;
```

```

118
119     // FPS计算
120     qint64 m_lastFrameTime = 0;
121     int m_frameCount = 0;
122     qreal m_fpsAccumulator = 0;
123
124     // 相机平滑移动
125     QPointF m_targetCenter;
126     bool m_cameraMoving = false;
127
128     /**
129      * @brief 计算FPS
130      * @param deltaTime 帧间隔
131      */
132     void updateFps(qreal deltaTime);
133 };
134
135 #endif // RENDERSERVICE_H

```

文件：src/services/render/RenderService.cpp

代码块

```

1  #include "RenderService.h"
2
3  #include <QDebug>
4  #include <QDateTime>
5  #include <QOpenGLWidget>
6  #include <QGraphicsRectItem>
7
8  RenderService::RenderService(QObject *parent)
9      : QObject(parent)
10 {
11     // 创建场景
12     m_scene = new QGraphicsScene(this);
13     m_scene->setSceneRect(0, 0, 1920, 1080); // 默认场景大小
14
15     // 创建视图
16     m_view = new QGraphicsView(m_scene);
17     m_view->setRenderHints(QPainter::Antialiasing |
18                             QPainter::SmoothPixmapTransform);
19     m_view->setViewportUpdateMode(QGraphicsView::SmartViewportUpdate);
20     m_view->setHorizontalScrollBarPolicy(Qt::ScrollBarAlwaysOff);
21     m_view->setVerticalScrollBarPolicy(Qt::ScrollBarAlwaysOff);
22     m_view->setFrameShape(QFrame::NoFrame);

```

```
23     // 背景色 (默认黑色)
24     m_view->setBackgroundBrush(Qt::black);
25
26     // 创建游戏定时器
27     m_gameTimer = new QTimer(this);
28     connect(m_gameTimer, &QTimer::timeout, this,
    &RenderService::onGameLoopTick);
29
30     qDebug() << "[RenderService] 渲染服务初始化完成";
31     qDebug() << "[RenderService] 默认场景大小: 1920x1080";
32     qDebug() << "[RenderService] 目标帧率: " << m_targetFps << "fps";
33 }
34
35 RenderService::~RenderService()
36 {
37     stopGameLoop();
38
39     // view会被它的父widget释放吗?
40     // 这里我们没有设置父widget, 所以需要手动管理
41     // 但实际上view会被设置到主窗口上, 由主窗口管理
42     // 这里不delete, 避免双重释放
43
44     qDebug() << "[RenderService] 渲染服务已销毁";
45 }
46
47 void RenderService::addItem(QGraphicsItem* item)
48 {
49     if (item && m_scene) {
50         m_scene->addItem(item);
51     }
52 }
53
54 void RenderService::removeItem(QGraphicsItem* item)
55 {
56     if (item && m_scene) {
57         m_scene->removeItem(item);
58     }
59 }
60
61 void RenderService::centerOn(const QPointF& pos)
62 {
63     if (m_view) {
64         m_view->centerOn(pos);
65     }
66 }
67
68 void RenderService::setZoom(qreal scale)
```

```
69  {
70      if (m_view && scale > 0.1 && scale < 10.0) {
71          // 计算缩放比例
72          qreal factor = scale / m_zoom;
73
74          m_view->scale(factor, factor);
75          m_zoom = scale;
76
77          emit zoomChanged(m_zoom);
78      }
79  }
80
81  void RenderService::setSceneRect(const QRectF& rect)
82  {
83      if (m_scene) {
84          m_scene->setSceneRect(rect);
85      }
86  }
87
88  QRectF RenderService::getViewportRect() const
89  {
90      if (!m_view) {
91          return QRectF();
92      }
93
94      // 获取视口在场景中的矩形
95      QRect viewportRect = m_view->viewport()->rect();
96      QPointF topLeft = m_view->mapToScene(viewportRect.topLeft());
97      QPointF bottomRight = m_view->mapToScene(viewportRect.bottomRight());
98
99      return QRectF(topLeft, bottomRight);
100 }
101
102 QPointF RenderService::mapToScene(const QPoint& screenPos) const
103 {
104     if (m_view) {
105         return m_view->mapToScene(screenPos);
106     }
107     return QPointF();
108 }
109
110 QPoint RenderService::mapFromScene(const QPointF& scenePos) const
111 {
112     if (m_view) {
113         return m_view->mapFromScene(scenePos);
114     }
115     return QPoint();
116 }
```

```
116 }
117
118 void RenderService::setBackgroundColor(const QColor& color)
119 {
120     if (m_view) {
121         m_view->setBackgroundBrush(color);
122     }
123 }
124
125 void RenderService::setOpenGL(bool enabled)
126 {
127     if (!m_view) return;
128
129     if (enabled) {
130         // 启用OpenGL渲染
131         QOpenGLWidget* glWidget = new QOpenGLWidget();
132         m_view->setViewport(glWidget);
133         qDebug() << "[RenderService] OpenGL渲染已启用";
134     } else {
135         // 换回默认渲染
136         m_view->setViewport(nullptr);
137         qDebug() << "[RenderService] OpenGL渲染已禁用";
138     }
139 }
140
141 void RenderService::update()
142 {
143     if (m_scene) {
144         m_scene->update();
145     }
146 }
147
148 void RenderService::setTargetFps(int fps)
149 {
150     m_targetFps = fps;
151
152     if (m_gameTimer->isActive()) {
153         m_gameTimer->setInterval(1000 / m_targetFps);
154     }
155
156     qDebug() << "[RenderService] 目标帧率设置为:" << fps << "fps";
157 }
158
159 void RenderService::startGameLoop()
160 {
161     if (m_gameTimer->isActive()) {
162         qWarning() << "[RenderService] 游戏循环已经在运行";
```

```
163         return;
164     }
165
166     m_lastFrameTime = QDateTime::currentMSecsSinceEpoch();
167     m_frameCount = 0;
168     m_fpsAccumulator = 0;
169
170     m_gameTimer->setInterval(1000 / m_targetFps);
171     m_gameTimer->start();
172
173     qDebug() << "[RenderService] 游戏循环已启动, 目标" << m_targetFps << "fps";
174 }
175
176 void RenderService::stopGameLoop()
177 {
178     if (m_gameTimer->isActive()) {
179         m_gameTimer->stop();
180         qDebug() << "[RenderService] 游戏循环已停止";
181     }
182 }
183
184 void RenderService::onGameLoopTick()
185 {
186     // 计算deltaTime
187     qint64 currentTime = QDateTime::currentMSecsSinceEpoch();
188     qreal deltaTime = (currentTime - m_lastFrameTime) / 1000.0;
189     m_lastFrameTime = currentTime;
190
191     // 限制最大deltaTime, 防止卡顿后跳帧太大
192     if (deltaTime > 0.1) {
193         deltaTime = 0.1;
194     }
195
196     // 更新FPS统计
197     updateFps(deltaTime);
198
199     // 发出tick信号, 所有游戏逻辑在这里更新
200     emit tick(deltaTime);
201
202     // 推进场景中所有图元的动画
203     // QGraphicsScene::advance()会调用每个item的advance()方法
204     m_scene->advance();
205 }
206
207 void RenderService::updateFps(qreal deltaTime)
208 {
209     m_frameCount++;
```

```

210     m_fpsAccumulator += deltaTime;
211
212     // 每秒更新一次FPS
213     if (m_fpsAccumulator >= 1.0) {
214         m_currentFps = qRound(m_frameCount / m_fpsAccumulator);
215         m_frameCount = 0;
216         m_fpsAccumulator = 0;
217
218         // 可以在这里打日志，调试用
219         // qDebug() << "[RenderService] 当前FPS:" << m_currentFps;
220     }
221 }

```

2.5.3 services模块的CMake配置

文件：src/services/CMakeLists.txt

代码块

```

1  # 服务实现库
2  # 把所有服务实现编译成一个静态库
3
4  add_library(Services STATIC
5      render/RenderService.h
6      render/RenderService.cpp
7
8      input/InputService.h
9      input/InputService.cpp
10
11     audio/AudioService.h
12     audio/AudioService.cpp
13
14     statemachine/StateMachineService.h
15     statemachine/StateMachineService.cpp
16
17     save/SaveService.h
18     save/SaveService.cpp
19 )
20
21 target_link_libraries(Services PUBLIC
22     Core
23     Qt6::Core
24     Qt6::Gui
25     Qt6::Widgets
26     Qt6::Multimedia
27     Qt6::StateMachine
28 )

```

```

29
30 target_include_directories(Services PUBLIC
31     ${CMAKE_CURRENT_SOURCE_DIR}
32     ${CMAKE_CURRENT_SOURCE_DIR}/render
33     ${CMAKE_CURRENT_SOURCE_DIR}/input
34     ${CMAKE_CURRENT_SOURCE_DIR}/audio
35     ${CMAKE_CURRENT_SOURCE_DIR}/statemachine
36     ${CMAKE_CURRENT_SOURCE_DIR}/save
37 )
38
39 set_target_properties(Services PROPERTIES
40     OUTPUT_NAME "starabyss-services"
41 )
42
43 message(STATUS "Services模块配置完成")

```

2.6 第六步：基础服务 - 输入服务

输入服务封装键盘、鼠标、手柄输入，提供统一的输入接口。

2.6.1 输入服务接口

文件：src/core/services/IInputService.h

代码块

```

1  #ifndef IINPUTSERVICE_H
2  #define IINPUTSERVICE_H
3
4  #include <QObject>
5  #include <Qt>
6  #include <QPoint>
7  #include <QMap>
8
9  /**
10   * @brief 输入动作枚举
11   *
12   * 把物理按键映射为游戏动作，这样玩家可以自定义按键。
13   * 游戏逻辑只关心"跳跃"这个动作，不关心是按空格还是按W。
14   */
15  enum class InputAction
16  {
17      // 移动
18      MoveLeft,      // 向左移动
19      MoveRight,     // 向右移动

```

```

20     MoveUp,          // 向上移动
21     MoveDown,        // 向下移动
22
23     // 跳跃
24     Jump,            // 跳跃
25
26     // 攻击
27     Attack,          // 普通攻击
28     Skill1,          // 技能1
29     Skill2,          // 技能2
30
31     // 交互
32     Interact,        // 交互（对话、拾取等）
33
34     // 菜单
35     Pause,           // 暂停
36     Inventory,       // 背包
37     Map,             // 地图
38
39     // 形态切换
40     SwitchFormNext,  // 下一个形态
41     SwitchFormPrev, // 上一个形态
42
43     // 伙伴
44     PartnerSkill,    // 伙伴技能
45
46     // 时间系统
47     TimeRewind,      // 时间回溯
48     TimeVision,      // 时间视野
49
50     // 计数用
51     Count
52 };
53
54 /**
55  * @brief 输入服务接口
56  *
57  * 统一管理键盘、鼠标、手柄输入。
58  *
59  * 特点：
60  * - 按键映射：物理按键 -> 游戏动作，支持自定义
61  * - 状态查询：某键是否按下、刚按下、刚释放
62  * - 输入缓冲：支持输入队列（格斗游戏的搓招）
63  * - 多设备支持：键盘、鼠标、手柄
64  */
65 class IInputService
66 {

```

```
67 public:
68     virtual ~IInputService() = default;
69
70     /**
71      * @brief 动作是否正在按下
72      * @param action 输入动作
73      * @return 是否按下
74      */
75     virtual bool isActionPressed(InputAction action) const = 0;
76
77     /**
78      * @brief 动作是否在这一帧刚按下
79      * @param action 输入动作
80      * @return 是否刚按下
81      *
82      * 用于只触发一次的操作，比如跳跃、攻击。
83      * 按住不放只会返回true。
84      */
85     virtual bool isActionJustPressed(InputAction action) const = 0;
86
87     /**
88      * @brief 动作是否在这一帧刚释放
89      * @param action 输入动作
90      * @return 是否刚释放
91      */
92     virtual bool isActionJustReleased(InputAction action) const = 0;
93
94     /**
95      * @brief 获取鼠标位置（屏幕坐标）
96      * @return 鼠标位置
97      */
98     virtual QPoint getMousePosition() const = 0;
99
100    /**
101     * @brief 获取鼠标位置（场景坐标）
102     * @return 鼠标在场景中的位置
103     */
104    virtual QPointF getMouseScenePosition() const = 0;
105
106    /**
107     * @brief 鼠标左键是否按下
108     * @return 是否按下
109     */
110    virtual bool isMouseLeftPressed() const = 0;
111
112    /**
113     * @brief 鼠标右键是否按下
```

```
114     * @return 是否按下
115     */
116     virtual bool isMouseRightPressed() const = 0;
117
118     /**
119     * @brief 获取鼠标滚轮delta
120     * @return 滚轮增量
121     */
122     virtual int getMouseWheelDelta() const = 0;
123
124     /**
125     * @brief 获取移动方向向量
126     * @return 归一化的方向向量
127     *
128     * 综合键盘输入，返回一个归一化的移动方向。
129     * 这样斜向移动不会比横向快。
130     */
131     virtual QPointF getMoveDirection() const = 0;
132
133     /**
134     * @brief 设置按键映射
135     * @param action 游戏动作
136     * @param key Qt按键码
137     */
138     virtual void setKeyMapping(InputAction action, Qt::Key key) = 0;
139
140     /**
141     * @brief 获取按键映射
142     * @param action 游戏动作
143     * @return Qt按键码
144     */
145     virtual Qt::Key getKeyMapping(InputAction action) const = 0;
146
147     /**
148     * @brief 重置为默认按键映射
149     */
150     virtual void resetDefaultMapping() = 0;
151
152     /**
153     * @brief 启用/禁用输入
154     * @param enabled 是否启用
155     *
156     * 过场动画、对话时可以禁用输入。
157     */
158     virtual void setInputEnabled(bool enabled) = 0;
159
160     /**
```

```

161      * @brief 输入是否启用
162      * @return 是否启用
163      */
164      virtual bool isEnabled() const = 0;
165  };
166
167  #endif // IINPUTSERVICE_H

```

2.6.2 输入服务实现

文件：src/services/input/IService.h

代码块

```

1  #ifndef INPUTSERVICE_H
2  #define INPUTSERVICE_H
3
4  #include <QObject>
5  #include <QKeyEvent>
6  #include <QMouseEvent>
7  #include <QWheelEvent>
8  #include <QMap>
9  #include <QSet>
10
11 #include "IService.h"
12
13 // 前向声明
14 class IRenderService;
15
16 /**
17  * @brief 输入服务实现
18  *
19  * 处理键盘和鼠标输入，管理按键状态。
20  *
21  * 工作原理：
22  * 1. 安装事件过滤器，捕获所有按键和鼠标事件
23  * 2. 维护每个按键的状态（按下/释放）
24  * 3. 每帧更新"刚按下"和"刚释放"的状态
25  * 4. 提供查询接口给游戏逻辑使用
26  *
27  * 为什么用事件过滤器而不是重写keyPressEvent?
28  * - 可以在任何地方安装，不用继承QWidget
29  * - 可以同时监听多个窗口的输入
30  * - 更灵活
31  */
32 class IService : public QObject, public IInputService
33 {

```

```
34     Q_OBJECT
35
36 public:
37     explicit InputService(QObject *parent = nullptr);
38     ~InputService() override;
39
40     // IInputService接口实现
41     bool isActionPressed(InputAction action) const override;
42     bool isActionJustPressed(InputAction action) const override;
43     bool isActionJustReleased(InputAction action) const override;
44
45     QPoint getMousePosition() const override { return m_mousePos; }
46     QPointF getMouseScenePosition() const override;
47     bool isMouseLeftPressed() const override { return m_mouseLeftPressed; }
48     bool isMouseRightPressed() const override { return m_mouseRightPressed; }
49     int getMouseWheelDelta() const override { return m_mouseWheelDelta; }
50
51     QPointF getMoveDirection() const override;
52
53     void setKeyMapping(InputAction action, Qt::Key key) override;
54     Qt::Key getKeyMapping(InputAction action) const override;
55     void resetDefaultMapping() override;
56
57     void setInputEnabled(bool enabled) override { m_inputEnabled = enabled; }
58     bool isInputEnabled() const override { return m_inputEnabled; }
59
60     /**
61      * @brief 设置渲染服务（用于鼠标坐标转换）
62      * @param renderService 渲染服务指针
63      */
64     void setRenderService(IRenderService* renderService) { m_renderService =
renderService; }
65
66     /**
67      * @brief 每帧更新（必须在游戏循环中调用）
68      *
69      * 更新"刚按下"和"刚释放"的状态。
70      * 这些状态只在一帧内有效。
71      */
72     void update();
73
74 protected:
75     // 事件过滤器，捕获所有输入事件
76     bool eventFilter(QObject* obj, QEvent* event) override;
77
78 private:
79     // 渲染服务（用于坐标转换）
```

```

80     IRenderService* m_renderService = nullptr;
81
82     // 输入是否启用
83     bool m_inputEnabled = true;
84
85     // 按键映射：动作 -> 按键
86     QMap<InputAction, Qt::Key> m_keyMapping;
87
88     // 当前按下的按键
89     QSet<Qt::Key> m_pressedKeys;
90
91     // 这一帧刚按下的按键
92     QSet<Qt::Key> m_justPressedKeys;
93
94     // 这一帧刚释放的按键
95     QSet<Qt::Key> m_justReleasedKeys;
96
97     // 鼠标状态
98     QPoint m_mousePos;
99     bool m_mouseLeftPressed = false;
100    bool m_mouseRightPressed = false;
101    int m_mouseWheelDelta = 0;
102
103    // 上一帧的鼠标滚轮delta（用于清零）
104    int m_lastWheelDelta = 0;
105
106    /**
107     * @brief 初始化默认按键映射
108     */
109    void initDefaultMapping();
110
111    /**
112     * @brief 处理按键按下
113     */
114    void handleKeyPress(Qt::Key key);
115
116    /**
117     * @brief 处理按键释放
118     */
119    void handleKeyRelease(Qt::Key key);
120 };
121
122 #endif // INPUTSERVICE_H

```

文件：src/services/input/InputService.cpp

```

1 代码块#include "InputService.h"
2  #include "IRenderService.h"
3
4  #include <QDebug>
5  #include <QEvent>
6  #include <QKeyEvent>
7  #include <QMouseEvent>
8  #include <QWheelEvent>
9  #include <QApplication>
10
11 InputService::InputService(QObject *parent)
12     : QObject(parent)
13 {
14     // 初始化默认按键映射
15     initDefaultMapping();
16
17     // 安装事件过滤器到应用程序
18     // 这样可以捕获所有窗口的输入事件
19     qApp->installEventFilter(this);
20
21     qDebug() << "[InputService] 输入服务初始化完成";
22 }
23
24 InputService::~InputService()
25 {
26     qApp->removeEventFilter(this);
27     qDebug() << "[InputService] 输入服务已销毁";
28 }
29
30 bool InputService::isActionPressed(InputAction action) const
31 {
32     if (!m_inputEnabled) return false;
33
34     Qt::Key key = m_keyMapping.value(action, Qt::Key_unknown);
35     return m_pressedKeys.contains(key);
36 }
37
38 bool InputService::isActionJustPressed(InputAction action) const
39 {
40     if (!m_inputEnabled) return false;
41
42     Qt::Key key = m_keyMapping.value(action, Qt::Key_unknown);
43     return m_justPressedKeys.contains(key);
44 }
45
46 bool InputService::isActionJustReleased(InputAction action) const
47 {

```

```

48     if (!m_inputEnabled) return false;
49
50     Qt::Key key = m_keyMapping.value(action, Qt::Key_unknown);
51     return m_justReleasedKeys.contains(key);
52 }
53
54 QPointF InputService::getMouseScenePosition() const
55 {
56     if (m_renderService) {
57         return m_renderService->mapToScene(m_mousePos);
58     }
59     return QPointF(m_mousePos);
60 }
61
62 QPointF InputService::getMoveDirection() const
63 {
64     if (!m_inputEnabled) {
65         return QPointF(0, 0);
66     }
67
68     qreal x = 0;
69     qreal y = 0;
70
71     if (isActionPressed(InputAction::MoveLeft)) {
72         x -= 1;
73     }
74     if (isActionPressed(InputAction::MoveRight)) {
75         x += 1;
76     }
77     if (isActionPressed(InputAction::MoveUp)) {
78         y -= 1;
79     }
80     if (isActionPressed(InputAction::MoveDown)) {
81         y += 1;
82     }
83
84     // 归一化（斜向移动速度不会更快）
85     QPointF dir(x, y);
86     qreal length = sqrt(x * x + y * y);
87     if (length > 0) {
88         dir /= length;
89     }
90
91     return dir;
92 }
93
94 void InputService::setKeyMapping(InputAction action, Qt::Key key)

```

```
95  {
96      m_keyMapping[action] = key;
97      qDebug() << "[InputService] 按键映射更新:"
98              << static_cast<int>(action)
99              << "->" << key;
100 }
101
102 Qt::Key InputService::getKeyMapping(InputAction action) const
103 {
104     return m_keyMapping.value(action, Qt::Key_unknown);
105 }
106
107 void InputService::resetDefaultMapping()
108 {
109     initDefaultMapping();
110     qDebug() << "[InputService] 已重置为默认按键映射";
111 }
112
113 void InputService::update()
114 {
115     // 清空"刚按下"和"刚释放"的状态
116     // 这些状态只在一帧内有效
117     m_justPressedKeys.clear();
118     m_justReleasedKeys.clear();
119
120     // 鼠标滚轮delta也要清零（滚轮事件是瞬时的）
121     m_mouseWheelDelta = 0;
122 }
123
124 bool InputService::eventFilter(QObject* obj, QEvent* event)
125 {
126     Q_UNUSED(obj);
127
128     if (!m_inputEnabled) {
129         return false; // 不消费事件，让事件继续传递
130     }
131
132     switch (event->type()) {
133     case QEvent::KeyPress: {
134         QKeyEvent* keyEvent = static_cast<QKeyEvent*>(event);
135         if (!keyEvent->isAutoRepeat()) {
136             handleKeyPress(static_cast<Qt::Key>(keyEvent->key()));
137         }
138         break;
139     }
140
141     case QEvent::KeyRelease: {
```

```

142         QKeyEvent* keyEvent = static_cast<QKeyEvent*>(event);
143         if (!keyEvent->isAutoRepeat()) {
144             handleKeyRelease(static_cast<Qt::Key>(keyEvent->key()));
145         }
146         break;
147     }
148
149     case QEvent::MouseMove: {
150         QMouseEvent* mouseEvent = static_cast<QMouseEvent*>(event);
151         m_mousePos = mouseEvent->pos();
152         break;
153     }
154
155     case QEvent::MouseButtonPress: {
156         QMouseEvent* mouseEvent = static_cast<QMouseEvent*>(event);
157         if (mouseEvent->button() == Qt::LeftButton) {
158             m_mouseLeftPressed = true;
159         } else if (mouseEvent->button() == Qt::RightButton) {
160             m_mouseRightPressed = true;
161         }
162         break;
163     }
164
165     case QEvent::MouseButtonRelease: {
166         QMouseEvent* mouseEvent = static_cast<QMouseEvent*>(event);
167         if (mouseEvent->button() == Qt::LeftButton) {
168             m_mouseLeftPressed = false;
169         } else if (mouseEvent->button() == Qt::RightButton) {
170             m_mouseRightPressed = false;
171         }
172         break;
173     }
174
175     case QEvent::Wheel: {
176         QWheelEvent* wheelEvent = static_cast<QWheelEvent*>(event);
177         m_mouseWheelDelta = wheelEvent->angleDelta().y();
178         break;
179     }
180
181     default:
182         break;
183 }
184
185 return false; // 不消费事件，让事件继续传递给目标对象
186 }
187
188 void InputService::initDefaultMapping()

```

```

189 {
190     m_keyMapping.clear();
191
192     // 默认按键映射（类魂类游戏的键位）
193     m_keyMapping[InputAction::MoveLeft] = Qt::Key_A;
194     m_keyMapping[InputAction::MoveRight] = Qt::Key_D;
195     m_keyMapping[InputAction::MoveUp] = Qt::Key_W;
196     m_keyMapping[InputAction::MoveDown] = Qt::Key_S;
197
198     m_keyMapping[InputAction::Jump] = Qt::Key_Space;
199
200     m_keyMapping[InputAction::Attack] = Qt::Key_J;
201     m_keyMapping[InputAction::Skill1] = Qt::Key_K;
202     m_keyMapping[InputAction::Skill2] = Qt::Key_L;
203
204     m_keyMapping[InputAction::Interact] = Qt::Key_E;
205
206     m_keyMapping[InputAction::Pause] = Qt::Key_Escape;
207     m_keyMapping[InputAction::Inventory] = Qt::Key_I;
208     m_keyMapping[InputAction::Map] = Qt::Key_M;
209
210     m_keyMapping[InputAction::SwitchFormNext] = Qt::Key_Q;
211     m_keyMapping[InputAction::SwitchFormPrev] = Qt::Key_0; // 数字0
212
213     m_keyMapping[InputAction::PartnerSkill] = Qt::Key_R;
214
215     m_keyMapping[InputAction::TimeRewind] = Qt::Key_Shift;
216     m_keyMapping[InputAction::TimeVision] = Qt::Key_Tab;
217 }
218
219 void InputService::handleKeyPress(Qt::Key key)
220 {
221     if (!m_pressedKeys.contains(key)) {
222         m_pressedKeys.insert(key);
223         m_justPressedKeys.insert(key);
224     }
225 }
226
227 void InputService::handleKeyRelease(Qt::Key key)
228 {
229     if (m_pressedKeys.contains(key)) {
230         m_pressedKeys.remove(key);
231         m_justReleasedKeys.insert(key);
232     }
233 }

```

2.7 第七步：基础服务 - 音频服务

音频服务管理背景音乐和音效。

2.7.1 音频服务接口

文件：src/core/services/IAudioService.h

代码块

```
1  #ifndef IAUDIOSERVICE_H
2  #define IAUDIOSERVICE_H
3
4  #include <QObject>
5  #include <QString>
6
7  /**
8   * @brief 音频服务接口
9   *
10  * 管理游戏中的所有音频：背景音乐、音效、语音等。
11  *
12  * 音频类型：
13  * - BGM：背景音乐，长音频，循环播放
14  * - SFX：音效，短音频，一次性播放
15  * - Voice：语音，对话等
16  *
17  * 音量控制：
18  * - 主音量：控制所有音频
19  * - 音乐音量：只控制BGM
20  * - 音效音量：只控制SFX
21  * - 语音音量：只控制语音
22  */
23 class IAudioService
24 {
25 public:
26     virtual ~IAudioService() = default;
27
28     /**
29      * @brief 播放背景音乐
30      * @param name 音乐文件名
31      * @param loop 是否循环（默认循环）
32      *
33      * 如果已经在播放音乐，会平滑过渡到新音乐。
34      */
35     virtual void playBGM(const QString& name, bool loop = true) = 0;
36
```

```

37     /**
38      * @brief 停止背景音乐
39      * @param fadeOutTime 淡出时间 (毫秒)
40      */
41     virtual void stopBGM(int fadeOutTime = 1000) = 0;
42
43     /**
44      * @brief 暂停背景音乐
45      */
46     virtual void pauseBGM() = 0;
47
48     /**
49      * @brief 恢复背景音乐
50      */
51     virtual void resumeBGM() = 0;
52
53     /**
54      * @brief 播放音效
55      * @param name 音效文件名
56      * @param volume 音量 (0.0 - 1.0)
57      * @param loop 循环次数 (-1为无限循环)
58      * @return 音效实例ID, 用于控制
59      */
60     virtual int playSFX(const QString& name, float volume = 1.0f, int loop =
61     0) = 0;
62
63     /**
64      * @brief 停止音效
65      * @param sfxId 音效实例ID
66      */
67     virtual void stopSFX(int sfxId) = 0;
68
69     /**
70      * @brief 停止所有音效
71      */
72     virtual void stopAllSFX() = 0;
73
74     /**
75      * @brief 设置主音量
76      * @param volume 音量 (0.0 - 1.0)
77      */
78     virtual void setMasterVolume(float volume) = 0;
79
80     /**
81      * @brief 获取主音量
82      * @return 音量
83      */

```

```

83     virtual float getMasterVolume() const = 0;
84
85     /**
86      * @brief 设置音乐音量
87      * @param volume 音量
88      */
89     virtual void setMusicVolume(float volume) = 0;
90
91     /**
92      * @brief 获取音乐音量
93      * @return 音量
94      */
95     virtual float getMusicVolume() const = 0;
96
97     /**
98      * @brief 设置音效音量
99      * @param volume 音量
100     */
101     virtual void setSFXVolume(float volume) = 0;
102
103     /**
104      * @brief 获取音效音量
105      * @return 音量
106      */
107     virtual float getSFXVolume() const = 0;
108
109     /**
110      * @brief 设置静音
111      * @param muted 是否静音
112      */
113     virtual void setMuted(bool muted) = 0;
114
115     /**
116      * @brief 是否静音
117      * @return 是否静音
118      */
119     virtual bool isMuted() const = 0;
120 };
121
122 #endif // IAUDIOSERVICE_H

```

注意：由于篇幅限制，音频服务、状态机服务、存档服务的完整实现我会在后续代码中继续给出。这里先给出接口，让大家了解整体架构。

2.8 第八步：基础服务 - 状态机服务

基于QStateMachine的游戏状态管理。

2.8.1 状态机服务接口

文件：src/core/services/IStateMachineService.h

代码块

```
1  #ifndef ISTATEMACHINESERVICE_H
2  #define ISTATEMACHINESERVICE_H
3
4  #include <QObject>
5  #include <QStateMachine>
6  #include <QState>
7  #include <QString>
8
9  /**
10   * @brief 游戏状态枚举
11   *
12   * 游戏的主要状态。
13   * 每个状态有独立的逻辑和渲染。
14   */
15  enum class GameState
16  {
17      None,           // 无状态（初始）
18      MainMenu,       // 主菜单
19      Loading,        // 加载中
20      Playing,        // 游戏中
21      Paused,         // 暂停
22      Dialog,         // 对话中
23      Cutscene,       // 过场动画
24      GameOver,       // 游戏结束
25      Victory         // 胜利
26  };
27
28  /**
29   * @brief 状态机服务接口
30   *
31   * 管理游戏的全局状态。
32   *
33   * 为什么用状态机？
34   * - 状态转换清晰，不会出现混乱的状态
35   * - 每个状态有明确的进入和退出逻辑
36   * - Qt的状态机框架支持可视化调试
37   * - 支持子状态、并行状态、历史状态等高级特性
38   */
```

```
39 class IStateMachineService
40 {
41 public:
42     virtual ~IStateMachineService() = default;
43
44     /**
45      * @brief 获取当前状态
46      * @return 当前游戏状态
47      */
48     virtual GameState getCurrentState() const = 0;
49
50     /**
51      * @brief 切换状态
52      * @param state 目标状态
53      *
54      * 会触发当前状态的退出逻辑和目标状态的进入逻辑。
55      */
56     virtual void changeState(GameState state) = 0;
57
58     /**
59      * @brief 注册状态
60      * @param state 状态枚举
61      * @param stateObj 状态对象
62      *
63      * 插件可以注册自定义状态。
64      */
65     virtual void registerState(GameState state, QState* stateObj) = 0;
66
67     /**
68      * @brief 获取状态对象
69      * @param state 状态枚举
70      * @return 状态对象指针
71      */
72     virtual QState* getState(GameState state) const = 0;
73
74     /**
75      * @brief 获取状态机
76      * @return 状态机指针
77      *
78      * 高级用法，可以直接操作状态机。
79      */
80     virtual QStateMachine* getStateMachine() const = 0;
81
82     /**
83      * @brief 启动状态机
84      */
85     virtual void start() = 0;
```

```
86
87     /**
88     * @brief 停止状态机
89     */
90     virtual void stop() = 0;
91 };
92
93 #endif // ISTATEMACHINESERVICE_H
```

2.9 第九步：基础服务 - 存档服务

管理游戏存档的读取和写入。

2.9.1 存档服务接口

文件：src/core/services/ISaveService.h

代码块

```
1  #ifndef ISAVESERVICE_H
2  #define ISAVESERVICE_H
3
4  #include <QObject>
5  #include <QString>
6  #include <QVariantMap>
7  #include <QDateTime>
8
9  /**
10   * @brief 存档信息结构体
11   */
12   struct SaveInfo
13   {
14       int slot = -1;           // 存档槽位
15       QString name;           // 存档名称
16       QDateTime timestamp;    // 保存时间
17       QString playTime;       // 游戏时长
18       QString currentLayer;   // 当前层级
19       int playerLevel = 1;    // 玩家等级
20       QPixmap thumbnail;      // 缩略图
21       bool isValid = false;   // 是否有效
22   };
23
24   /**
25   * @brief 存档服务接口
26   *
```

```

27  * 管理游戏存档的保存和加载。
28  *
29  * 存档格式：
30  * - 二进制格式（QDataStream），体积小，加载快
31  * - 带校验和，防止篡改
32  * - 支持多存档槽位
33  * - 支持自动存档和快速存档
34  *
35  * 存档内容：
36  * - 玩家状态（位置、血量、能量等）
37  * - 已解锁的能力和装备
38  * - 世界状态（已探索区域、已完成任务等）
39  * - 游戏设置（音量、按键映射等）
40  */
41 class ISaveService
42 {
43 public:
44     virtual ~ISaveService() = default;
45
46     /**
47      * @brief 保存游戏
48      * @param slot 存档槽位（0-9）
49      * @param name 存档名称
50      * @return 是否保存成功
51      */
52     virtual bool saveGame(int slot, const QString& name = QString()) = 0;
53
54     /**
55      * @brief 加载游戏
56      * @param slot 存档槽位
57      * @return 是否加载成功
58      */
59     virtual bool loadGame(int slot) = 0;
60
61     /**
62      * @brief 获取存档信息
63      * @param slot 存档槽位
64      * @return 存档信息
65      */
66     virtual SaveInfo getSaveInfo(int slot) const = 0;
67
68     /**
69      * @brief 获取所有存档信息
70      * @return 存档信息列表
71      */
72     virtual QList<SaveInfo> getAllSaveInfos() const = 0;
73

```

```
74     /**
75      * @brief 删除存档
76      * @param slot 存档槽位
77      * @return 是否删除成功
78      */
79     virtual bool deleteSave(int slot) = 0;
80
81     /**
82      * @brief 快速保存
83      * @return 是否保存成功
84      *
85      * 保存到快速存档槽位（槽位0）。
86      */
87     virtual bool quickSave() = 0;
88
89     /**
90      * @brief 快速读取
91      * @return 是否加载成功
92      */
93     virtual bool quickLoad() = 0;
94
95     /**
96      * @brief 自动保存
97      * @return 是否保存成功
98      *
99      * 保存到自动存档槽位（槽位9）。
100     * 游戏会在特定时机自动调用。
101     */
102     virtual bool autoSave() = 0;
103
104     /**
105      * @brief 设置游戏数据
106      * @param key 键
107      * @param value 值
108      *
109      * 各系统通过这个接口把数据存入存档。
110      */
111     virtual void setSaveData(const QString& key, const QVariant& value) = 0;
112
113     /**
114      * @brief 获取游戏数据
115      * @param key 键
116      * @param defaultValue 默认值
117      * @return 值
118      */
119     virtual QVariant getSaveData(const QString& key, const QVariant&
defaultValue = QVariant()) const = 0;
```

```

120
121     /**
122     * @brief 获取存档目录
123     * @return 存档目录路径
124     */
125     virtual QString getSaveDirectory() const = 0;
126 };
127
128 #endif // ISAVESERVICE_H

```

2.7.2 音频服务实现

文件：src/services/audio/AudioService.h

代码块

```

1  #ifndef AUDIOSERVICE_H
2  #define AUDIOSERVICE_H
3
4  #include <QObject>
5  #include <QMediaPlayer>
6  #include <QAudioOutput>
7  #include <QSoundEffect>
8  #include <QMap>
9  #include <QTimer>
10
11 #include "IAudioService.h"
12 #include "ResourceManager.h"
13
14 /**
15  * @brief 音频服务实现
16  *
17  * 基于Qt Multimedia的音频服务。
18  *
19  * 设计要点：
20  * - BGM用QMediaPlayer（支持长音频、流式播放）
21  * - SFX用QSoundEffect（低延迟，适合短音效）
22  * - 音量分层控制：主音量 × 分类音量
23  * - 音乐淡入淡出，切换不突兀
24  * - 音效对象池，避免频繁创建销毁
25  */
26 class AudioService : public QObject, public IAudioService
27 {
28     Q_OBJECT
29

```

```
30 public:
31     explicit AudioService(QObject *parent = nullptr);
32     ~AudioService() override;
33
34     // IAudioService接口实现
35     void playBGM(const QString& name, bool loop = true) override;
36     void stopBGM(int fadeOutTime = 1000) override;
37     void pauseBGM() override;
38     void resumeBGM() override;
39
40     int playSFX(const QString& name, float volume = 1.0f, int loop = 0)
override;
41     void stopSFX(int sfxId) override;
42     void stopAllSFX() override;
43
44     void setMasterVolume(float volume) override;
45     float getMasterVolume() const override { return m_masterVolume; }
46
47     void setMusicVolume(float volume) override;
48     float getMusicVolume() const override { return m_musicVolume; }
49
50     void setSFXVolume(float volume) override;
51     float getSFXVolume() const override { return m_sfxVolume; }
52
53     void setMuted(bool muted) override;
54     bool isMuted() const override { return m_muted; }
55
56 private slots:
57     /// BGM状态变化
58     void onBGMMediaStatusChanged(QMediaPlayer::MediaStatus status);
59
60     /// BGM播放位置变化（用于淡入淡出）
61     void onBGMVolumeFadeTick();
62
63 private:
64     // 资源管理器
65     ResourceManager* m_resourceManager = nullptr;
66
67     // BGM播放器
68     QMediaPlayer* m_bgmPlayer = nullptr;
69     QAudioOutput* m_bgmAudioOutput = nullptr;
70     QString m_currentBGM;
71
72     // 音量
73     float m_masterVolume = 1.0f;
74     float m_musicVolume = 0.8f;
75     float m_sfxVolume = 1.0f;
```

```

76     bool m_muted = false;
77
78     // 音效池
79     struct SFXInstance
80     {
81         int id;
82         QSoundEffect* effect;
83         QString name;
84         float baseVolume;
85     };
86
87     QList<SFXInstance> m_activeSFX;
88     int m_nextSFXId = 1;
89
90     // 淡入淡出
91     QTimer* m_fadeTimer = nullptr;
92     float m_fadeTargetVolume = 0.0f;
93     float m_fadeSpeed = 0.0f;
94     bool m_fadeIn = false;
95     QString m_pendingBGM; // 淡出后要播放的新BGM
96
97     /**
98      * @brief 更新实际音量
99      *
100     * 根据主音量、分类音量、静音状态计算实际音量。
101     */
102     void updateActualVolumes();
103
104     /**
105     * @brief 开始淡入淡出
106     * @param targetVolume 目标音量
107     * @param duration 持续时间（毫秒）
108     */

```

代码块

```

1     */
2     void startFade(float targetVolume, int duration);
3
4     /**
5     * @brief 获取音效文件路径
6     */
7     QString getSoundPath(const QString& name) const;
8
9     /**
10    * @brief 获取音乐文件路径

```

```
11     */
12     QString getMusicPath(const QString& name) const;
```

```
};
```

```
#endif // AUDIOSERVICE_H
```

代码块

```
1
2  **文件: src/services/audio/AudioService.cpp**
3  ```cpp
4  #include "AudioService.h"
5
6  #include <QDebug>
7  #include <QDir>
8  #include <QCoreApplication>
9
10 AudioService::AudioService(QObject *parent)
11     : QObject(parent)
12 {
13     m_resourceManager = ResourceManager::instance();
14
15     // 创建BGM播放器
16     m_bgmPlayer = new QMediaPlayer(this);
17     m_bgmAudioOutput = new QAudioOutput(this);
18     m_bgmPlayer->setAudioOutput(m_bgmAudioOutput);
19
20     connect(m_bgmPlayer, &QMediaPlayer::mediaStatusChanged,
21            this, &AudioService::onBGMMediaStatusChanged);
22
23     // 创建淡入淡出定时器
24     m_fadeTimer = new QTimer(this);
25     m_fadeTimer->setInterval(50); // 20fps的平滑度足够了
26     connect(m_fadeTimer, &QTimer::timeout,
27            this, &AudioService::onBGMVolumeFadeTick);
28
29     // 更新初始音量
30     updateActualVolumes();
31
32     qDebug() << "[AudioService] 音频服务初始化完成";
33 }
34
35 AudioService::~AudioService()
36 {
37     stopAllSFX();
38     qDebug() << "[AudioService] 音频服务已销毁";
```

```
39 }
40
41 void AudioService::playBGM(const QString& name, bool loop)
42 {
43     if (m_muted) {
44         m_pendingBGM = name; // 静音时也记住要播放的音乐
45         return;
46     }
47
48     QString filePath = getMusicPath(name);
49     if (filePath.isEmpty()) {
50         qWarning() << "[AudioService] BGM文件未找到:" << name;
51         return;
52     }
53
54     // 如果正在播放同一首，直接返回
55     if (m_currentBGM == name && m_bgmPlayer->playbackState() ==
QMediaPlayer::PlayingState) {
56         return;
57     }
58
59     // 如果正在播放其他音乐，先淡出再切换
60     if (m_bgmPlayer->playbackState() == QMediaPlayer::PlayingState &&
!m_fadeTimer->isActive()) {
61         m_pendingBGM = name;
62         startFade(0.0f, 1000); // 1秒淡出
63         return;
64     }
65
66     // 直接播放
67     m_bgmPlayer->setSource(QUrl::fromLocalFile(filePath));
68     m_bgmPlayer->setLoops(loop ? QMediaPlayer::Infinite : 1);
69     m_bgmPlayer->play();
70
71     m_currentBGM = name;
72
73     // 淡入
74     m_bgmAudioOutput->setVolume(0);
75     startFade(m_musicVolume * m_masterVolume, 1000);
76
77     qDebug() << "[AudioService] 播放BGM:" << name;
78 }
79
80 void AudioService::stopBGM(int fadeOutTime)
81 {
82     if (m_bgmPlayer->playbackState() == QMediaPlayer::StoppedState) {
83         return;
```

```
84     }
85
86     if (fadeOutTime > 0) {
87         // 淡出后停止
88         m_pendingBGM.clear();
89         startFade(0.0f, fadeOutTime);
90     } else {
91         // 立即停止
92         m_bgmPlayer->stop();
93         m_currentBGM.clear();
94     }
95 }
96
97 void AudioService::pauseBGM()
98 {
99     if (m_bgmPlayer->playbackState() == QMediaPlayer::PlayingState) {
100         m_bgmPlayer->pause();
101         qDebug() << "[AudioService] BGM已暂停";
102     }
103 }
104
105 void AudioService::resumeBGM()
106 {
107     if (m_bgmPlayer->playbackState() == QMediaPlayer::PausedState) {
108         m_bgmPlayer->play();
109         qDebug() << "[AudioService] BGM已恢复";
110     }
111 }
112
113 int AudioService::playSFX(const QString& name, float volume, int loop)
114 {
115     if (m_muted) {
116         return -1;
117     }
118
119     QString filePath = getSoundPath(name);
120     if (filePath.isEmpty()) {
121         qWarning() << "[AudioService] 音效文件未找到:" << name;
122         return -1;
123     }
124
125     // 创建新的音效实例
126     QSoundEffect* effect = new QSoundEffect(this);
127     effect->setSource(QUrl::fromLocalFile(filePath));
128     effect->setVolume(volume * m_sfxVolume * m_masterVolume);
129     effect->setLoopCount(loop);
130     effect->play();
```

```
131
132     // 记录
133     SFXInstance instance;
134     instance.id = m_nextSFXId++;
135     instance.effect = effect;
136     instance.name = name;
137     instance.baseVolume = volume;
138
139     m_activeSFX.append(instance);
140
141     // 音效播放完后自动清理
142     // 注意：QSoundEffect没有finished信号，我们用定时器检查
143     // 或者用QMediaPlayer来做，但延迟会高一些
144     // 对于短音效，手动管理更高效
145
146     qDebug() << "[AudioService] 播放音效:" << name << "ID:" << instance.id;
147
148     return instance.id;
149 }
150
151 void AudioService::stopSFX(int sfxId)
152 {
153     for (int i = 0; i < m_activeSFX.size(); ++i) {
154         if (m_activeSFX[i].id == sfxId) {
155             m_activeSFX[i].effect->stop();
156             m_activeSFX[i].effect->deleteLater();
157             m_activeSFX.removeAt(i);
158             return;
159         }
160     }
161 }
162
163 void AudioService::stopAllSFX()
164 {
165     for (const SFXInstance& instance : m_activeSFX) {
166         instance.effect->stop();
167         instance.effect->deleteLater();
168     }
169     m_activeSFX.clear();
170 }
171
172 void AudioService::setMasterVolume(float volume)
173 {
174     m_masterVolume = qBound(0.0f, volume, 1.0f);
175     updateActualVolumes();
176     qDebug() << "[AudioService] 主音量设置为:" << m_masterVolume;
177 }
```

```
178
179 void AudioService::setMusicVolume(float volume)
180 {
181     m_musicVolume = qBound(0.0f, volume, 1.0f);
182     updateActualVolumes();
183     qDebug() << "[AudioService] 音乐音量设置为:" << m_musicVolume;
184 }
185
186 void AudioService::setSFXVolume(float volume)
187 {
188     m_sfxVolume = qBound(0.0f, volume, 1.0f);
189     updateActualVolumes();
190     qDebug() << "[AudioService] 音效音量设置为:" << m_sfxVolume;
191 }
192
193 void AudioService::setMuted(bool muted)
194 {
195     m_muted = muted;
196     updateActualVolumes();
197
198     if (muted) {
199         pauseBGM();
200         stopAllSFX();
201     } else {
202         resumeBGM();
203     }
204
205     qDebug() << "[AudioService] 静音:" << (muted ? "开启" : "关闭");
206 }
207
208 void AudioService::onBGMMediaStatusChanged(QMediaPlayer::MediaStatus status)
209 {
210     if (status == QMediaPlayer::LoadedMedia) {
211         // 媒体加载完成
212     } else if (status == QMediaPlayer::EndOfMedia) {
213         // 播放结束
214         if (m_bgmPlayer->loops() == 1) {
215             m_currentBGM.clear();
216         }
217     }
218 }
219
220 void AudioService::onBGMVolumeFadeTick()
221 {
222     float currentVolume = m_bgmAudioOutput->volume();
223
224     if (m_fadeIn) {
```

```
225         currentVolume += m_fadeSpeed;
226         if (currentVolume >= m_fadeTargetVolume) {
227             currentVolume = m_fadeTargetVolume;
228             m_fadeTimer->stop();
229         }
230     } else {
231         currentVolume -= m_fadeSpeed;
232         if (currentVolume <= m_fadeTargetVolume) {
233             currentVolume = m_fadeTargetVolume;
234             m_fadeTimer->stop();
235
236             // 淡出完成后, 如果有等待播放的新BGM
237             if (!m_pendingBGM.isEmpty()) {
238                 QString nextBGM = m_pendingBGM;
239                 m_pendingBGM.clear();
240
241                 // 停止当前音乐, 播放新的
242                 m_bgmPlayer->stop();
243                 m_currentBGM.clear();
244
245                 // 延迟一帧再播放, 避免状态问题
246                 QTimer::singleShot(50, this, [this, nextBGM]() {
247                     playBGM(nextBGM);
248                 });
249                 return;
250             }
251
252             // 如果淡出到0, 停止播放
253             if (m_fadeTargetVolume <= 0.0f) {
254                 m_bgmPlayer->stop();
255                 m_currentBGM.clear();
256             }
257         }
258     }
259
260     m_bgmAudioOutput->setVolume(currentVolume);
261 }
262
263 void AudioService::updateActualVolumes()
264 {
265     if (m_muted) {
266         m_bgmAudioOutput->setVolume(0);
267         for (const SFXInstance& instance : m_activeSFX) {
268             instance.effect->setVolume(0);
269         }
270     } else {
271         // BGM音量
```

```
272         float actualMusicVolume = m_musicVolume * m_masterVolume;
273         // 注意：如果正在淡入淡出，不要直接改，让fade定时器控制
274         if (!m_fadeTimer->isActive()) {
275             m_bgmAudioOutput->setVolume(actualMusicVolume);
276         }
277
278         // SFX音量
279         float actualSFXVolume = m_sfxVolume * m_masterVolume;
280         for (const SFXInstance& instance : m_activeSFX) {
281             instance.effect->setVolume(instance.baseVolume * actualSFXVolume);
282         }
283     }
284 }
285
286 void AudioService::startFade(float targetVolume, int duration)
287 {
288     m_fadeTargetVolume = targetVolume;
289
290     float currentVolume = m_bgmAudioOutput->volume();
291     m_fadeIn = targetVolume > currentVolume;
292
293     // 计算每帧变化量
294     int steps = duration / 50; // 50ms一帧
295     if (steps <= 0) steps = 1;
296     m_fadeSpeed = abs(targetVolume - currentVolume) / steps;
297
298     m_fadeTimer->start();
299 }
300
301 QString AudioService::getSoundPath(const QString& name) const
302 {
303     // 先通过资源管理器找
304     QString path = m_resourceManager->getResourcePath(name);
305     if (!path.isEmpty()) {
306         return path;
307     }
308
309     // 再在音乐目录找
310     QString appDir = QApplication::applicationDirPath();
311     QString soundDir = appDir + "/resources/sounds/";
312
313     QString fullPath = soundDir + name;
314     if (QFile::exists(fullPath)) {
315         return fullPath;
316     }
317
318     return QString();

```

```

319     }
320
321     QString AudioService::getMusicPath(const QString& name) const
322     {
323         QString appDir = QApplication::applicationDirPath();
324         QString musicDir = appDir + "/resources/music/";
325
326         QString fullPath = musicDir + name;
327         if (QFile::exists(fullPath)) {
328             return fullPath;
329         }
330
331         // 也检查qrc资源
332         QString qrcPath = ":/music/" + name;
333         if (QFile::exists(qrcPath)) {
334             return qrcPath;
335         }
336
337         return QString();
338     }

```

2.8 第八步：基础服务 - 状态机服务（完整实现）

2.8.1 状态机服务实现

文件：src/services/statemachine/StateMachineService.h

代码块

```

1  #ifndef STATEMACHINESERVICE_H
2  #define STATEMACHINESERVICE_H
3
4  #include <QObject>
5  #include <QStateMachine>
6  #include <QState>
7  #include <QFinalState>
8  #include <QSignalTransition>
9  #include <QMap>
10
11 #include "IStateMachineService.h"
12 #include "EventBus.h"
13
14 /**
15  * @brief 状态机服务实现
16  *

```

17 * 基于QStateMachine的游戏状态管理。

18 *

19 * 状态图：

20 *

21 *



39 *

40 * Playing状态还有子状态：

41 * - Exploring (探索)

42 * - Combat (战斗)

43 * - Dialog (对话)

44 * - Cutscene (过场)

45 *

46 * Qt状态机的优势：

47 * 1. 可视化调试 (GammaRay可以看状态图)

48 * 2. 自动管理状态进入/退出

49 * 3. 支持历史状态 (回到子状态)

50 * 4. 支持并行状态

51 */

52 class StateMachineService : public QObject, public IStateMachineService

53 {

54 Q_OBJECT

55

56 public:

57 explicit StateMachineService(QObject *parent = nullptr);

58 ~StateMachineService() override;

59

60 // IStateMachineService接口实现

61 GameState getCurrentState() const override { return m_currentState; }

62 void changeState(GameState state) override;

63

```
64     void registerState(GameState state, QState* stateObj) override;
65     QState* getState(GameState state) const override;
66
67     QStateMachine* getStateMachine() const override { return m_stateMachine; }
68
69     void start() override;
70     void stop() override;
71
72 signals:
73     /**
74      * @brief 状态改变信号
75      * @param newState 新状态
76      *
77      * 各系统可以监听这个信号来响应状态变化。
78      */
79     void stateChanged(GameState newState);
80
81 private slots:
82     /// 状态进入时调用
83     void onStateEntered();
84
85 private:
86     // 状态机
87     QStateMachine* m_stateMachine = nullptr;
88
89     // 当前状态
90     GameState m_currentState = GameState::None;
91
92     // 状态映射
93     QMap<GameState, QState*> m_stateMap;
94
95     // 事件总线
96     EventBus* m_eventBus = nullptr;
97
98     /**
99      * @brief 创建默认状态
100     *
101     * 创建游戏的基础状态：主菜单、加载、游戏中、暂停等。
102     */
103     void createDefaultStates();
104
105     /**
106      * @brief 设置状态转换
107     *
108     * 定义状态之间的转换关系。
109     */
110     void setupTransitions();
```

```
111
112     /**
113     * @brief 状态枚举转字符串
114     */
115     QString stateToString(GameState state) const;
116 };
117
118 #endif // STATEMACHINESERVICE_H
```

文件：src/services/statemachine/StateMachineService.cpp

代码块

```
1  #include "StateMachineService.h"
2
3  #include <QDebug>
4  #include <QFinalState>
5  #include <QSignalTransition>
6
7  StateMachineService::StateMachineService(QObject *parent)
8      : QObject(parent)
9  {
10     m_eventBus = EventBus::instance();
11
12     // 创建状态机
13     m_stateMachine = new QStateMachine(this);
14     m_stateMachine->setGlobalRestorePolicy(QStateMachine::RestoreProperties);
15
16     // 创建默认状态
17     createDefaultStates();
18
19     // 设置状态转换
20     setupTransitions();
21
22     qDebug() << "[StateMachineService] 状态机服务初始化完成";
23 }
24
25 StateMachineService::~StateMachineService()
26 {
27     stop();
28     qDebug() << "[StateMachineService] 状态机服务已销毁";
29 }
30
31 void StateMachineService::changeState(GameState state)
32 {
33     QState* targetState = m_stateMap.value(state, nullptr);
```

```
34     if (!targetState) {
35         qWarning() << "[StateMachineService] 状态不存在:" << static_cast<int>
    (state);
36         return;
37     }
38
39     // 通过事件触发状态转换
40     // 这里我们直接用状态机的postEvent来触发
41     // 或者更简单：每个状态有一个进入的信号
42     // 为了简单，我们用一个自定义事件
43
44     // 实际上，QStateMachine的标准用法是：
45     // 定义状态和转换，然后通过信号触发转换
46     // 这里我们用一个更灵活的方式：直接设置当前状态
47
48     // 先停止状态机
49     m_stateMachine->stop();
50
51     // 设置初始状态
52     m_stateMachine->setInitialState(targetState);
53
54     // 重新启动
55     m_stateMachine->start();
56
57     qDebug() << "[StateMachineService] 切换到状态:" << stateToString(state);
58 }
59
60 void StateMachineService::registerState(GameState state, QState* stateObj)
61 {
62     if (stateObj) {
63         m_stateMap[state] = stateObj;
64         qDebug() << "[StateMachineService] 注册状态:" << stateToString(state);
65     }
66 }
67
68 QState* StateMachineService::getState(GameState state) const
69 {
70     return m_stateMap.value(state, nullptr);
71 }
72
73 void StateMachineService::start()
74 {
75     if (!m_stateMachine->isRunning()) {
76         m_stateMachine->start();
77         qDebug() << "[StateMachineService] 状态机已启动";
78     }
79 }
```

```
80
81 void StateMachineService::stop()
82 {
83     if (m_stateMachine->isRunning()) {
84         m_stateMachine->stop();
85         qDebug() << "[StateMachineService] 状态机已停止";
86     }
87 }
88
89 void StateMachineService::onStateEntered()
90 {
91     // 找到发送信号的状态
92     QState* state = qobject_cast<QState*>(sender());
93     if (!state) return;
94
95     // 查找对应的枚举值
96     for (auto it = m_stateMap.constBegin(); it != m_stateMap.constEnd(); ++it)
97     {
98         if (it.value() == state) {
99             m_currentState = it.key();
100
101             qDebug() << "[StateMachineService] 进入状态:" <<
stateToString(m_currentState);
102
103             // 发出状态改变信号
104             emit stateChanged(m_currentState);
105
106             // 发布事件
107             m_eventBus->publish(EventTypes::System_StateChanged, {
108                 {"state", static_cast<int>(m_currentState)}
109             });
110
111             break;
112         }
113     }
114
115 void StateMachineService::createDefaultStates()
116 {
117     // 创建各个状态
118     QState* mainMenuState = new QState();
119     QState* loadingState = new QState();
120     QState* playingState = new QState();
121     QState* pausedState = new QState();
122     QState* dialogState = new QState();
123     QState* cutsceneState = new QState();
124     QState* gameOverState = new QState();
```

```
125     QState* victoryState = new QState();
126
127     // 设置状态名称（方便调试）
128     mainMenuState->setObjectName("MainMenu");
129     loadingState->setObjectName("Loading");
130     playingState->setObjectName("Playing");
131     pausedState->setObjectName("Paused");
132     dialogState->setObjectName("Dialog");
133     cutsceneState->setObjectName("Cutscene");
134     gameOverState->setObjectName("GameOver");
135     victoryState->setObjectName("Victory");
136
137     // 连接进入信号
138     connect(mainMenuState, &QState::entered, this,
139             &StateMachineService::onStateEntered);
140     connect(loadingState, &QState::entered, this,
141             &StateMachineService::onStateEntered);
142     connect(playingState, &QState::entered, this,
143             &StateMachineService::onStateEntered);
144     connect(pausedState, &QState::entered, this,
145             &StateMachineService::onStateEntered);
146     connect(dialogState, &QState::entered, this,
147             &StateMachineService::onStateEntered);
148     connect(cutsceneState, &QState::entered, this,
149             &StateMachineService::onStateEntered);
150     connect(gameOverState, &QState::entered, this,
151             &StateMachineService::onStateEntered);
152     connect(victoryState, &QState::entered, this,
153             &StateMachineService::onStateEntered);
154
155     // 注册到状态机
156     m_stateMachine->addState(mainMenuState);
157     m_stateMachine->addState(loadingState);
158     m_stateMachine->addState(playingState);
159     m_stateMachine->addState(pausedState);
160     m_stateMachine->addState(dialogState);
161     m_stateMachine->addState(cutsceneState);
162     m_stateMachine->addState(gameOverState);
163     m_stateMachine->addState(victoryState);
164
165     // 保存到映射
166     m_stateMap[GameState::MainMenu] = mainMenuState;
167     m_stateMap[GameState::Loading] = loadingState;
168     m_stateMap[GameState::Playing] = playingState;
169     m_stateMap[GameState::Paused] = pausedState;
170     m_stateMap[GameState::Dialog] = dialogState;
171     m_stateMap[GameState::Cutscene] = cutsceneState;
```

```

164     m_stateMap[GameState::GameOver] = gameOverState;
165     m_stateMap[GameState::Victory] = victoryState;
166
167     // 设置初始状态
168     m_stateMachine->setInitialState(mainMenuState);
169
170     // Playing状态的子状态（探索、战斗等）
171     // 这里先留空，后续玩法系统可以添加子状态
172
173     qDebug() << "[StateMachineService] 默认状态创建完成，共" << m_stateMap.size()
174     << "个状态";
175 }
176
177 void StateMachineService::setupTransitions()
178 {
179     // 注意：为了简化，我们用changeState()方法来切换状态
180     // 而不是通过信号触发转换
181     // 这样更灵活，插件可以随时切换到任意状态
182
183     // 如果需要用信号触发的方式，可以在这里添加转换
184     // 例如：
185     // mainMenuState->addTransition(startGameSignal, loadingState);
186     // loadingState->addTransition(loadCompleteSignal, playingState);
187     // playingState->addTransition(pauseSignal, pausedState);
188     // pausedState->addTransition(resumeSignal, playingState);
189     // pausedState->addTransition(quitToMenuSignal, mainMenuState);
190
191     // 目前先用changeState()的方式，更简单直接
192     // 后续如果需要更复杂的状态转换逻辑，可以改成信号触发的方式
193 }
194
195 QString StateMachineService::stateToString(GameState state) const
196 {
197     switch (state) {
198         case GameState::None: return "None";
199         case GameState::MainMenu: return "MainMenu";
200         case GameState::Loading: return "Loading";
201         case GameState::Playing: return "Playing";
202         case GameState::Paused: return "Paused";
203         case GameState::Dialog: return "Dialog";
204         case GameState::Cutscene: return "Cutscene";
205         case GameState::GameOver: return "GameOver";
206         case GameState::Victory: return "Victory";
207         default: return "Unknown";
208     }
209 }

```

2.9 第九步：基础服务 - 存档服务（完整实现）

2.9.1 存档服务实现

文件：src/services/save/SaveService.h

代码块

```
1  #ifndef SAVESERVICE_H
2  #define SAVESERVICE_H
3
4  #include <QObject>
5  #include <QVariantMap>
6  #include <QList>
7  #include <QDateTime>
8  #include <QDir>
9
10 #include "ISaveService.h"
11
12 /**
13  * @brief 存档服务实现
14  *
15  * 基于QDataStream的二进制存档系统。
16  *
17  * 存档格式：
18  * - 文件头：魔数 + 版本号 + 校验和
19  * - 存档数据：序列化的QVariantMap
20  * - 各系统通过key-value的方式存取数据
21  *
22  * 为什么用二进制而不是JSON？
23  * - 体积小：比JSON小30-50%
24  * - 加载快：直接反序列化，不用解析文本
25  * - 防篡改：玩家不容易修改
26  * - 支持更多类型：QDataStream支持所有Qt基本类型
27  *
28  * 当然，如果希望玩家可以修改存档，可以提供JSON导出功能。
29  */
30 class SaveService : public QObject, public ISaveService
31 {
32     Q_OBJECT
33
34 public:
35     explicit SaveService(QObject *parent = nullptr);
36     ~SaveService() override;
37
38     // ISaveService接口实现
```

```
39     bool saveGame(int slot, const QString& name = QString()) override;
40     bool loadGame(int slot) override;
41
42     SaveInfo getSaveInfo(int slot) const override;
43     QList<SaveInfo> getAllSaveInfos() const override;
44
45     bool deleteSave(int slot) override;
46
47     bool quickSave() override;
48     bool quickLoad() override;
49     bool autoSave() override;
50
51     void setSaveData(const QString& key, const QVariant& value) override;
52     QVariant getSaveData(const QString& key, const QVariant& defaultValue =
53     QVariant()) const override;
54
55     QString getSaveDirectory() const override;
56
57     /**
58      * @brief 刷新存档信息缓存
59      *
60      * 当外部修改了存档文件时调用。
61      */
62     void refreshSaveInfos();
63
64     /**
65      * @brief 获取存档槽位数量
66      */
67     int getSlotCount() const { return m_slotCount; }
68
69     signals:
70     /**
71      * @brief 存档保存成功信号
72      * @param slot 槽位
73      */
74     void saveCompleted(int slot);
75
76     /**
77      * @brief 存档加载成功信号
78      * @param slot 槽位
79      */
80     void loadCompleted(int slot);
81
82     /**
83      * @brief 存档加载失败信号
84      * @param slot 槽位
85      * @param error 错误信息
```

```

85     */
86     void loadFailed(int slot, const QString& error);
87
88 private:
89     // 存档目录
90     QDir m_saveDir;
91
92     // 存档槽位数量
93     int m_slotCount = 10;
94
95     // 快速存档槽位
96     int m_quickSaveSlot = 0;
97
98     // 自动存档槽位
99     int m_autoSaveSlot = 9;
100
101     // 当前游戏数据（内存中的存档数据）
102     QVariantMap m_saveData;
103
104     // 存档信息缓存
105     mutable QList<SaveInfo> m_saveInfoCache;
106     mutable bool m_saveInfoCacheValid = false;
107
108     // 存档文件魔数（用于验证文件格式）
109     static const quint32 SAVE_MAGIC = 0x53415645; // "SAVE"
110
111     // 存档版本号
112     static const quint16 SAVE_VERSION = 1;
113
114     /**
115      * @brief 确保存档目录存在
116      */
117     void ensureSaveDirectory();
118
119     /**
120      * @brief 获取存档文件路径
121      * @param slot 槽位
122      * @return 文件路径
123      */
124     QString getSaveFilePath(int slot) const;
125
126     /**
127      * @brief 计算数据的校验和
128      * @param data 数据
129      * @return 校验和
130      */
131     quint32 calculateChecksum(const QByteArray& data) const;

```

```

132
133     /**
134     * @brief 从存档文件读取基本信息（不加载全部数据）
135     * @param slot 槽位
136     * @return 存档信息
137     *
138     * 用于快速显示存档列表，不用加载整个存档。
139     */
140     SaveInfo readSaveHeader(int slot) const;
141 };
142
143 #endif // SAVESERVICE_H

```

文件：src/services/save/SaveService.cpp

代码块

```

1  #include "SaveService.h"
2
3  #include <QDebug>
4  #include <QFile>
5  #include <QDataStream>
6  #include <QStandardPaths>
7  #include <QCryptographicHash>
8  #include <QPixmap>
9
10 SaveService::SaveService(QObject *parent)
11     : QObject(parent)
12 {
13     // 设置存档目录
14     // Windows: C:/Users/用户名/AppData/Local/StarAbyss/saves
15     // macOS: ~/Library/Application Support/StarAbyss/saves
16     // Linux: ~/.local/share/StarAbyss/saves
17     QString appDataDir =
18     QStandardPaths::writableLocation(QStandardPaths::AppDataLocation);
19     m_saveDir.setPath(appDataDir + "/saves");
20
21     ensureSaveDirectory();
22
23     qDebug() << "[SaveService] 存档服务初始化完成";
24     qDebug() << "[SaveService] 存档目录:" << m_saveDir.absolutePath();
25     qDebug() << "[SaveService] 存档槽位:" << m_slotCount << "个";
26 }
27
28 SaveService::~SaveService()
29 {

```

```
29     qDebug() << "[SaveService] 存档服务已销毁";
30 }
31
32 bool SaveService::saveGame(int slot, const QString& name)
33 {
34     if (slot < 0 || slot >= m_slotCount) {
35         qWarning() << "[SaveService] 无效的存档槽位:" << slot;
36         return false;
37     }
38
39     QString filePath = getSaveFilePath(slot);
40
41     // 添加基本信息
42     m_saveData["_slot"] = slot;
43     m_saveData["_name"] = name.isEmpty() ? QString("存档 %1").arg(slot + 1) :
name;
44     m_saveData["_timestamp"] = QDateTime::currentDateTime();
45     m_saveData["_version"] = SAVE_VERSION;
46
47     // 序列化数据
48     QByteArray data;
49     QDataStream stream(&data, QIODevice::WriteOnly);
50     stream.setVersion(QDataStream::Qt_6_10);
51     stream << m_saveData;
52
53     // 计算校验和
54     quint32 checksum = calculateChecksum(data);
55
56     // 写入文件
57     QFile file(filePath);
58     if (!file.open(QIODevice::WriteOnly)) {
59         qWarning() << "[SaveService] 无法打开存档文件:" << filePath;
60         return false;
61     }
62
63     QDataStream out(&file);
64     out.setVersion(QDataStream::Qt_6_10);
65
66     // 写入文件头
67     out << SAVE_MAGIC;           // 魔数
68     out << SAVE_VERSION;        // 版本号
69     out << checksum;            // 校验和
70     out << data;                // 实际数据
71
72     file.close();
73
74     // 清除缓存
```

```
75     m_saveInfoCacheValid = false;
76
77     qDebug() << "[SaveService] 存档保存成功, 槽位:" << slot
78         << "大小:" << file.size() << "字节";
79
80     emit saveCompleted(slot);
81
82     return true;
83 }
84
85 bool SaveService::loadGame(int slot)
86 {
87     if (slot < 0 || slot >= m_slotCount) {
88         qWarning() << "[SaveService] 无效的存档槽位:" << slot;
89         emit loadFailed(slot, "无效的槽位");
90         return false;
91     }
92
93     QString filePath = getSaveFilePath(slot);
94
95     QFile file(filePath);
96     if (!file.open(QIODevice::ReadOnly)) {
97         qWarning() << "[SaveService] 无法打开存档文件:" << filePath;
98         emit loadFailed(slot, "文件不存在");
99         return false;
100     }
101
102     QDataStream in(&file);
103     in.setVersion(QDataStream::Qt_6_10);
104
105     // 读取文件头
106     quint32 magic;
107     quint16 version;
108     quint32 checksum;
109     QByteArray data;
110
111     in >> magic;
112     in >> version;
113     in >> checksum;
114     in >> data;
115
116     file.close();
117
118     // 验证魔数
119     if (magic != SAVE_MAGIC) {
120         qWarning() << "[SaveService] 存档文件格式错误 (魔数不匹配) ";
121         emit loadFailed(slot, "存档文件格式错误");
```

```
122         return false;
123     }
124
125     // 验证版本（可以做向下兼容）
126     if (version > SAVE_VERSION) {
127         qWarning() << "[SaveService] 存档版本过高:" << version;
128         emit loadFailed(slot, "存档版本过高，请更新游戏");
129         return false;
130     }
131
132     // 验证校验和
133     quint32 actualChecksum = calculateChecksum(data);
134     if (actualChecksum != checksum) {
135         qWarning() << "[SaveService] 存档校验失败，文件可能已损坏";
136         emit loadFailed(slot, "存档文件已损坏");
137         return false;
138     }
139
140     // 反序列化数据
141     QDataStream dataStream(data);
142     dataStream.setVersion(QDataStream::Qt_6_10);
143     dataStream >> m_saveData;
144
145     qDebug() << "[SaveService] 存档加载成功，槽位:" << slot;
146     qDebug() << "[SaveService] 存档名称:" << m_saveData["_name"].toString();
147     qDebug() << "[SaveService] 存档时间:" <<
    m_saveData["_timestamp"].toDateTime().toString();
148
149     emit loadCompleted(slot);
150
151     return true;
152 }
153
154 SaveInfo SaveService::getSaveInfo(int slot) const
155 {
156     if (slot < 0 || slot >= m_slotCount) {
157         return SaveInfo();
158     }
159
160     return readSaveHeader(slot);
161 }
162
163 QList<SaveInfo> SaveService::getAllSaveInfos() const
164 {
165     if (m_saveInfoCacheValid) {
166         return m_saveInfoCache;
167     }
```

```
168
169     QList<SaveInfo> infos;
170     for (int i = 0; i < m_slotCount; ++i) {
171         infos.append(readSaveHeader(i));
172     }
173
174     m_saveInfoCache = infos;
175     m_saveInfoCacheValid = true;
176
177     return infos;
178 }
179
180 bool SaveService::deleteSave(int slot)
181 {
182     if (slot < 0 || slot >= m_slotCount) {
183         return false;
184     }
185
186     QString filePath = getSaveFilePath(slot);
187     if (QFile::exists(filePath)) {
188         bool success = QFile::remove(filePath);
189         if (success) {
190             m_saveInfoCacheValid = false;
191             qDebug() << "[SaveService] 存档已删除, 槽位:" << slot;
192         }
193         return success;
194     }
195
196     return true; // 文件不存在也算删除成功
197 }
198
199 bool SaveService::quickSave()
200 {
201     return saveGame(m_quickSaveSlot, "快速存档");
202 }
203
204 bool SaveService::quickLoad()
205 {
206     return loadGame(m_quickSaveSlot);
207 }
208
209 bool SaveService::autoSave()
210 {
211     return saveGame(m_autoSaveSlot, "自动存档");
212 }
213
214 void SaveService::setSaveData(const QString& key, const QVariant& value)
```

```
215 {
216     m_saveData[key] = value;
217 }
218
219 QVariant SaveService::getSaveData(const QString& key, const QVariant&
    defaultValue) const
220 {
221     return m_saveData.value(key, defaultValue);
222 }
223
224 QString SaveService::getSaveDirectory() const
225 {
226     return m_saveDir.absolutePath();
227 }
228
229 void SaveService::refreshSaveInfos()
230 {
231     m_saveInfoCacheValid = false;
232 }
233
234 void SaveService::ensureSaveDirectory()
235 {
236     if (!m_saveDir.exists()) {
237         if (m_saveDir.mkpath(".")) {
238             qDebug() << "[SaveService] 存档目录已创建:" <<
                m_saveDir.absolutePath();
239         } else {
240             qWarning() << "[SaveService] 无法创建存档目录:" <<
                m_saveDir.absolutePath();
241         }
242     }
243 }
244
245 QString SaveService::getSaveFilePath(int slot) const
246 {
247     return m_saveDir.absoluteFilePath(QString("save%1.sav").arg(slot, 2, 10,
        QChar('0')));
248 }
249
250 quint32 SaveService::calculateChecksum(const QByteArray& data) const
251 {
252     // 用简单的CRC32或者直接取MD5的前4字节
253     // 这里用QCryptographicHash的MD5，取前4个字节作为校验和
254     QByteArray hash = QCryptographicHash::hash(data, QCryptographicHash::Md5);
255
256     quint32 checksum = 0;
257     for (int i = 0; i < 4 && i < hash.size(); ++i) {
```

```
258         checksum = (checksum << 8) | (quint8)hash[i];
259     }
260
261     return checksum;
262 }
263
264 SaveInfo SaveService::readSaveHeader(int slot) const
265 {
266     SaveInfo info;
267     info.slot = slot;
268     info.isValid = false;
269
270     QString filePath = getSaveFilePath(slot);
271     QFile file(filePath);
272
273     if (!file.exists()) {
274         return info;
275     }
276
277     if (!file.open(QIODevice::ReadOnly)) {
278         return info;
279     }
280
281     QDataStream in(&file);
282     in.setVersion(QDataStream::Qt_6_10);
283
284     quint32 magic;
285     quint16 version;
286     quint32 checksum;
287     QByteArray data;
288
289     in >> magic;
290     in >> version;
291     in >> checksum;
292     in >> data;
293
294     file.close();
295
296     // 验证魔数
297     if (magic != SAVE_MAGIC) {
298         return info;
299     }
300
301     // 反序列化（只读头部信息）
302     // 为了性能，我们只读取前几个key
303     // 但QDataStream是顺序的，所以我们还是要全部读出来
304     // 对于小存档，这不是问题
```

```
305
306     QVariantMap saveData;
307     QDataStream dataStream(data);
308     dataStream.setVersion(QDataStream::Qt_6_10);
309     dataStream >> saveData;
310
311     info.name = saveData["_name"].toString();
312     info.timestamp = saveData["_timestamp"].toDateTIme();
313     info.currentLayer = saveData["currentLayer"].toString();
314     info.playerLevel = saveData["playerLevel"].toInt();
315     info.playTime = saveData["playTime"].toString();
316     info.isValid = true;
317
318     // 缩略图 (如果有)
319     if (saveData.contains("thumbnail")) {
320         info.thumbnail = saveData["thumbnail"].value<QPixmap>();
321     }
322
323     return info;
324 }
```

2.10 第十步：主菜单UI实现

现在我们来实现游戏的主菜单。这是第一个UI插件，展示如何用QSS做样式。

2.10.1 主菜单界面

文件：src/plugins/ui/mainmenu/MainMenuPlugin.h

代码块

```
1  #ifndef MAINMENUPLUGIN_H
2  #define MAINMENUPLUGIN_H
3
4  #include <QObject>
5  #include <QWidget>
6  #include <QVBoxLayout>
7  #include <QPushButton>
8  #include <QLabel>
9  #include <QGraphicsOpacityEffect>
10 #include <QPropertyAnimation>
11
12 #include "IPlugin.h"
13 #include "IUIPlugin.h"
14
```

```
15  /**
16   * @brief 主菜单插件
17   *
18   * 实现游戏的主菜单界面。
19   *
20   * 包含：
21   * - 游戏标题
22   * - 开始游戏按钮
23   * - 继续游戏按钮
24   * - 设置按钮
25   * - 关于按钮
26   * - 退出游戏按钮
27   *
28   * UI设计原则：
29   * - 像素风格，和游戏整体风格统一
30   * - 简洁大气，不花哨
31   * - 按钮有悬停、点击动画
32   * - 背景有星空粒子效果
33   */
34  class MainMenuPlugin : public QObject, public IPlugin, public IUIPlugin
35  {
36      Q_OBJECT
37      Q_PLUGIN_METADATA(IID IPlugin_iid)
38      Q_INTERFACES(IPlugin IUIPlugin)
39
40  public:
41      MainMenuPlugin();
42      ~MainMenuPlugin() override;
43
44      // IPlugin接口实现
45      QString pluginId() const override { return "com.starabyss.ui.mainmenu"; }
46      QString pluginName() const override { return "主菜单"; }
47      QString pluginVersion() const override { return "1.0.0"; }
48      QString pluginAuthor() const override { return "StarAbyss Team"; }
49      QString pluginDescription() const override { return "游戏主菜单界面"; }
50
51      QStringList dependencies() const override { return {}; }
52
53      bool initialize(PluginManager* manager, const QVariantMap& config = {})
54      override;
55      void shutdown() override;
56      bool isInitialized() const override { return m_initialized; }
57
58      // IUIPlugin接口实现
59      QWidget* getWidget() override { return m_mainMenuWidget; }
60      void showUI() override;
61      void hideUI() override;
```

```
61
62 private slots:
63     /// 开始游戏按钮点击
64     void onStartGameClicked();
65
66     /// 继续游戏按钮点击
67     void onContinueGameClicked();
68
69     /// 设置按钮点击
70     void onSettingsClicked();
71
72     /// 关于按钮点击
73     void onAboutClicked();
74
75     /// 退出游戏按钮点击
76     void onExitClicked();
77
78 private:
79     // 主菜单窗口
80     QWidget* m_mainMenuWidget = nullptr;
81
82     // 布局
83     QVBoxLayout* m_mainLayout = nullptr;
84
85     // 标题
86     QLabel* m_titleLabel = nullptr;
87     QLabel* m_subtitleLabel = nullptr;
88
89     // 按钮
90     QPushButton* m_startButton = nullptr;
91     QPushButton* m_continueButton = nullptr;
92     QPushButton* m_settingsButton = nullptr;
93     QPushButton* m_aboutButton = nullptr;
94     QPushButton* m_exitButton = nullptr;
95
96     // 版本信息
97     QLabel* m_versionLabel = nullptr;
98
99     // 背景效果
100    QGraphicsOpacityEffect* m_fadeEffect = nullptr;
101    QPropertyAnimation* m_fadeAnimation = nullptr;
102
103    /**
104     * @brief 创建UI
105     */
106    void createUI();
107
```

```

108     /**
109     * @brief 加载QSS样式
110     */
111     void loadStyleSheet();
112
113     /**
114     * @brief 设置按钮动画
115     */
116     void setupButtonAnimations();
117 };
118
119 #endif // MAINMENUPLUGIN_H

```

文件：src/plugins/ui/mainmenu/MainMenuPlugin.cpp

代码块

```

1  #include "MainMenuPlugin.h"
2
3  #include <QDebug>
4  #include <QApplication>
5  #include <QGraphicsDropShadowEffect>
6  #include <QTimer>
7
8  #include "PluginManager.h"
9  #include "EventBus.h"
10 #include "EventTypes.h"
11 #include "ResourceManager.h"
12 #include "IStateMachineService.h"
13
14 MainMenuPlugin::MainMenuPlugin()
15 {
16 }
17
18 MainMenuPlugin::~MainMenuPlugin()
19 {
20     if (m_mainMenuWidget) {
21         m_mainMenuWidget->deleteLater();
22     }
23 }
24
25 bool MainMenuPlugin::initialize(PluginManager* manager, const QVariantMap&
26     config)
27 {
28     Q_UNUSED(config);

```

```
29     m_pluginManager = manager;
30
31     // 创建UI
32     createUI();
33
34     // 加载样式
35     loadStyleSheet();
36
37     m_initialized = true;
38
39     qDebug() << "[MainMenuPlugin] 主菜单插件初始化完成";
40
41     return true;
42 }
43
44 void MainMenuPlugin::shutdown()
45 {
46     m_initialized = false;
47     qDebug() << "[MainMenuPlugin] 主菜单插件已关闭";
48 }
49
50 void MainMenuPlugin::showUI()
51 {
52     if (m_mainMenuWidget) {
53         m_mainMenuWidget->show();
54
55         // 淡入动画
56         if (m_fadeAnimation) {
57             m_fadeAnimation->stop();
58             m_fadeAnimation->setStartValue(0.0);
59             m_fadeAnimation->setEndValue(1.0);
60             m_fadeAnimation->start();
61         }
62
63         EventBus::instance()->publish(EventTypes::UI_MainMenuOpened);
64     }
65 }
66
67 void MainMenuPlugin::hideUI()
68 {
69     if (m_mainMenuWidget) {
70         m_mainMenuWidget->hide();
71         EventBus::instance()->publish(EventTypes::UI_MainMenuClosed);
72     }
73 }
74
75 void MainMenuPlugin::onStartGameClicked()
```

```
76 {
77     qDebug() << "[MainMenuPlugin] 开始游戏";
78
79     // 发布事件
80     EventBus::instance()->publish(EventTypes::Game_NewGame);
81
82     // 切换到加载状态
83     // 这里先简单处理，后续会有加载场景
84     // 实际项目中应该先切到Loading状态，加载完再切到Playing
85
86     // 隐藏主菜单
87     hideUI();
88 }
89
90 void MainMenuPlugin::onContinueGameClicked()
91 {
92     qDebug() << "[MainMenuPlugin] 继续游戏";
93
94     // TODO: 显示存档选择界面
95     // 目前先直接加载快速存档
96
97     EventBus::instance()->publish(EventTypes::Game_LoadSave, {
98         {"slot", 0}
99     });
100
101     hideUI();
102 }
103
104 void MainMenuPlugin::onSettingsClicked()
105 {
106     qDebug() << "[MainMenuPlugin] 打开设置";
107     // TODO: 显示设置界面
108 }
109
110 void MainMenuPlugin::onAboutClicked()
111 {
112     qDebug() << "[MainMenuPlugin] 关于";
113     // TODO: 显示关于对话框
114 }
115
116 void MainMenuPlugin::onExitClicked()
117 {
118     qDebug() << "[MainMenuPlugin] 退出游戏";
119
120     // 发布退出事件
121     EventBus::instance()->publish(EventTypes::System_Shutdown);
122 }
```

```
123     // 退出应用
124     QApplication::quit();
125 }
126
127 void MainMenuPlugin::createUI()
128 {
129     // 创建主菜单窗口
130     m_mainMenuWidget = new QWidget();
131     m_mainMenuWidget->setObjectName("MainMenu");
132     m_mainMenuWidget->setFixedSize(1280, 720); // 暂时固定大小，后续改成自适应
133
134     // 主布局（垂直居中）
135     m_mainLayout = new QVBoxLayout(m_mainMenuWidget);
136     m_mainLayout->setAlignment(Qt::AlignCenter);
137     m_mainLayout->setSpacing(20);
138     m_mainLayout->setContentsMargins(100, 100, 100, 100);
139
140     // 添加弹性空间（把内容推到中间）
141     m_mainLayout->addStretch();
142
143     // 游戏标题
144     m_titleLabel = new QLabel("星渊旅人", m_mainMenuWidget);
145     m_titleLabel->setObjectName("GameTitle");
146     m_titleLabel->setAlignment(Qt::AlignCenter);
147     m_mainLayout->addWidget(m_titleLabel);
148
149     // 副标题
150     m_subtitleLabel = new QLabel("Star Abyss Traveler", m_mainMenuWidget);
151     m_subtitleLabel->setObjectName("GameSubtitle");
152     m_subtitleLabel->setAlignment(Qt::AlignCenter);
153     m_mainLayout->addWidget(m_subtitleLabel);
154
155     // 间距
156     m_mainLayout->addSpacing(60);
157
158     // 开始游戏按钮
159     m_startButton = new QPushButton("开始游戏", m_mainMenuWidget);
160     m_startButton->setObjectName("MenuButton");
161     m_startButton->setCursor(Qt::PointingHandCursor);
162     connect(m_startButton, &QPushButton::clicked, this,
163             &MainMenuPlugin::onStartGameClicked);
164     m_mainLayout->addWidget(m_startButton);
165
166     // 继续游戏按钮
167     m_continueButton = new QPushButton("继续游戏", m_mainMenuWidget);
168     m_continueButton->setObjectName("MenuButton");
169     m_continueButton->setCursor(Qt::PointingHandCursor);
```

```
169     connect(m_continueButton, &QPushButton::clicked, this,
170             &MainMenuPlugin::onContinueGameClicked);
171
172     // 设置按钮
173     m_settingsButton = new QPushButton("游戏设置", m_mainMenuWidget);
174     m_settingsButton->setObjectName("MenuButton");
175     m_settingsButton->setCursor(Qt::PointingHandCursor);
176     connect(m_settingsButton, &QPushButton::clicked, this,
177             &MainMenuPlugin::onSettingsClicked);
178
179     // 关于按钮
180     m_aboutButton = new QPushButton("关于游戏", m_mainMenuWidget);
181     m_aboutButton->setObjectName("MenuButton");
182     m_aboutButton->setCursor(Qt::PointingHandCursor);
183     connect(m_aboutButton, &QPushButton::clicked, this,
184             &MainMenuPlugin::onAboutClicked);
185
186     // 退出按钮
187     m_exitButton = new QPushButton("退出游戏", m_mainMenuWidget);
188     m_exitButton->setObjectName("MenuButton");
189     m_exitButton->setCursor(Qt::PointingHandCursor);
190     connect(m_exitButton, &QPushButton::clicked, this,
191             &MainMenuPlugin::onExitClicked);
```

代码块

```
1  m_mainLayout->addWidget(m_exitButton);
2
3  // 弹性空间
4  m_mainLayout->addStretch();
5
6  // 版本信息
7  m_versionLabel = new QLabel("v0.1.0 Alpha Build", m_mainMenuWidget);
8  m_versionLabel->setObjectName("VersionLabel");
9  m_versionLabel->setAlignment(Qt::AlignRight | Qt::AlignBottom);
10 m_mainLayout->addWidget(m_versionLabel);
11
12 // 设置淡入淡出效果
13 m_fadeEffect = new QGraphicsOpacityEffect(m_mainMenuWidget);
14 m_fadeEffect->setOpacity(0.0);
15 m_mainMenuWidget->setGraphicsEffect(m_fadeEffect);
16
```

```
17 m_fadeAnimation = new QPropertyAnimation(m_fadeEffect, "opacity", this);
18 m_fadeAnimation->setDuration(500);
19 m_fadeAnimation->setEasingCurve(QEasingCurve::InOutQuad);
20
21 // 设置按钮动画
22 setupButtonAnimations();
23
24 qDebug() << "[MainMenuPlugin] UI创建完成";
```

```
}
```

```
void MainMenuPlugin::loadStyleSheet()
```

```
{
```

```
// 主菜单QSS样式
```

```
// 实际项目中应该从文件加载，这里先写死方便测试
```

代码块

```
1  QString styleSheet = R"(
2      #MainMenu {
3          background-color: #0a0a1a;
4          background-image: url(/images/menu_bg.png);
5          background-position: center;
6          background-repeat: no-repeat;
7      }
8
9      #GameTitle {
10         font-size: 72px;
11         font-weight: bold;
12         color: #ffffff;
13         letter-spacing: 8px;
14         text-shadow: 0 0 20px rgba(100, 200, 255, 0.8);
15     }
16
17     #GameSubtitle {
18         font-size: 24px;
19         color: #8899aa;
20         letter-spacing: 4px;
21         font-style: italic;
22     }
23
24     #MenuButton {
25         min-width: 300px;
26         min-height: 50px;
27         font-size: 20px;
28         color: #ccddee;
```

```

29         background-color: rgba(20, 30, 50, 0.8);
30         border: 2px solid #335577;
31         border-radius: 8px;
32         padding: 10px 20px;
33     }
34
35     #MenuButton:hover {
36         color: #ffffff;
37         background-color: rgba(40, 60, 100, 0.9);
38         border-color: #5588bb;
39     }
40
41     #MenuButton:pressed {
42         background-color: rgba(60, 80, 120, 1.0);
43         border-color: #77aadd;
44     }
45
46     #VersionLabel {
47         font-size: 12px;
48         color: #556677;
49     }
50 )";
51
52 m_mainMenuWidget->setStyleSheet(styleSheet);
53
54 qDebug() << "[MainMenuPlugin] 样式表加载完成";

```

```

}

void MainMenuPlugin::setupButtonAnimations()
{

```

// 按钮悬停放大效果

// 用QPropertyAnimation实现

// 这里先简单用QSS的hover状态实现

// 后续可以加更复杂的动画

代码块

```

1 // 给每个按钮添加阴影效果
2 QList<QPushButton*> buttons = {
3     m_startButton, m_continueButton, m_settingsButton,
4     m_aboutButton, m_exitButton
5 };
6
7 for (QPushButton* button : buttons) {
8     QGraphicsDropShadowEffect* shadow = new QGraphicsDropShadowEffect(button);

```

```
9     shadow->setBlurRadius(10);
10    shadow->setColor(QColor(0, 100, 200, 100));
11    shadow->setOffset(0, 0);
12    button->setGraphicsEffect(shadow);
13 }
```

```
}
```

代码块

```
1
2  ##### 2.10.2 主菜单插件的CMake配置
3
4  **文件: src/plugins/ui/mainmenu/CMakeLists.txt**
5  ```cmake
6  # 主菜单插件
7
8  add_library(MainMenuPlugin SHARED
9      MainMenuPlugin.h
10     MainMenuPlugin.cpp
11 )
12
13 target_link_libraries(MainMenuPlugin PRIVATE
14     Core
15     Services
16     Qt6::Core
17     Qt6::Gui
18     Qt6::Widgets
19 )
20
21 set_target_properties(MainMenuPlugin PROPERTIES
22     OUTPUT_NAME "mainmenu"
23     PREFIX "" # 不要lib前缀
24 )
25
26 # 输出到plugins目录
27 set_target_properties(MainMenuPlugin PROPERTIES
28     LIBRARY_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/plugins
29     RUNTIME_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/plugins
30 )
31
32 message(STATUS "MainMenu插件配置完成")
```

2.11 第十一步：游戏主循环框架

现在我们把所有东西串起来，创建游戏的主应用类。

2.11.1 游戏应用类

文件：src/app/GameApplication.h

代码块

```
1  #ifndef GAMEAPPLICATION_H
2  #define GAMEAPPLICATION_H
3
4  #include <QMainWindow>
5  #include <QStackedWidget>
6  #include <QTimer>
7
8  // 前向声明
9  class PluginManager;
10 class EventBus;
11 class ResourceManager;
12 class RenderService;
13 class InputService;
14 class AudioService;
15 class StateMachineService;
16 class SaveService;
17
18 /**
19  * @brief 游戏主应用类
20  *
21  * 这是游戏的核心类，负责：
22  * 1. 初始化所有核心系统
23  * 2. 管理主窗口和视图
24  * 3. 驱动游戏主循环
25  * 4. 处理全局事件
26  *
27  * 游戏启动流程：
28  * 1. 创建GameApplication
29  * 2. 初始化核心服务
30  * 3. 加载插件
31  * 4. 显示主菜单
32  * 5. 进入事件循环
33  */
34 class GameApplication : public QMainWindow
35 {
36     Q_OBJECT
37
38 public:
```

```
39     explicit GameApplication(QWidget *parent = nullptr);
40     ~GameApplication();
41
42     /**
43      * @brief 初始化游戏
44      * @return 是否成功
45      *
46      * 初始化所有系统，加载插件。
47      */
48     bool initialize();
49
50     /**
51      * @brief 启动游戏
52      *
53      * 显示主窗口，开始游戏循环。
54      */
55     void start();
56
57     /**
58      * @brief 关闭游戏
59      *
60      * 保存数据，关闭插件，释放资源。
61      */
62     void shutdown();
63
64     private slots:
65         /// 游戏主循环的一帧
66         void onGameTick(qreal deltaTime);
67
68         /// 状态改变
69         void onStateChanged(GameState newState);
70
71         /// 系统关闭事件
72         void onSystemShutdown();
73
74         /// 新游戏
75         void onNewGame();
76
77         /// 加载存档
78         void onLoadSave(const GameEvent& event);
79
80     private:
81         // 核心管理器
82         PluginManager* m_pluginManager = nullptr;
83         EventBus* m_eventBus = nullptr;
84         ResourceManager* m_resourceManager = nullptr;
85
```

```
86 // 基础服务
87 RenderService* m_renderService = nullptr;
88 InputService* m_inputService = nullptr;
89 AudioService* m_audioService = nullptr;
90 StateMachineService* m_stateMachineService = nullptr;
91 SaveService* m_saveService = nullptr;
92
93 // 主窗口部件
94 QStackedWidget* m_centralWidget = nullptr;
95
96 // 游戏时间
97 qreal m_gameTime = 0.0;
98 int m_frameCount = 0;
99
100 /**
101  * @brief 初始化核心服务
102  */
103 bool initServices();
104
105 /**
106  * @brief 加载插件
107  */
108 bool loadPlugins();
109
110 /**
111  * @brief 设置主窗口
112  */
113 void setupMainWindow();
114
115 /**
116  * @brief 连接事件
117  *
118  * 连接各个系统之间的事件。
119  */
120 void connectEvents();
121
122 /**
123  * @brief 显示主菜单
124  */
125 void showMainMenu();
126
127 /**
128  * @brief 进入游戏
129  */
130 void enterGame();
131
132 /**
```

```
133      * @brief 退出到主菜单
134      */
135      void exitToMainMenu();
136  };
137
138  #endif // GAMEAPPLICATION_H
```

文件：src/app/GameApplication.cpp

代码块

```
1  #include "GameApplication.h"
2
3  #include <QDebug>
4  #include <QApplication>
5  #include <QVBoxLayout>
6  #include <QLabel>
7  #include <QStatusBar>
8
9  // 核心头文件
10 #include "PluginManager.h"
11 #include "EventBus.h"
12 #include "EventTypes.h"
13 #include "ResourceManager.h"
14
15 // 服务头文件
16 #include "RenderService.h"
17 #include "InputService.h"
18 #include "AudioService.h"
19 #include "StateMachineService.h"
20 #include "SaveService.h"
21
22 GameApplication::GameApplication(QWidget *parent)
23     : QMainWindow(parent)
24 {
25     qDebug() << "[GameApplication] 游戏应用创建";
26 }
27
28 GameApplication::~GameApplication()
29 {
30     shutdown();
31     qDebug() << "[GameApplication] 游戏应用已销毁";
32 }
33
34 bool GameApplication::initialize()
35 {
```

```
36     qDebug() << "[GameApplication] 开始初始化...";
37
38     // 步骤1: 初始化核心管理器
39     m_pluginManager = PluginManager::instance();
40     m_eventBus = EventBus::instance();
41     m_resourceManager = ResourceManager::instance();
42
43     // 步骤2: 初始化基础服务
44     if (!initServices()) {
45         qDebug() << "[GameApplication] 服务初始化失败";
46         return false;
47     }
48
49     // 步骤3: 设置主窗口
50     setupMainWindow();
51
52     // 步骤4: 连接事件
53     connectEvents();
54
55     // 步骤5: 加载插件
56     if (!loadPlugins()) {
57         qDebug() << "[GameApplication] 部分插件加载失败，但游戏仍可运行";
58     }
59
60     // 步骤6: 启动状态机
61     m_stateMachineService->start();
62
63     qDebug() << "[GameApplication] 初始化完成";
64
65     return true;
66 }
67
68 void GameApplication::start()
69 {
70     qDebug() << "[GameApplication] 启动游戏...";
71
72     // 显示主窗口
73     show();
74
75     // 显示主菜单
76     showMainMenu();
77
78     // 发布启动完成事件
79     m_eventBus->publish(EventTypes::System_StartupComplete);
80
81     qDebug() << "[GameApplication] 游戏已启动";
82 }
```

```
83
84 void GameApplication::shutdown()
85 {
86     qDebug() << "[GameApplication] 关闭游戏...";
87
88     // 停止游戏循环
89     if (m_renderService) {
90         m_renderService->stopGameLoop();
91     }
92
93     // 关闭状态机
94     if (m_stateMachineService) {
95         m_stateMachineService->stop();
96     }
97
98     // 关闭所有插件
99     if (m_pluginManager) {
100         m_pluginManager->shutdownAll();
101     }
102
103     // 发布关闭事件
104     if (m_eventBus) {
105         m_eventBus->publish(EventTypes::System_Shutdown);
106     }
107
108     qDebug() << "[GameApplication] 游戏已关闭";
109 }
110
111 void GameApplication::onGameTick(qreal deltaTime)
112 {
113     m_gameTime += deltaTime;
114     m_frameCount++;
115
116     // 更新输入服务（清除justPressed/justReleased）
117     if (m_inputService) {
118         m_inputService->update();
119     }
120
121     // 发布tick事件
122     m_eventBus->publish(EventTypes::Game_Tick, {
123         {"deltaTime", deltaTime},
124         {"gameTime", m_gameTime}
125     });
126
127     // 状态栏显示FPS（调试用）
128     if (m_renderService && statusBar()) {
129         statusBar()->showMessage(QString("FPS: %1   gameTime: %2s")
```

```
130         .arg(m_renderService->getCurrentFps())
131         .arg(m_gameTime, 0, 'f', 1));
132     }
133 }
134
135 void GameApplication::onStateChanged(GameState newState)
136 {
137     qDebug() << "[GameApplication] 游戏状态改变:" << static_cast<int>(newState);
138
139     switch (newState) {
140         case GameState::MainMenu:
141             showMainMenu();
142             break;
143
144         case GameState::Loading:
145             // TODO: 显示加载界面
146             break;
147
148         case GameState::Playing:
149             enterGame();
150             break;
151
152         case GameState::Paused:
153             // TODO: 显示暂停菜单
154             break;
155
156         default:
157             break;
158     }
159 }
160
161 void GameApplication::onSystemShutdown()
162 {
163     qDebug() << "[GameApplication] 收到系统关闭事件";
164     close();
165 }
166
167 void GameApplication::onNewGame()
168 {
169     qDebug() << "[GameApplication] 新游戏";
170
171     // 重置游戏数据
172     // TODO: 重置玩家、世界等
173
174     // 切换到游戏状态
175     m_stateMachineService->changeState(GameState::Playing);
176 }
```

```
177
178 void GameApplication::onLoadSave(const GameEvent& event)
179 {
180     int slot = event.data["slot"].toInt();
181     qDebug() << "[GameApplication] 加载存档, 槽位:" << slot;
182
183     if (m_saveService->loadGame(slot)) {
184         // 加载成功, 进入游戏
185         m_stateMachineService->changeState(GameState::Playing);
186     } else {
187         qWarning() << "[GameApplication] 存档加载失败";
188         // TODO: 显示错误提示
189     }
190 }
191
192 bool GameApplication::initServices()
193 {
194     qDebug() << "[GameApplication] 初始化基础服务...";
195
196     // 渲染服务
197     m_renderService = new RenderService(this);
198
199     // 输入服务
200     m_inputService = new InputService(this);
201     m_inputService->setRenderService(m_renderService);
202
203     // 音频服务
204     m_audioService = new AudioService(this);
205
206     // 状态机服务
207     m_stateMachineService = new StateMachineService(this);
208
209     // 存档服务
210     m_saveService = new SaveService(this);
211
212     qDebug() << "[GameApplication] 基础服务初始化完成";
213     return true;
214 }
215
216 bool GameApplication::loadPlugins()
217 {
218     qDebug() << "[GameApplication] 加载插件...";
219
220     // 设置插件目录
221     QString pluginDir = QApplication::applicationDirPath() + "/plugins";
222     m_pluginManager->setPluginDirectory(pluginDir);
223
```

```
224 // 加载所有插件
225 int count = m_pluginManager->loadPlugins();
226
227 qDebug() << "[GameApplication] 成功加载" << count << "个插件";
228
229 // 检查错误
230 QStringList errors = m_pluginManager->getErrors();
231 if (!errors.isEmpty()) {
232     qWarning() << "[GameApplication] 插件加载错误:";
233     for (const QString& error : errors) {
234         qWarning() << "  -" << error;
235     }
236 }
237
238 return count > 0;
239 }
240
241 void GameApplication::setupMainWindow()
242 {
243     // 设置窗口标题
244     setWindowTitle("星渊旅人 - Star Abyss Traveler");
245
246     // 设置窗口大小
247     resize(1280, 720);
248
249     // 创建中心部件（用堆叠窗口管理不同界面）
250     m_centralWidget = new QStackedWidget(this);
251     setCentralWidget(m_centralWidget);
252
253     // 把渲染视图加到堆叠窗口
254     m_centralWidget->addWidget(m_renderService->getView());
255
256     // 状态栏
257     QStatusBar* statusBar = new QStatusBar(this);
258     setStatusBar(statusBar);
259     statusBar->showMessage("就绪");
260
261     qDebug() << "[GameApplication] 主窗口设置完成";
262 }
263
264 void GameApplication::connectEvents()
265 {
266     // 连接游戏循环
267     connect(m_renderService, &RenderService::tick,
268            this, &GameApplication::onGameTick);
269
270     // 连接状态机
```

```
271     connect(m_stateMachineService, &StateMachineService::stateChanged,
272             this, &GameApplication::onStateChanged);
273
274     // 连接事件总线
275     m_eventBus->subscribe(EventTypes::System_Shutdown, this,
276                           &GameApplication::onSystemShutdown);
277
278     m_eventBus->subscribe(EventTypes::Game_NewGame, this,
279                           [this](const GameEvent&) { onNewGame(); });
280
281     m_eventBus->subscribe(EventTypes::Game_LoadSave, this,
282                           &GameApplication::onLoadSave);
283
284     qDebug() << "[GameApplication] 事件连接完成";
285 }
286
287 void GameApplication::showMainMenu()
288 {
289     qDebug() << "[GameApplication] 显示主菜单";
290
291     // 停止游戏循环
292     m_renderService->stopGameLoop();
293
294     // 查找主菜单插件
295     IPlugin* mainMenuPlugin = m_pluginManager-
296 >getPlugin("com.starabyss.ui.mainmenu");
297     if (mainMenuPlugin) {
298         IUIPlugin* uiPlugin = dynamic_cast<IUIPlugin*>(mainMenuPlugin);
299         if (uiPlugin) {
300             // 把主菜单添加到堆叠窗口
301             QWidget* menuWidget = uiPlugin->getWidget();
302             if (m_centralWidget->indexOf(menuWidget) == -1) {
303                 m_centralWidget->addWidget(menuWidget);
304             }
305             m_centralWidget->setCurrentWidget(menuWidget);
306             uiPlugin->showUI();
307         }
308     } else {
309         // 没有主菜单插件，显示一个简单的占位界面
310         QLabel* placeholder = new QLabel("星渊旅人\n\n主菜单插件未加载\n\n按ESC退出", this);
311         placeholder->setAlignment(Qt::AlignCenter);
312         placeholder->setStyleSheet("font-size: 24px; color: white; background-
313 color: #111;");
314         m_centralWidget->addWidget(placeholder);
315         m_centralWidget->setCurrentWidget(placeholder);
316     }
```

```

315 }
316
317 void GameApplication::enterGame()
318 {
319     qDebug() << "[GameApplication] 进入游戏";
320
321     // 切换到游戏视图
322     m_centralWidget->setCurrentWidget(m_renderService->getView());
323
324     // 给视图设置焦点（否则输入不会生效）
325     m_renderService->getView()->setFocus();
326
327     // 启动游戏循环
328     m_renderService->startGameLoop();
329
330     // 播放游戏BGM（如果有）
331     // m_audioService->playBGM("game_bgm.ogg");
332 }
333
334 void GameApplication::exitToMainMenu()
335 {
336     qDebug() << "[GameApplication] 退出到主菜单";
337
338     // 停止游戏循环
339     m_renderService->stopGameLoop();
340
341     // 切换到主菜单状态
342     m_stateMachineService->changeState(GameState::MainMenu);
343 }

```

2.11.2 主程序入口

文件：src/app/main.cpp

代码块

```

1  #include <QApplication>
2  #include <QDebug>
3  #include <QTextCodec>
4
5  #include "GameApplication.h"
6
7  /**
8   * @brief 游戏主入口
9   *
10  * 启动流程：
11  * 1. 创建QApplication

```

```
12  * 2. 创建GameApplication
13  * 3. 初始化
14  * 4. 启动
15  * 5. 进入事件循环
16  */
17 int main(int argc, char *argv[])
18 {
19     // 高DPI支持
20     QApplication::setAttribute(Qt::AA_EnableHighDpiScaling);
21     QApplication::setAttribute(Qt::AA_UseHighDpiPixmaps);
22
23     QApplication app(argc, argv);
24
25     // 设置应用信息
26     QApplication::setApplicationName("StarAbyss");
27     QApplication::setApplicationDisplayName("星渊旅人");
28     QApplication::setApplicationVersion("0.1.0");
29     QApplication::setOrganizationName("StarAbyss Team");
30     QApplication::setOrganizationDomain("starabyss.com");
31
32     // 设置编码 (Qt6默认UTF-8, 这步可以省略, 但写上更保险)
33     QTextCodec::setCodecForLocale(QTextCodec::codecForName("UTF-8"));
34
35     qDebug() << "=====";
36     qDebug() << " 星渊旅人 - Star Abyss Traveler";
37     qDebug() << " 版本: 0.1.0 Alpha";
38     qDebug() << " Qt版本:" << qVersion();
39     qDebug() << "=====";
40
41     // 创建游戏应用
42     GameApplication game;
43
44     // 初始化
45     if (!game.initialize()) {
46         qCritical() << "游戏初始化失败! ";
47         return 1;
48     }
49
50     // 启动
51     game.start();
52
53     // 进入事件循环
54     int result = app.exec();
55
56     qDebug() << "游戏退出, 返回码:" << result;
57
58     return result;
```

2.11.3 app模块的CMake配置

文件：src/app/CMakeLists.txt

代码块

```
1  # 游戏主程序
2
3  add_executable(StarAbyss
4      main.cpp
5      GameApplication.h
6      GameApplication.cpp
7  )
8
9  target_link_libraries(StarAbyss PRIVATE
10     Core
11     Services
12     Qt6::Core
13     Qt6::Gui
14     Qt6::Widgets
15     Qt6::Multimedia
16     Qt6::StateMachine
17 )
18
19 # 输出到bin目录
20 set_target_properties(StarAbyss PROPERTIES
21     RUNTIME_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/bin
22 )
23
24 # Windows下设置控制台（调试用）
25 if(WIN32)
26     # Debug模式显示控制台
27     set_target_properties(StarAbyss PROPERTIES
28         WIN32_EXECUTABLE $<NOT:$<CONFIG:Debug>>
29     )
30 endif()
31
32 message(STATUS "游戏主程序配置完成")
```

2.12 第一阶段总结与验证

2.12.1 第一阶段完成的功能

模块	状态	说明
插件管理器	✅ 完成	支持插件扫描、加载、依赖排序
事件总线	✅ 完成	支持订阅、发布、通配符、优先级
资源管理器	✅ 完成	支持图片、音效、字体、JSON等资源
渲染服务	✅ 完成	Graphics View封装，游戏主循环
输入服务	✅ 完成	键盘鼠标输入，按键映射
音频服务	✅ 完成	BGM和SFX，音量控制，淡入淡出
状态机服务	✅ 完成	游戏状态管理
存档服务	✅ 完成	二进制存档，校验和
主菜单UI	✅ 完成	基础主菜单界面
游戏主循环	✅ 完成	完整的游戏启动流程

2.12.2 编译验证步骤

代码块

```
1  步骤1：在Qt Creator中打开项目
2      文件 → 打开文件或项目 → 选择 CMakeLists.txt
3
4  步骤2：选择构建套件
5      选择 "Desktop Qt 6.10.1 MSVC2022 64bit"
6
7  步骤3：配置项目
8      等待CMake配置完成
9      如果有错误，根据错误信息修复
10
11 步骤4：构建项目
12      点击左下角的锤子图标（构建）
13      或按 Ctrl+B
14
15 步骤5：运行项目
16      点击左下角的绿色三角（运行）
17      或按 Ctrl+R
18
19 步骤6：验证功能
20      - 主菜单是否正常显示
21      - 按钮是否有悬停效果
22      - 点击"开始游戏"是否进入游戏场景
```

- 23 - 状态栏是否显示FPS
- 24 - 点击"退出游戏"是否正常退出

2.12.3 常见问题排查

问题	可能原因	解决方案
插件加载失败	插件没编译到plugins目录	检查CMake输出目录设置
插件加载了但没初始化	依赖关系问题	检查插件的dependencies()
输入没反应	视图没有焦点	调用view->setFocus()
图片不显示	资源路径不对	检查资源目录和搜索路径
音效没声音	音频格式不支持	QSoundEffect只支持wav
中文乱码	编码问题	确保源文件是UTF-8编码

2.12.4 Qt Creator调试技巧

代码块

- 1 调试技巧1：查看qDebug输出
- 2 - 点击底部的"应用程序输出"面板
- 3 - 所有QDebug()的输出都会显示在这里
- 4 - 可以过滤、搜索、复制
- 5
- 6 调试技巧2：断点调试
- 7 - 在代码行号旁边点击，设置断点
- 8 - 按F5开始调试
- 9 - F10单步跳过，F11单步进入，Shift+F11单步跳出
- 10 - 可以查看变量值、调用栈
- 11
- 12 调试技巧3：GammaRay（高级）
- 13 - 分析 → GammaRay → 启动
- 14 - 可以实时查看所有QObject
- 15 - 可以查看状态机状态图
- 16 - 可以实时修改属性调试UI
- 17
- 18 调试技巧4：性能分析
- 19 - 分析 → CPU性能分析器
- 20 - 可以看到每个函数的耗时
- 21 - 找到性能瓶颈

第三部分：第二阶段开发 - 核心玩法系统（第3-8个月）

- 目标：实现三大核心系统（时间褶皱、星核共鸣、生态共生），完成玩家角色、战斗系统、前5层关卡
- 预计耗时：约6个月（单人开发）
- 验收标准：能完整玩通前5层，三大系统都能正常使用

3.1 玩家角色系统

3.1.1 玩家基类设计

文件：src/plugins/gameplay/player/Player.h（占位设计）

代码块

```
1  /**
2   * 玩家角色类（设计占位）
3   *
4   * 功能：
5   * - 移动：左右移动、跳跃、冲刺
6   * - 攻击：普通攻击、技能攻击
7   * - 形态切换：5种星核形态
8   * - 时间能力：时间回溯、时间视野
9   * - 伙伴召唤：召唤共生伙伴
10  *
11  * 属性：
12  * - 生命值、能量值
13  * - 攻击力、防御力
14  * - 移动速度、跳跃高度
15  * - 当前形态、当前伙伴
16  *
17  * 动画状态机：
18  * - Idle（待机）
19  * - Run（跑步）
20  * - Jump（跳跃）
21  * - Fall（下落）
22  * - Attack（攻击）
23  * - Hurt（受伤）
24  * - Dead（死亡）
25  */
26
27 // 玩家类继承自QGraphicsPixmapItem和QObject
28 // 这样既能用Graphics View，又能用信号槽
29 class Player : public QObject, public QGraphicsPixmapItem
```

```
30 {
31     Q_OBJECT
32
33 public:
34     // 移动、跳跃、攻击等方法
35     void moveLeft();
36     void moveRight();
37     void jump();
38     void attack();
39
40     // 受伤、死亡
41     void takeDamage(int amount);
42     void die();
43
44     // 形态切换
45     void switchForm(StarCoreForm form);
46
47     // 时间回溯
48     void startRewind();
49     void stopRewind();
50
51     // ... 更多方法
52
53 private:
54     // 属性
55     int m_health = 100;
56     int m_maxHealth = 100;
57     int m_energy = 100;
58     int m_maxEnergy = 100;
59
60     // 移动
61     qreal m_speed = 300.0; // 像素/秒
62     qreal m_jumpForce = 500.0;
63     QPointF m_velocity;
64
65     // 状态
66     bool m_isGrounded = false;
67     bool m_isAttacking = false;
68     bool m_isDead = false;
69
70     // 形态
71     StarCoreForm m_currentForm = StarCoreForm::Light;
72
73     // 动画
74     int m_currentFrame = 0;
75     qreal m_animationTimer = 0;
76
```

```
77     // 时间回溯相关
78     QList<PlayerState> m_stateHistory;
79     int m_rewindIndex = 0;
80     bool m_isRewinding = false;
81 };
```

3.1.2 玩家动画系统

代码块

```
1  实现要点：
2  1. 精灵图 (Sprite Sheet)：把所有动画帧放在一张图上
3  2. 帧动画：每帧显示精灵图的不同区域
4  3. 状态机：根据玩家状态切换动画
5  4. 过渡：动画之间的平滑过渡
6
7  Qt实现方式：
8  - 用QPixmap存储精灵图
9  - 用QRect记录每帧的位置
10 - 在advance()方法中更新当前帧
11 - 用QStateMachine管理动画状态
```

3.1.3 物理系统

代码块

```
1  简单的2D物理：
2  - 重力：每帧给速度加一个向下的加速度
3  - 碰撞：和地面、平台、墙壁的碰撞检测
4  - 移动：根据速度更新位置
5
6  Graphics View的碰撞检测：
7  - collidesWithItem()：检测和另一个图元的碰撞
8  - shape()：返回精确的碰撞形状（比boundingRect更准）
9  - collisionAtPoint()：检测某点是否碰撞
10
11  优化：
12  - 不用每帧检测所有物体
13  - 用空间分区（网格、四叉树）减少检测次数
14  - 先检测boundingRect，再检测精确形状
```

3.2 星核共鸣系统

3.2.1 五种基础形态

形态	元素	特点	技能示例
光明形态	光	平衡型，攻防兼备	光刃斩、圣光护盾
自然形态	自然	治疗、持续伤害	荆棘缠绕、生命绽放
大地形态	大地	高防御、控制	岩石护甲、地震波
水流形态	水	灵活、减速	水流冲刺、寒冰之息
火焰形态	火	高伤害、爆发	烈焰冲击、爆炎

3.2.2 形态切换机制

代码块

1

设计要点：

2

1. 切换有冷却时间（防止无脑切）

3

2. 切换有短暂无敌帧（操作感好）

4

3. 切换时的视觉特效（元素爆发）

5

4. 切换连招：在特定时机切换可以触发特殊效果

6

7

共鸣组合技：

8

- 光+火 = 神圣烈焰（高伤害+灼烧）

9

- 水+冰 = 绝对零度（冻结）

10

- 大+雷 = 岩石崩裂（范围伤害+眩晕）

11

- 等等...

12

13

实现方式：

14

- 每种形态是一个独立的类，继承自FormBase

15

- FormBase定义了通用接口（技能、被动、属性加成）

16

- 玩家持有当前形态的指针

17

- 切换形态时，旧形态退出，新形态进入

3.2.3 星核碎片收集

代码块

1

星核碎片是游戏中的核心收集品：

2

- 每层都有隐藏的星核碎片

3

- 收集碎片可以解锁新形态、升级技能

4

- 碎片收集进度影响剧情发展

5

6

实现：

- 7 - 碎片作为可拾取的图元
- 8 - 玩家碰到后自动拾取
- 9 - 拾取后播放特效和音效
- 10 - 更新收集进度

3.3 时间褶皱系统

3.3.1 时间回溯

代码块

- 1 核心机制：
 - 2 - 玩家可以回溯最多3秒前的状态
 - 3 - 回溯有能量消耗
 - 4 - 回溯期间玩家无敌
 - 5 - 回溯结束后，玩家回到那个时间点的状态
- 6
- 7 实现思路：
 - 8 1. 每帧记录玩家的状态（位置、速度、血量、朝向等）
 - 9 2. 保存最近3秒的状态到一个环形缓冲区
 - 10 3. 玩家按下回溯键时，倒着播放这些状态
 - 11 4. 回溯期间，世界暂停（或慢放）
- 12
- 13 状态记录：
 - 14 struct PlayerState {
 - 15 QPointF position;
 - 16 QPointF velocity;
 - 17 int health;
 - 18 int energy;
 - 19 StarCoreForm form;
 - 20 bool isGrounded;
 - 21 // ... 其他需要回溯的状态
 - 22 };
- 23
- 24 性能优化：
 - 25 - 不用每帧都记录，比如每0.05秒记录一次（20Hz足够）
 - 26 - 记录的数据结构尽量小
 - 27 - 用固定大小的环形缓冲区，不用动态分配

3.3.2 时间残影

代码块

- 1 时间残影是玩家过去的影子，会重复玩家的动作：

- 2 - 残影会重复玩家几秒前的动作
- 3 - 残影可以攻击敌人
- 4 - 残影有持续时间
- 5 - 可以有多个残影同时存在
- 6
- 7 实现：
- 8 - 用时间回溯的状态记录
- 9 - 创建残影对象，按顺序播放状态
- 10 - 残影有自己的碰撞体，可以造成伤害
- 11 - 残影是半透明的，视觉上区分

3.3.3 时间视野

- 代码块
- 1 时间视野可以看到未来的危险：
 - 2 - 开启后，敌人的攻击会有预判线
 - 3 - 可以看到隐藏的时间裂缝
 - 4 - 消耗能量，持续时间有限
 - 5
 - 6 实现：
 - 7 - 一个全屏的shader效果（或者简单的颜色叠加）
 - 8 - 敌人的攻击预警提前显示
 - 9 - 隐藏的物品/路径在时间视野中显现
 - 10 - 用QGraphicsColorizeEffect或自定义shader

3.4 生态共生系统

3.4.1 五个共生伙伴

伙伴	元素	能力	进化方向
光萤	光	照明、治疗、光弹	治疗量↑ / 攻击↑
岩甲兽	大地	护盾、冲撞、嘲讽	防御↑ / 血量↑
风羽鸟	风	滑翔、加速、风刃	速度↑ / 伤害↑
水游鱼	水	水下呼吸、水流冲击	持续时间↑ / 范围↑
火蜥蜴	火	火焰吐息、燃烧	伤害↑ / 灼烧时间↑

3.4.2 伙伴系统设计

代码块

```
1  伙伴基类:
2  class SymbiosisPartner : public QObject, public QGraphicsPixmapItem
3  {
4      Q_OBJECT
5
6  public:
7      // 伙伴信息
8      virtual QString name() const = 0;
9      virtual PartnerType type() const = 0;
10
11     // 伙伴技能
12     virtual void useSkill() = 0;
13     virtual void passiveEffect() = 0;
14
15     // 进化
16     virtual void evolve() = 0;
17     virtual int evolutionLevel() const = 0;
18
19     // 跟随玩家
20     void followPlayer(const QPointF& playerPos);
21
22 protected:
23     // 伙伴属性
24     int m_level = 1;
25     int m_evolutionLevel = 1;
26     qreal m_skillCooldown = 0;
27     qreal m_skillMaxCooldown = 5.0;
28 };
29
30 伙伴AI:
31 - 平时跟随玩家，保持一定距离
32 - 战斗时自动攻击附近的敌人
33 - 玩家使用技能时，伙伴配合释放技能
34 - 用简单的状态机：跟随 / 战斗 / 技能
35
36 基因融合:
37 - 收集不同的基因片段
38 - 可以融合到伙伴身上，增加属性
39 - 每个伙伴有多个基因槽位
40 - 基因有稀有度和套装效果
```

3.5 战斗系统

3.5.1 战斗核心机制

代码块

```
1  伤害计算：
2  最终伤害 = (攻击力 × 技能倍率 - 防御力) × 元素反应倍率 × 暴击倍率
3
4  元素反应系统：
5  - 火+水 = 蒸发 (伤害×2.0)
6  - 火+冰 = 融化 (伤害×1.5)
7  - 雷+水 = 感电 (持续伤害)
8  - 冰+风 = 扩散 (范围扩大)
9  - 等等...
10
11 硬直系统：
12 - 攻击命中敌人会造成硬直
13 - 硬直期间敌人无法行动
14 - 不同攻击硬直时间不同
15 - 可以通过连招延长硬直
```

3.5.2 敌人AI

代码块

```
1  敌人基类：
2  class Enemy : public QObject, public QGraphicsPixmapItem
3  {
4      Q_OBJECT
5
6  public:
7      // AI状态
8      enum class AIState {
9          Idle,          // 待机
10         Patrol,        // 巡逻
11         Chase,          // 追击
12         Attack,         // 攻击
13         Hurt,           // 受伤
14         Dead            // 死亡
15     };
16
17     // 行为
18     virtual void updateAI(qreal deltaTime) = 0;
19     virtual void attack() = 0;
20     virtual void takeDamage(int damage);
21     virtual void die();
22
23 protected:
```

```

24     AIState m_aiState = AIState::Idle;
25
26     // 属性
27     int m_health = 50;
28     int m_maxHealth = 50;
29     int m_damage = 10;
30     qreal m_speed = 100.0;
31
32     // 检测范围
33     qreal m_detectRange = 300.0;
34     qreal m_attackRange = 50.0;
35
36     // 目标
37     Player* m_target = nullptr;
38 };
39
40 简单的敌人类型：
41 1. 近战型：冲过来打玩家
42 2. 远程型：保持距离，发射弹幕
43 3. 坦克型：血厚，伤害低，有护盾
44 4. 精英怪：技能多，伤害高，有特殊机制
45 5. BOSS：多阶段，复杂机制

```

3.5.3 碰撞检测优化

代码块

```

1  碰撞检测是性能瓶颈，需要优化：
2
3  1. 空间分区（网格法）
4      - 把场景分成一个个格子
5      - 每个物体只和同格子及相邻格子的物体检测
6      - 大幅减少检测次数
7
8  2. 分层检测
9      - 第一层：boundingRect检测（快，但是不准）
10     - 第二层：shape()精确检测（慢，但是准）
11     - 只有第一层通过了才做第二层
12
13 3. 对象池
14     - 敌人、子弹、特效等频繁创建销毁的对象
15     - 用对象池复用，避免内存分配
16
17 4. 休眠机制
18     - 离玩家远的敌人进入休眠
19     - 休眠的敌人不更新AI

```

3.6 地图与关卡系统

3.6.1 瓦片地图

代码块

```

1  瓦片地图 (Tile Map) :
2  - 地图由一个个瓦片 (Tile) 组成
3  - 每个瓦片是一张小图片
4  - 用二维数组存储瓦片ID
5  - 渲染时根据ID绘制对应的图片
6
7  优点:
8  - 节省内存 (重复的瓦片复用同一张图)
9  - 编辑方便 (有专门的地图编辑器, 比如Tiled)
10 - 碰撞简单 (瓦片有碰撞属性)
11
12 Qt实现:
13 - 可以自己写瓦片地图渲染器
14 - 也可以用第三方库 (比如QTileEngine)
15 - 推荐: 自己实现, 更灵活, 更可控
16
17 瓦片类型:
18 - 地面: 可以站
19 - 墙壁: 不能通过
20 - 平台: 可以从下面跳上去
21 - 陷阱: 造成伤害
22 - 传送门: 传送到其他区域
23 - 等等...
```

3.6.2 层级管理

代码块

```

1  18层大陆, 每层都是独立的关卡:
2
3  层数据结构:
4  struct LayerData {
5      int layerId;           // 层ID
6      QString name;         // 层名称
7      QString theme;        // 主题 (光明、自然等)
8      QSize mapSize;        // 地图大小
```

```

9      QVector<QVector<int>> tiles; // 瓦片数据
10     QList<EnemySpawn> enemies; // 敌人出生点
11     QList<ItemSpawn> items;    // 道具位置
12     QString bgmName;           // 背景音乐
13     QColor backgroundColor;    // 背景色
14     // ... 更多
15 };
16
17 层级切换：
18 - 走到层的边缘，触发传送
19 - 显示加载界面
20 - 卸载当前层资源
21 - 加载下一层资源
22 - 显示过渡动画
23 - 玩家出现在新层的入口

```

3.6.3 相机跟随

代码块

```

1  相机跟随玩家移动：
2
3  简单跟随：
4  - 相机中心 = 玩家位置
5  - 太生硬，体验不好
6
7  平滑跟随：
8  - 相机位置 = 相机位置 + (目标位置 - 相机位置) * 系数
9  - 系数越小，跟随越平滑
10 - 玩家移动时，相机慢慢跟上
11 - 体验好很多
12
13 边界限制：
14 - 相机不能移出地图边界
15 - 当玩家靠近边界时，相机停在边界
16 - 这样不会看到地图外面
17
18 其他效果：
19 - 屏幕震动：受击、爆炸时
20 - 镜头缩放：BOSS战、特殊场景
21 - 慢动作：特殊技能、击杀

```

3.7 UI系统

3.7.1 HUD（游戏内界面）

代码块

```
1  HUD元素：
2  1. 生命值条
3      - 显示当前/最大生命值
4      - 受伤时红色闪烁
5      - 回血时有动画
6
7  2. 能量值条
8      - 显示当前/最大能量
9      - 使用技能时消耗
10     - 自动恢复
11
12 3. 形态指示器
13     - 显示当前星核形态
14     - 显示切换冷却
15     - 点击可以切换（或者按键）
16
17 4. 伙伴头像
18     - 显示当前伙伴
19     - 显示技能冷却
20     - 显示伙伴血量
21
22 5. 小地图
23     - 显示玩家位置
24     - 显示已探索区域
25     - 显示敌人、道具位置
26
27 6. 任务提示
28     - 当前任务目标
29     - 完成进度
30
31 实现方式：
32 - 用QWidget做HUD，叠加在游戏视图上
33 - 或者用QGraphicsItem做HUD，在场景里渲染
34 - 推荐用QWidget，布局方便，样式好做
```

3.7.2 背包系统

代码块

```
1  背包界面：
2  - 物品格子（比如4×6=24格）
3  - 物品分类：消耗品、装备、材料、重要物品
4  - 物品详情：点击物品显示描述和属性
```

- 5 - 操作：使用、装备、丢弃、整理
- 6
- 7 物品系统：
- 8 - 物品基类：Item
- 9 - 子类：Consumable（消耗品）、Equipment（装备）、Material（材料）
- 10 - 物品有ID、名称、描述、图标、堆叠数量
- 11
- 12 装备系统：
- 13 - 装备槽位：武器、头盔、胸甲、护腿、鞋子、饰品
- 14 - 装备有属性加成
- 15 - 装备有稀有度（普通、稀有、史诗、传说）
- 16 - 装备可以强化、附魔

3.7.3 QSS主题系统

代码块

- 1 每个层级有不同的UI主题：
- 2
- 3 主题文件结构：
- 4 styles/
 - 5 |— base.qss # 基础样式
 - 6 |— themes/
 - 7 | |— light.qss # 光明层主题（金色、白色）
 - 8 | |— nature.qss # 自然层主题（绿色、棕色）
 - 9 | |— earth.qss # 大地层主题（土黄、灰色）
 - 10 | |— water.qss # 水流层主题（蓝色、青色）
 - 11 | |— fire.qss # 火焰层主题（红色、橙色）
 - 12 |— components/
 - 13 |— buttons.qss
 - 14 |— progressbars.qss
 - 15 |— panels.qss
- 16
- 17 主题切换：
- 18 - 进入新层级时，加载对应的主题QSS
- 19 - 用qApp->setStyleSheet()应用
- 20 - 可以加过渡动画（颜色渐变）
- 21 - 玩家也可以手动选择主题

第四部分：第三阶段开发 - 内容填充与优化（第9-18个月）

目标：完成所有18层内容，优化性能，修复Bug

预计耗时：约10个月

4.1 层级内容开发流程

4.1.1 层级开发流水线

代码块

1 每个层级的开发流程：

2

3 1. 设计阶段（1-2周）

4 - 关卡设计：地图布局、敌人配置、谜题设计

5 - 美术设计：瓦片集、背景、敌人和道具美术

6 - 剧情设计：该层的剧情、NPC对话

7

8 2. 制作阶段（2-3周）

9 - 制作瓦片集和精灵图

10 - 用地图编辑器编辑关卡

11 - 配置敌人和道具

12 - 编写剧情对话

13

14 3. 实现阶段（1-2周）

15 - 实现该层的特殊机制

16 - 实现该层的敌人AI

17 - 实现该层的谜题

18 - 实现BOSS战

19

20 4. 测试阶段（1周）

21 - 关卡可玩性测试

22 - 难度调整

23 - Bug修复

24 - 性能优化

25

26 总计：每个层级约5-8周

27 18层就是90-144周？不对，这样太慢了

28

29 优化：

30 - 前5层详细做（每层5-8周）

31 - 中间8层中等做（每层3-4周）

32 - 最后5层简化做（每层2周）

33 - 总时间： $5 \times 6 + 8 \times 3.5 + 5 \times 2 = 30 + 28 + 10 = 68 \text{周} \approx 17 \text{个月}$

34 - 加上其他系统，总时间差不多24个月

4.1.2 数据驱动的关卡设计

代码块JSON配置关卡，不用改代码：

```
2
3  {
4    "layerId": 1,
5    "name": "光明层",
6    "theme": "light",
7    "mapSize": [100, 30],
8    "tileSize": 32,
9    "tileset": "tiles_light.png",
10   "tileData": [
11     [0, 0, 0, 1, 1, 1, ...],
12     ...
13   ],
14   "enemies": [
15     {"type": "light_slime", "x": 500, "y": 800},
16     {"type": "light_knight", "x": 1200, "y": 800}
17   ],
18   "items": [
19     {"type": "health_potion", "x": 300, "y": 700},
20     {"type": "star_fragment", "x": 1500, "y": 500}
21   ],
22   "npcs": [
23     {"id": "elder", "x": 200, "y": 800, "dialog": "dialog_elder.json"}
24   ],
25   "boss": {
26     "type": "light_guardian",
27     "x": 3000,
28     "y": 800
29   },
30   "bgm": "light_bgm.ogg",
31   "backgroundColor": "#fff8e7"
32 }
```

好处：

- 35 - 策划可以自己改配置，不用程序
- 36 - 可以做关卡编辑器
- 37 - MOD友好
- 38 - 热更新方便

4.2 剧情与任务系统

4.2.1 对话系统

代码块

```
1  对话系统设计：
2
3  对话数据（JSON）：
4  {
5      "id": "elder_greeting",
6      "speaker": "长老",
7      "portrait": "elder_portrait.png",
8      "lines": [
9          {
10             "text": "欢迎来到光明层，年轻的旅人。",
11             "next": "elder_explain"
12         },
13         {
14             "id": "elder_explain",
15             "text": "你需要收集星核碎片，才能继续前进。",
16             "choices": [
17                 {"text": "我明白了", "next": "elder_end"},
18                 {"text": "星核碎片是什么？", "next": "elder_explain_more"}
19             ]
20         }
21     ]
22 }
23
24 UI实现：
25 - 底部对话框，显示说话人名字和文字
26 - 头像在左边
27 - 文字逐字显示（打字机效果）
28 - 选项在下方，点击选择
29 - 按空格或回车继续
30
31 打字机效果：
32 - 用QTimer逐字添加文字
33 - 可以设置速度
34 - 可以按空格直接显示全部
```

4.2.2 任务系统

代码块

```
1  任务类型：
2  1. 主线任务：推动剧情发展
3  2. 支线任务：可选，奖励丰富
4  3. 日常任务：每天刷新，重复做
5  4. 成就：收集、挑战类
6
7  任务数据结构：
```

```
8 struct Quest {
9     QString id;
10    QString name;
11    QString description;
12    QuestType type;
13
14    // 目标
15    QList<QuestObjective> objectives;
16
17    // 奖励
18    int expReward;
19    int goldReward;
20    QList<ItemReward> itemRewards;
21
22    // 状态
23    QuestStatus status;
24 };
25
26 任务追踪:
27 - 任务日志界面: 查看所有任务
28 - 当前任务追踪: 屏幕右侧显示当前任务进度
29 - 任务完成提示: 弹窗通知
```

4.3 性能优化

4.3.1 渲染优化

代码块

```
1 Graphics View性能优化:
2
3 1. 图元缓存
4 - 静态图元设置CacheMode
5 - item->setCacheMode(QGraphicsItem::DeviceCoordinateCache);
6 - 缓存渲染结果, 不用每帧重绘
7
8 2. 视口裁剪
9 - Graphics View自动做了
10 - 但自己的逻辑也要注意:
11   不在视口内的物体不更新AI
12
13 3. 减少图元数量
14 - 合并小图元
15 - 背景用一张大图, 不用很多小瓦片
```

```
16     - 远处的物体简化
17
18 4. 分层渲染
19     - 背景层：静态，不常更新
20     - 游戏层：动态，每帧更新
21     - 前景层：半透明效果
22     - UI层：最上层，独立
23
24 5. OpenGL加速
25     - 复杂场景可以启用
26     - view->setViewport(new QOpenGLWidget());
27     - 注意兼容性问题
```

4.3.2 内存优化

代码块

```
1  内存优化策略：
2
3  1. 对象池
4     - 敌人、子弹、特效等
5     - 避免频繁new/delete
6     - 减少内存碎片
7
8  2. 资源管理
9     - 不用的资源及时释放
10    - 场景切换时清理缓存
11    - 大资源（音乐、大图）按需加载
12
13  3. 图片优化
14     - 用合适的格式（PNG压缩、WebP更小）
15     - 用合适的尺寸（不要比需要的大）
16     - 图集（Sprite Sheet）减少绘制调用
17
18  4. 内存泄漏检测
19     - 定期用Memcheck检测
20     - 确保没有泄漏
21     - QObject父子关系帮助管理
```

4.3.3 加载优化

代码块

```
1  加载优化：
2
3  1. 异步加载
```

- 4 - 资源加载在后台线程
- 5 - 不卡主线程
- 6 - 显示加载进度条
- 7
- 8 2. 分块加载
- 9 - 大地图分成几块
- 10 - 只加载玩家附近的块
- 11 - 玩家移动时动态加载/卸载
- 12
- 13 3. 预加载
- 14 - 加载界面时预加载接下来需要的资源
- 15 - 过场动画时加载下一场景
- 16
- 17 4. 懒加载
- 18 - 不常用的资源等需要时再加载
- 19 - 比如某些稀有道具的图标

4.4 Bug修复与测试

4.4.1 测试策略

代码块

- 1 测试类型：
- 2
- 3 1. 单元测试
- 4 - 核心系统的单元测试
- 5 - 用Qt Test框架
- 6 - 每次提交自动运行
- 7
- 8 2. 集成测试
- 9 - 系统之间的交互
- 10 - 手动测试为主
- 11
- 12 3. 玩法测试
- 13 - 关卡可玩性
- 14 - 难度平衡
- 15 - BOSS战设计
- 16
- 17 4. 性能测试
- 18 - 帧率测试
- 19 - 内存使用
- 20 - 加载时间
- 21

- 22 5. 兼容性测试
- 23 - 不同操作系统
- 24 - 不同硬件配置
- 25 - 不同分辨率
- 26
- 27 测试流程：
- 28 - 每个功能完成后自测
- 29 - 每周做一次完整测试
- 30 - 每个大版本做全面测试
- 31 - 发布前做回归测试

4.4.2 常见Bug类型

代码块

- 1 游戏开发常见Bug：
- 2
- 3 1. 碰撞相关
- 4 - 穿墙：碰撞检测没做好
- 5 - 卡墙：卡在墙里出不来
- 6 - 落地抖动：地面检测有问题
- 7
- 8 2. 动画相关
- 9 - 动画不切换：状态机逻辑错了
- 10 - 动画闪烁：帧切换太快或太慢
- 11 - 动画错位：锚点没对准
- 12
- 13 3. 状态相关
- 14 - 死亡后还能移动：状态没切换
- 15 - 暂停了还在动：没处理暂停状态
- 16 - 状态切换死循环：状态转换条件错了
- 17
- 18 4. 内存相关
- 19 - 内存泄漏：对象没释放
- 20 - 野指针：访问已删除的对象
- 21 - 栈溢出：递归太深
- 22
- 23 5. 性能相关
- 24 - 帧率低：渲染或逻辑太耗时
- 25 - 卡顿：GC或加载导致
- 26 - 内存暴涨：泄漏或加载太多

第五部分：第四阶段开发 - 发布与后续（第19-24个月）

目标：完成测试、打包发布、MOD支持、DLC规划

预计耗时：约6个月

5.1 打包与部署

5.1.1 Windows打包

代码块

```
1  使用windeployqt工具：
2
3  步骤1：Release模式编译
4  步骤2：打开Qt命令行（Qt 6.10.1 MSVC 2022 64-bit）
5  步骤3：cd到编译输出目录
6  步骤4：执行：
7      windeployqt StarAbyss.exe --release --no-translations
8
9  步骤5：复制游戏资源文件
10     - resources/ 目录
11     - plugins/ 目录
12     - saves/ 目录（可选，首次运行会自动创建）
13
14  步骤6：测试
15     - 把整个目录复制到另一台没装Qt的电脑
16     - 运行游戏，确保正常
17
18  步骤7：打包
19     - 可以用Inno Setup做安装包
20     - 或者直接打包成zip
21     - Steam发布用SteamPipe
22
23  可选优化：
24     - UPX压缩dll，减少体积
25     - 去掉不需要的Qt模块
26     - 签名（防止SmartScreen警告）
```

5.1.2 macOS打包

代码块

```
1  使用macdeployqt工具：
2
3  步骤1：Release模式编译，生成StarAbyss.app
4  步骤2：执行：
5      macdeployqt StarAbyss.app -dmg -no-strip
```

```
6
7  步骤3: 复制资源到app包内
8    - StarAbyss.app/Contents/Resources/
9
10 步骤4: 测试
11    - 在另一台Mac上测试
12
13 步骤5: 公证 (可选但推荐)
14    - 用开发者账号签名
15    - 上传苹果公证
16    - 用户打开就不会有警告
```

5.1.3 Linux打包

代码块

```
1  使用AppImage格式:
2
3  步骤1: Release模式编译
4  步骤2: 下载linuxdeployqt
5  步骤3: 执行:
6    linuxdeployqt StarAbyss -appimage -no-translations
7
8  步骤4: 测试
9    - chmod +x StarAbyss-x86_64.AppImage
10   - ./StarAbyss-x86_64.AppImage
11
12 优点:
13    - 单文件, 用户直接运行
14    - 几乎支持所有发行版
15    - 自带所有依赖
```

5.2 MOD支持

5.2.1 MOD开发指南

代码块

```
1  基于Qt Plugin的MOD系统:
2
3  MOD能做什么:
4    1. 添加新层级
5    2. 添加新敌人
6    3. 添加新道具
```

```

7   4. 添加新技能
8   5. 修改游戏平衡
9   6. 自定义UI
10  7. 等等...
11
12  MOD开发步骤：
13  1. 安装游戏和MOD开发工具
14  2. 创建插件项目
15  3. 实现插件接口
16  4. 编译成dll/so
17  5. 放到游戏plugins目录
18  6. 测试
19
20  MOD示例（添加一个新敌人）：
21  class MyEnemyPlugin : public QObject, public IPlugin, public IContentPlugin
22  {
23      Q_OBJECT
24      Q_PLUGIN_METADATA(IID IPlugin_iid)
25      Q_INTERFACES(IPlugin IContentPlugin)
26
27  public:
28      QString pluginId() const override { return "com.mymod.coolenemy"; }
29      QString pluginName() const override { return "酷炫敌人MOD"; }
30
31      bool initialize(PluginManager* manager, const QVariantMap& config)
32      override {
33          // 注册新敌人类型
34          EnemyManager::instance()->registerEnemyType("cool_enemy", []() {
35              return new CoolEnemy();
36          });
37          return true;
38
39      void shutdown() override {
40          // 清理
41      }
42  };

```

5.2.2 MOD管理器

代码块

- 1 游戏内MOD管理器功能：
- 2 - 列出所有已安装的MOD
- 3 - 显示MOD信息（名称、版本、作者、描述）
- 4 - 启用/禁用单个MOD

- 5 - 调整MOD加载顺序
- 6 - 检查MOD冲突
- 7 - 打开MOD目录
- 8
- 9 Steam创意工坊集成：
- 10 - 浏览创意工坊
- 11 - 订阅/取消订阅MOD
- 12 - 自动下载和更新MOD
- 13 - 分享自己的MOD

5.3 DLC规划

5.3.1 后续DLC计划

代码块

- 1 DLC1：深渊之心（发布后6个月）
- 2 - 新增3个隐藏层级
- 3 - 新增结局
- 4 - 新增共生伙伴
- 5 - 新的BOSS和敌人
- 6 - 新的装备和道具
- 7
- 8 DLC2：时间之外（发布后12个月）
- 9 - 时间系统扩展
- 10 - 新的时间能力
- 11 - 时间挑战模式
- 12 - 新的剧情线
- 13
- 14 DLC3：群星之境（发布后18个月）
- 15 - 全新的星界区域
- 16 - 多人合作模式（可选）
- 17 - 更多的MOD支持
- 18 - 终局内容
- 19
- 20 免费更新：
- 21 - 定期平衡调整
- 22 - 新的挑战模式
- 23 - 质量改进
- 24 - Bug修复

第六部分：Qt Creator工具链深度使用指南

目标：充分利用Qt Creator的各种工具，提升开发效率

6.1 开发调试工具

6.1.1 CPU性能分析器（CPU Profiler）

代码块

- 1 使用场景：
- 2 - 每个大功能完成后做性能检测
- 3 - 出现掉帧、卡顿问题时定位瓶颈
- 4 - 版本发布前做全流程性能体检
- 5
- 6 操作步骤：
- 7 1. 切换到"分析"模式（左侧边栏第5个图标）
- 8 2. 选择"CPU性能分析器"
- 9 3. 点击"记录"，正常操作游戏
- 10 4. 点击"停止"，等待分析结果
- 11 5. 查看火焰图，找到最宽的横条（最耗时的函数）
- 12 6. 双击跳转到代码，优化后重新测试
- 13
- 14 游戏开发重点关注：
- 15 - 渲染线程每帧耗时（目标 < 12ms）
- 16 - 碰撞检测函数耗时
- 17 - 游戏主循环tick函数耗时
- 18 - 内存分配频率

6.1.2 内存检测器（Memcheck）

代码块

- 1 使用场景：
- 2 - 每次大功能合并后必跑
- 3 - 出现崩溃、奇怪bug时排查
- 4 - 版本发布前全面体检
- 5
- 6 操作步骤：
- 7 1. 切换到"分析"模式
- 8 2. 选择"Valgrind内存分析"（Linux/macOS）
- 9 Windows下可以用MSVC的内存诊断
- 10 3. 点击启动，完整运行游戏核心流程
- 11 4. 正常退出，查看分析结果
- 12

13 重点关注：

- 14 - 确定泄漏 (Definitely Lost)：必须修复
- 15 - 间接泄漏 (Indirectly Lost)：通常伴随直接泄漏
- 16 - 可能泄漏 (Possibly Lost)：需要检查
- 17 - 仍可达 (Still Reachable)：全局单例等，正常

18

19 游戏常见泄漏点：

- 20 - QGraphicsItem没加到scene也没delete
- 21 - 动态创建的QWidget没设置parent
- 22 - 定时器没停止、信号槽没断开
- 23 - 线程里创建的对象没正确销毁

6.1.3 GammaRay (Qt专用调试工具)

代码块

1 GammaRay是Qt开发的神器，强烈推荐安装！

2

3 核心功能：

- 4 1. 对象浏览器
 - 5 - 查看所有QObject的树形结构
 - 6 - 实时修改属性值 (调试UI神器)
 - 7 - 查看对象的信号槽连接
- 8
- 9 2. 部件浏览器 (Widget Inspector)
 - 10 - 可视化查看所有QWidget
 - 11 - 点击选择控件，查看属性和样式
 - 12 - 实时修改QSS，立即看效果
 - 13 - 不用改代码 → 编译 → 运行的循环
- 14
- 15 3. 状态机查看器
 - 16 - 可视化显示QStateMachine的状态图
 - 17 - 实时高亮当前状态
 - 18 - 单步执行状态转换
 - 19 - 调试游戏状态机超级方便
- 20
- 21 4. 信号槽查看器
 - 22 - 查看所有信号槽连接
 - 23 - 统计每个信号触发次数
 - 24 - 调试信号槽问题
- 25
- 26 5. 属性动画查看器
 - 27 - 查看所有正在运行的动画
 - 28 - 实时显示进度和当前值
 - 29 - 可以暂停、单步
- 30

31 安装方法：
32 - Qt Creator → 工具 → 选项 → GammaRay
33 - 点击安装，选择对应Qt版本
34 - 或者自己编译安装
35
36 使用方法：
37 - 分析 → GammaRay → 启动
38 - 或者附加到运行中的进程

6.1.4 静态代码分析（Clang-Tidy）

代码块

1 Clang-Tidy是静态代码分析工具，写代码时就能发现问题。
2
3 开启方法：
4 1. 工具 → 选项 → C++ → 代码模型
5 2. 勾选"启用Clang-Tidy"
6 3. 配置检查项
7
8 推荐开启的检查：
9 - performance-*：性能类
10 - readability-*：可读性类
11 - modernize-*：现代C++类
12 - bugprone-*：Bug检测类
13 - cppcoreguidelines-*：核心指南类
14
15 游戏开发重点关注：
16 1. performance-unnecessary-value-param
17 - 不必要的值传递，建议改const引用
18 - 函数调用频繁的地方很重要
19
20 2. performance-for-range-copy
21 - 范围for循环里的不必要拷贝
22 - 用const auto&代替auto
23
24 3. bugprone-use-after-move
25 - 移动后使用的问题
26 - 移动语义用不好容易出bug
27
28 4. readability-identifier-naming
29 - 统一命名规范
30 - 类名大驼峰、变量小驼峰
31
32 使用方式：
33 - 实时提示：写代码时黄色波浪线

- 34 - 全项目扫描：分析 → 运行Clang-Tidy
- 35 - 自动修复：Alt+Enter应用修复建议

6.2 项目管理工具

6.2.1 Git可视化集成

代码块

- 1 Qt Creator内置了完整的Git支持，不用记命令行。
- 2
- 3 日常提交：
 - 4 1. 点击左侧"版本控制"图标
 - 5 2. 查看"已修改"文件列表，每个文件可以看diff
 - 6 3. 勾选要提交的文件
 - 7 4. 写提交信息（建议格式：feat：添加xxx功能）
 - 8 5. 点击"提交并推送"
- 9
- 10 分支管理：
 - 11 1. 左下角状态栏显示当前分支，点击
 - 12 2. 可以看到所有本地和远程分支
 - 13 3. 点击分支可以切换、重命名、删除
 - 14 4. 创建新分支
- 15
- 16 查看历史：
 - 17 1. 工具 → Git → 显示日志
 - 18 2. 图形化的提交历史
 - 19 3. 点击每个提交看修改了哪些文件
 - 20 4. 右键可以对比、拣选、重置
- 21
- 22 解决冲突：
 - 23 - 合并出现冲突时，自动弹出冲突解决界面
 - 24 - 左侧本地版本，右侧远程版本，中间合并结果
 - 25 - 点击"采用左侧"或"采用右侧"，或手动编辑
 - 26 - 解决完保存，标记为已解决
- 27
- 28 推荐Git工作流：
 - 29 - main分支：稳定可发布版本
 - 30 - develop分支：开发主分支
 - 31 - feature/xxx：每个功能一个分支
 - 32 - 每天下班前提交push
 - 33 - 每个大功能完成后合并回develop

6.2.2 任务面板 (TODO/FIXME)

代码块

```
1  Qt Creator会自动扫描代码中的TODO、FIXME等注释。
2
3  使用方法：
4  1. 点击左侧"问题"图标
5  2. 切换到"任务"标签页
6  3. 自动显示所有的TODO、FIXME、NOTE、XXX、BUG
7
8  代码中这样写：
9  // TODO：优化这个算法，现在是O(n²)的
10 // FIXME：这里有内存泄漏，需要修复
11 // NOTE：这里用了一个trick，因为Qt的bug
12 // BUG：玩家同时按跳跃和攻击会穿墙
13
14 任务面板功能：
15 - 按类型筛选
16 - 点击直接跳转到代码行
17 - 可以设置优先级 (@high @medium @low)
18 - 可以添加自定义任务
19 - 完成后标记为已完成
20
21 游戏开发中的用法：
22 - 想到优化点但现在不想做，写TODO
23 - 发现bug先记下来，写FIXME
24 - 重要设计决策旁边写NOTE
25 - 每周五整理一次任务面板
```

6.2.3 代码片段 (Snippets)

代码块

```
1  常用代码一键插入，提升编码效率。
2
3  创建自定义代码片段：
4  1. 工具 → 选项 → 文本编辑器 → 代码片段
5  2. 点击"添加"，输入触发词和代码内容
6  3. 保存后，编辑器里输入触发词，按Tab插入
7
8  游戏开发推荐片段：
9
10 1. 触发词：qclass (QObject子类)
11 class ClassName : public QObject
12 {
13     Q_OBJECT
```

```

14 public:
15     explicit ClassName(QObject *parent = nullptr);
16     ~ClassName();
17 signals:
18 public slots:
19 private:
20 };
21
22 2. 触发词: gitem (QGraphicsItem子类)
23 class ItemName : public QGraphicsItem
24 {
25 public:
26     ItemName(QGraphicsItem *parent = nullptr);
27     QRectF boundingRect() const override;
28     void paint(QPainter *painter, const QStyleOptionGraphicsItem *option,
29 QWidget *widget) override;
29 protected:
30     void advance(int phase) override;
31 };
32
33 3. 触发词: qprop (Qt属性)
34 Q_PROPERTY(Type name READ name WRITE setName NOTIFY nameChanged)
35 public:
36     Type name() const { return m_name; }
37     void setName(Type name) {
38         if (m_name == name) return;
39         m_name = name;
40         emit nameChanged();
41     }
42 signals:
43     void nameChanged();
44 private:
45     Type m_name;
46
47 4. 触发词: connect (信号槽连接)
48 connect(sender, &SenderClass::signal,
49         receiver, &ReceiverClass::slot);

```

6.3 UI设计工具

6.3.1 Qt Designer

代码块

1 Qt Designer是可视化UI设计工具，拖拖拽拽做界面。

2

3 游戏开发中的应用：

- 4 - 主菜单、设置界面、背包界面等复杂UI
- 5 - 比纯代码写UI快3-5倍
- 6 - 可以实时预览，不用反复编译运行

7

8 使用流程：

- 9 1. 项目右键 → 添加新文件 → Qt → Qt设计师界面类
- 10 2. 选择模板 (Widget、Dialog等)
- 11 3. 输入类名，自动生成.h/.cpp/.ui三个文件
- 12 4. 双击.ui文件，打开Designer
- 13 5. 拖拽控件到画布，布局，设置属性
- 14 6. 保存，编译时uic工具自动生成代码

15

16 设计技巧：

- 17 1. 用QFrame做容器
 - 18 - 复杂面板用QFrame当容器
 - 19 - 设置背景、边框、圆角
 - 20 - 里面再放其他控件
- 21
- 22 2. 自定义控件提升
 - 23 - 先拖拽一个QWidget或QFrame
 - 24 - 右键→提升为，输入自定义类名
 - 25 - Designer里是占位，运行时显示自定义控件
- 26
- 27 3. 样式表调试
 - 28 - Designer里直接写styleSheet，实时预览
 - 29 - 调好后再移到外部QSS文件
- 30
- 31 4. 布局管理
 - 32 - 善用水平布局、垂直布局、网格布局
 - 33 - 用spacer（弹簧）撑出空白
 - 34 - 布局比固定位置好，自适应不同分辨率

6.3.2 资源浏览器（Qt资源系统）

代码块

1 Qt资源系统（qrc）把资源编译进可执行文件。

2

3 创建资源文件：

- 4 1. 项目右键 → 添加新文件 → Qt → Qt资源文件
- 5 2. 输入文件名，比如resources.qrc
- 6 3. 自动生成resources.qrc

7

```
8  添加资源：
9  1. 双击resources.qrc，打开资源编辑器
10 2. 点击"添加" → "添加前缀"，输入前缀（比如/images）
11 3. 点击"添加" → "添加文件"，选择要添加的图片
12 4. 保存，资源就编译进exe了
13
14 代码中使用：
15 - 图片：QPixmap(":/images/player.png")
16 - QSS：background-image: url(":/images/background.png");
17 - 字体：QFontDatabase::addApplicationFont(":/fonts/pixel.ttf");
18
19 资源组织建议：
20 resources/
21 |— images/
22 |   |— ui/           # UI相关
23 |   |— characters/   # 角色
24 |   |— enemies/     # 敌人
25 |   |— tilesets/     # 地图瓦片
26 |   |— effects/      # 特效
27 |— sounds/           # 短音效
28 |— fonts/            # 字体
29 |— styles/           # QSS样式
30 |— data/             # 配置JSON
31
32 注意事项：
33 - qrc资源会增加exe体积
34 - 大的音频、关卡数据建议放外部文件
35 - 核心UI资源、小图标放qrc，发布时不容易丢
```

6.4 性能优化工具

6.4.1 帧率调试技巧

代码块

```
1  游戏帧率优化流程：
2
3  1. 确认问题
4     - 用CPU Profiler跑30秒
5     - 看火焰图，找到最耗时的函数
6     - 是渲染瓶颈还是逻辑瓶颈？
7
8  2. 渲染优化
9     - 同屏有多少个QGraphicsItem？
```

```
10     - 每个item的paint做了多少事?
11     - 启用图元缓存: setCacheMode
12     - 合并小图元, 减少绘制调用
13
14 3. 逻辑优化
15     - 碰撞检测是不是每帧检测所有物体?
16     - AI更新是不是太频繁?
17     - 有没有可以延后到下一帧的计算?
18
19 4. 内存优化
20     - 是不是频繁分配内存?
21     - 用对象池复用对象
22     - 用引用代替拷贝
23
24 5. 验证
25     - 优化后再跑一次Profiler
26     - 对比优化前后的数据
27     - 确保优化有效, 没有引入新问题
28
29 调试小技巧:
30     - 在游戏里显示FPS和帧时间
31     - 按F1显示/隐藏调试信息
32     - 可以显示各系统的耗时 (渲染、逻辑、AI等)
```

6.4.2 内存泄漏调试

代码块

```
1  游戏内存泄漏排查步骤:
2
3  1. 用Memcheck跑一遍完整流程
4     - 主菜单→进入游戏→玩一关→退出→再进入→退出
5     - 看有没有"确定泄漏"
6
7  2. 用GammaRay看对象数量
8     - 进入关卡前, 记一下QGraphicsItem数量
9     - 玩完一关退出后, 再看数量
10    - 如果数量增加了, 说明有对象没销毁
11
12 3. 对象池验证
13     - 敌人死亡时, 是delete了还是回收对象池?
14     - 在对象池的get/release函数里打日志
15     - 看获取和释放的数量是否匹配
16
17 常见泄漏点:
18 1. 特效对象: 播放完没销毁
```

```
19     - 解决: QTimer::singleShot延时删除, 或对象池
20
21 2. 临时QPixmap: 频繁创建没释放
22     - 解决: QPixmapCache缓存, 或预加载
23
24 3. 信号槽没断开: 对象销毁了信号还连着
25     - 解决: Qt::AutoConnection, 或销毁前断开
26
27 4. 子线程对象: 线程退出了对象没删
28     - 解决: moveToThread的对象, 线程quit后deleteLater
```

6.5 部署工具

6.5.1 一键打包脚本

代码块

```
1  写一个自动打包脚本, 发布版本时一键搞定。
2
3  Windows打包脚本示例 (batch) :
4  @echo off
5  echo =====
6  echo    星渊旅人 - Windows打包脚本
7  echo =====
8
9  set BUILD_DIR=build-StarAbyss-Desktop_Qt_6_10_1_MSVC2022_64bit-Release
10 set OUTPUT_DIR=Release/StarAbyss
11
12 echo [1/5] 清理输出目录...
13 if exist %OUTPUT_DIR% rmdir /s /q %OUTPUT_DIR%
14 mkdir %OUTPUT_DIR%
15
16 echo [2/5] 复制可执行文件...
17 copy %BUILD_DIR%\bin\StarAbyss.exe %OUTPUT_DIR%\
18
19 echo [3/5] 复制插件...
20 xcopy /e /i /y %BUILD_DIR%\plugins %OUTPUT_DIR%\plugins
21
22 echo [4/5] 运行windeployqt...
23 windeployqt %OUTPUT_DIR%\StarAbyss.exe --release --no-translations
24
25 echo [5/5] 复制资源文件...
26 xcopy /e /i /y resources %OUTPUT_DIR%\resources
27
```

```
28  echo.
29  echo 打包完成！输出目录：%OUTPUT_DIR%
30  echo =====
31  pause
32
33  类似的，macOS和Linux也可以写shell脚本。
34  这样每次发布版本，运行一下脚本就好了。
```

结语

恭喜你！看到这里，你已经对《星渊旅人》的完整开发流程有了全面的了解。

开发路线图回顾

阶段	时间	目标
第一阶段	第1-2个月	项目搭建、基础框架、主菜单
第二阶段	第3-8个月	三大核心系统、玩家、战斗、前5层
第三阶段	第9-18个月	剩余层级内容、剧情、优化、测试
第四阶段	第19-24个月	发布、MOD支持、DLC规划

单人开发建议

- 1. **先做最小可玩版本**：先把核心循环做出来，能玩就行，再慢慢加内容
- 2. **善用工具**：Qt Creator的工具很多，充分利用可以节省大量时间
- 3. **定期测试**：不要攒到最后才测，边做边测
- 4. **控制范围**：18层听起来很酷，但单人开发可以先做5层，后续DLC补充
- 5. **保持节奏**：每天固定时间开发，坚持比突击更有效
- 6. **多做备份**：Git提交要勤，云端备份不能少

后续学习资源

- Qt官方文档：<https://doc.qt.io/>
- Graphics View教程：Qt自带的Examples里有很多
- 游戏开发：《游戏编程模式》（强烈推荐）
- 像素美术：可以用Aseprite，很多教程

- 游戏设计：多玩同类游戏，分析设计

最后

游戏开发是一段漫长但充满乐趣的旅程。24个月听起来很长，但当你看着自己的游戏一天天成型，那种成就感是无与伦比的。

记住：

- **完成比完美更重要**：先做出来，再优化
- **玩家反馈是最好的老师**：早点放出试玩版，收集反馈
- **享受过程**：做游戏本身就是一件很快乐的事

祝你开发顺利，期待在Steam上看到《星渊旅人》！

文档版本：v1.0

最后更新：2026年6月

（注：文档部分内容可能由 AI 生成）